

THE COMPLETE PRACTITIONER'S COURSE

Big Data Analytics & Machine Learning

From SQL, Excel and SPSS to Spark, Deep Learning
and Production AI

Tatenda Makuvaza

"The Big Data Analyst"

First Edition · 2026

Table of Contents

Front Matter	4
The Extended Curriculum: Special Topics and the Craft of Teaching	8
Chapter 49: Building Data Products — Dashboards, Apps, and APIs	9
Chapter 50: Teaching Analytics — A Course Design and Instructor's Guide	14
Analytics Across Industries: Domain Playbooks	21
Chapter 51: Retail and E-Commerce — The Ndineka Fashion House Playbook	22
Chapter 52: Marketing and Customer Analytics — The Zuva Mobile Playbook	27
Chapter 53: Financial Services — The ZuvaPay Playbook	31
Chapter 54: Healthcare Analytics — The Batanai Health Trust Playbook	35
Chapter 55: Agriculture and Food Security — The Chikafu Foods and NGA Playbook	39
Chapter 56: Energy Analytics — The Sable Energy Playbook	43
Chapter 57: Government and Public Sector Analytics — The Harare Metro Playbook	47
Chapter 58: Manufacturing Analytics — The Sabvura Industries Playbook	51
Chapter 59: Transport and Logistics — The Kunaka Logistics Playbook	56
Chapter 60: Media and Sport Analytics — The Nhaka FC and Kumba Playbook	60
The Frontier: Advanced Methods and Emerging Practice	64
Chapter 61: Causal Inference — The Counterfactual Discipline	65
Chapter 62: Geospatial Analytics — Where the Data Lives	69
Chapter 63: Survival Analysis — Time Until the Event	73
Chapter 64: Advanced Forecasting — Hierarchies, Honest Intervals, and Intermittent Demand	77
Chapter 65: Low-Resource Language Analytics — Kumba and the Languages the Internet Forgot	81
Chapter 66: Simulation I — Queueing and the Utilisation Cliff	86
Chapter 67: Simulation II and Experimentation at Scale — Monte Carlo, Bandits, and CUPED	90

Chapter 68: Privacy Engineering — Sharing Data Without Sharing People 94

Chapter 69: Analytics Engineering — The Warehouse as a Product 98

Chapter 70: Staying Current — A Reading Discipline and Ten Predictions 102

The Practitioner's Path: Projects, Clients and the Craft of Delivery . 106

Chapter 71: The Anatomy of an Analytics Project 107

Chapter 72: Scoping and the Options Paper 111

Chapter 73: Working with Stakeholders — The Question Ladder 115

Chapter 74: Storytelling for Decision-Makers 119

Chapter 75: Documentation That Outlives You 123

Chapter 76: Audits and Due Diligence — Ten Questions 127

Chapter 77: The Constrained Practitioner — The \$200 Stack and the Offline-First World¹³¹

Chapter 78: Your First Ninety Days — Quick Wins and the Trust Ledger 135

Chapter 79: Building a Practice — The Service Catalogue 139

Chapter 80: The Business of the Craft — The Value Ledger, Productisation, and
Compounding Practice 143

Reference Section 148

Appendix E: Datasets and Resources 149

Appendix F: The Soulfya's Practice-Dataset Generator 153

Appendix I: Instructor Slide Kit 157

Appendix J: Extended Case Portfolio 163

About this Edition 171

Front Matter

About This Edition

Big Data Analytics and Machine Learning — The Complete Practitioner's Course: From SQL, Excel and SPSS to Spark, Deep Learning and Production AI

The Extended Practitioner's Edition — Parts IX-XII: the extended curriculum, the domain playbooks, the frontier, and the delivery craft

Tatenda Makuvaza — "The Big Data Analyst"

First Edition, 2026

How This Book Came to Be

I completed a six-module data analytics course — SQL, Excel, Python, Power BI, Stata and SPSS — and then faced the question every graduate faces: *what now?* The tools were in my hands, but the path from "I know some SQL and I can build a dashboard" to "I can run an analytics practice, teach a class, and ship a machine-learning system people trust" was nowhere written down.

This book is the answer, written in two volumes. The **foundation volume** (Parts I-VIII) builds the grammar: questions, data engineering, statistics, visualization, machine learning, big-data platforms, and the responsible operation of the whole. **This volume — the Extended Practitioner's Edition — begins where a solid foundations course ends.** It assumes you already carry a toolkit (any toolkit: the six modules above, a degree, a bootcamp, hard-won on-the-job training) and takes you the rest of the way: the special topics and the craft of teaching (Part IX), ten industry playbooks (Part X), the frontier methods that extend the working core (Part XI), and the delivery craft of the practising analyst (Part XII).

Every chapter is built on the same bridge: **From Your Toolkit** callouts connect each new idea back to the six foundation modules, because you never learn analytics by abandoning what you know — you learn it by extending it. And every concept lives in one running world: **Soulfya's Restaurant Group**, a fictional three-outlet restaurant chain in Harare whose data (generated by Appendix F, with every pattern planted and documented) carries the whole book — together with the ten companies of Part X's domain playbooks, who share its world.

Who This Book Is For

- **Graduates of foundation courses** who know the tools and need the disciplines that turn tools into practice.
- **Working analysts** who want to move from reports to models, from models to systems, and from systems to delivered engagements.
- **Instructors** building the second semester or the honours year: this volume is sequenced as one (Chapters 49–80), with a slide kit (Appendix I), a case portfolio for seminars (Appendix J), and a complete instructor's guide in Chapter 50.

- **Self-taught practitioners** who want their advanced knowledge to have the same foundations a degree would have given them.

The Arc of the Course

The complete course design is twelve parts; the foundation volume carries Parts I-VIII, and this edition carries the four that follow:

Part	Question it answers	What it holds	Volume
I — Foundations	What is analytics, and when is data actually big?	Questions, maturity, the Vs, data types, your six tools	Foundati on
II — Data Engineering	Where does trustworthy data come from?	Grain, joins, pipelines, warehouses, cleaning, quality	Foundati on
III — Math and Statistics	How do I argue with numbers honestly?	Descriptives, probability, sampling, testing, regression	Foundati on
IV — Visualization and BI	How do I make numbers change minds?	Chart craft, metric trees, dashboards, storytelling	Foundati on
V — Machine Learning	How do I make data predict?	Framing, leakage, baselines, models, evaluation, ethics	Foundati on
VI — Advanced ML	What lies beyond the first model?	Clustering, recommenders, NLP, deep learning, LLMs, explanation	Foundati on
VII — Big Data and Cloud	What changes at scale?	The decision test, DuckDB, Spark, streaming, cloud economics	Foundati on
VIII — Insight to Impact	How do we run this responsibly, forever?	Governance, MLOps, monitoring, ethics, careers	Foundati on
IX — Extended Curriculum	How do we go further — and teach it?	Anomaly detection, optimization, graphs, data products, teaching	This edition
X — Domain Playbooks	How is analytics actually used, industry by industry?	Ten sector playbooks with metrics, methods and running cases	This edition
XI — The Frontier	What lies beyond the working core?	Causality, space, time, language, simulation, privacy, staying current	This edition
XII — The Practitioner's Path	How is analytics actually delivered?	Projects, engagements, storytelling, documentation, audits, constraint, onboarding, the business of the craft	This edition

This edition contains **Chapters 49-80 in four parts, plus Appendices E, F, I and J**. Each part is coherent on its own, and Part X's playbooks in particular can be read in any order — though Part XII lands hardest after the frontier of Part XI. Where chapters cite "Chapter 22" or "Part II", they point to the foundation volume's treatment of the foundations (leakage, grain, baselines); the citation is given so you know where the idea's home is, not because you cannot proceed without it.

How to Read This Book

Three habits, sustained across every chapter, are worth more than any tool in it:

1. **Ask the decision question before the data question.** Every chapter's first move is "what will someone do differently?"

2. **Name a baseline before praising a result.** Seasonal-naive, majority class, last year: if you cannot beat the cheapest alternative, use the alternative.
3. **Write for a reader who is not you.** Memos, model cards, logs: the artefact is the analysis.

Read with a lab open. Every chapter ends with a Practice Lab, and the labs are load-bearing: the prose is scaffolding for the labs, not the other way around. The data comes from Appendix F's generator — planted patterns, documented — or from the public datasets of Appendix E.

A Teaching Plan for This Edition

This volume runs naturally as a second semester, an honours year, or a professional development programme:

Weeks	Chapters	Mode	Output
1–2	49–50	Seminar + teaching practice	One shipped data product; one live-coded lesson
3–7	51–60	Case-method seminars (one playbook per week)	A filled playbook canvas for a local organisation, weekly
8–12	61–70	One chapter, one paper, one replication (per week)	The frontier replication portfolio
13–14	71–80	Role-play and role-play again	The full engagement role-play: scoping, hostile questions, pricing, a refusal

The showcase closes the course: capstones presented to a real audience, with the Appendix J cases as the seminar corpus and Appendix I's twelve part-decks as the lecture skeletons.

The Appendices

Four appendices back this edition: **E** datasets and resources (public data, tools, and the reading list); **F** the Soulfya's practice-dataset generator (the script that fabricates the running case's universe, with every planted pattern documented); **I** the instructor slide kit (twelve starter decks, one per part); and **J** the extended case portfolio (five end-to-end cases with instructor numbers and traps). A companion reference section — Python and SQL primers, an algorithm quick-reference, a glossary, worked solutions, and the assessment bank — accompanies the foundation volume.

A Note on the Running Case

Soulfya's Restaurant Group is fictional, and so is everyone in it — but nothing about its data is arbitrary. Its outlets (Avondale, Borrowdale, Bulawayo), its Soul Circle loyalty programme, its delivery channel and its reviews carry every pattern this book teaches: weekly seasonality, payday spikes, promo lifts with their selection biases, outlet differences, a genuine churn signal with a survival shape. Its world extends through Part X: the fashion house that stocks it, the telco that connects it, the logistics firm that delivers its cold chain, the water authority that supplies it — one fictional economy, planted and documented, from the first chapter to the last. If the chain feels real by Chapter 51, the book is working.

— *Tatenda Makuva*, "The Big Data Analyst", Harare, 2026

PART IX

The Extended Curriculum: Special Topics and the Craft of Teaching

Chapters 46 – 50

Chapter 49: Building Data Products — Dashboards, Apps, and APIs

Part IX — The Extended Curriculum: Special Topics and the Craft of Teaching

"The last mile of analytics is a product, not a PDF."

In this chapter you will learn

- The product ladder: report → dashboard → app → API → automated decision service — and when to climb each rung.
- A working Streamlit app in one file: the churn explorer.
- A production-shape scoring API with FastAPI.
- The product crafts: versioning, logging, documentation, authentication.
- Deployment realities: containers, CI, and what to run where.
- When *not* to build a product — the discipline of the last rung.

49.1 The Product Ladder

Analysis creates understanding; a product *delivers* it, repeatedly, to someone who was not in the room. Each rung of the ladder adds interactivity, engineering, and maintenance — climb only as far as the decision demands:

Ru ng	Form	Interaction	Maintenance	Soulfya's example
1	Report / memo	None	None	Monthly performance write-up
2	Dashboard (Chapter 18–19)	Filter and drill	Refresh, model upkeep	"Are we healthy?" page
3	Notebook-as-app	Parameter play	Low	Scenario planner for the founder
4	Web app	Full interaction	Real	Churn campaign workbench
5	API	Machine-to-machine	Real + latency SLAs	Live delivery-ETA scoring
6	Automated decision service	Acts on its own	Everything in Part VIII's MLOps	Dynamic prep recommendations

The professional error is skipping rungs: teams build a web app (rung 4) for an audience that needed a well-written memo (rung 1), or an API (rung 5) for a question asked quarterly. The other professional error is *camping* at rung 1 forever while a weekly, repeated, human-heavy ritual begs for rung 2.

49.2 Rung 3-4 in One File: Streamlit

Streamlit turns a Python script into a web app: the script reruns top-to-bottom on every interaction, widgets become function calls, and the whole framework fits in a morning. The churn campaign workbench:

```
import streamlit as st
import pandas as pd
import joblib

st.set_page_config(page_title="Churn Explorer", layout="wide")
st.title("Soul Circle Churn Explorer")

@st.cache_data
def load_scores():
    scores = pd.read_csv("member_scores.csv", parse_dates=["score_date"])
    scores["segment"] = scores["segment"].fillna("unknown")
    return scores

@st.cache_resource
def load_model():
    return joblib.load("churn_pipeline.joblib")

scores = load_scores()
model = load_model()

with st.sidebar:
    outlets = st.multiselect("Outlets", sorted(scores["outlet_id"].unique()))
    min_risk = st.slider("Minimum risk score", 0.0, 1.0, 0.5)
    top_n = st.number_input("Campaign size", 50, 2000, 200)
    run = st.button("Build campaign")

view = scores[scores["risk"] >= min_risk]
if outlets:
    view = view[view["outlet_id"].isin(outlets)]

col1, col2, col3 = st.columns(3)
col1.metric("Members above threshold", f"{len(view):,}")
col2.metric("Churners in view (last 60d)", f"{view['churned'].sum():,}")
col3.metric("Est. value of campaign", f"${view.nlargest(top_n,
    'risk')['expected_value'].sum():.0f}")

st.scatter_chart(view, x="days_since_visit", y="risk", color="segment", height=380)

if run:
    campaign = view.nlargest(top_n, "risk")
    campaign.to_csv("this_weeks_campaign.csv", index=False)
    st.success(f"Campaign built: {len(campaign)} members, "
        f"{campaign['member_id'].nunique()} unique, CSV ready for download.")
    st.data_editor(campaign[["member_id", "risk", "days_since_visit", "segment"]])
```

Fifteen minutes of work produces what a month of email round-trips never did: the retention team explores risk thresholds themselves, sees the expected value move, and pulls the campaign list when they are satisfied. The `@st.cache_data` / `@st.cache_resource` decorators are the entire performance strategy — data loads once, the model loads once.

The honest limits: Streamlit apps are internal tools. Authentication is basic, multi-user state is awkward, and anything customer-facing eventually wants a real frontend — by which point the app has already paid for itself as the prototype that proved the need.

49.3 Rung 5: A Scoring API with FastAPI

When another system needs your model — the delivery app's backend, the POS, a partner — you serve it as an API. FastAPI is the modern standard: type hints become validation, and documentation is generated automatically.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
import joblib, pandas as pd

app = FastAPI(title="Soulfyas Scoring API", version="1.2.0")
model = joblib.load("churn_pipeline.joblib")

class MemberFeatures(BaseModel):
    days_since_visit: float = Field(..., ge=0, le=730)
    visits_90d: int = Field(..., ge=0)
    avg_basket: float = Field(..., ge=0)
    delivery_share: float = Field(..., ge=0, le=1)
    complaint_filed: int = Field(..., ge=0, le=1)

class ChurnScore(BaseModel):
    risk: float
    model_version: str

@app.post("/v1/churn", response_model=ChurnScore)
def score(m: MemberFeatures):
    X = pd.DataFrame([m.model_dump()])
    risk = float(model.predict_proba(X)[0, 1])
    return ChurnScore(risk=risk, model_version="1.2.0")

# run: uvicorn scoring_api:app --host 0.0.0.0 --port 8000
```

Read what the fifteen lines include: input *validation* (impossible values rejected with a 422 before they reach the model), a *versioned route* (`/v1/` — so v2 can ship beside v1), and the model version in every response — Chapter 43's traceability requirements, implemented as a side effect of good design. Swagger documentation appears at `/docs` for free. In production, add authentication (an API key header at minimum), structured logging of inputs and scores (the drift dataset of Chapter 43), and latency monitoring.

49.4 Product Craft: The Four Habits

1. **Version everything together** — code (git), model artefact (registry), API route (`/v1/`), and the version echoed in responses. "Which model produced this score?" must be answerable from the score itself.
2. **Log like you will be debugged** — every request (timestamp, features, score, latency, caller), because your monitoring data *is* your log, and Chapter 43's drift detection will read exactly this table.
3. **Document the contract** — the API's generated docs plus one human page: what the score means, its calibration, its known weaknesses (Chapter 29's model card, linked from the product).
4. **Degrade gracefully** — a model failure should not take down the caller: fall back to a default score, a cached result, or an explicit "unavailable" that the caller is designed to handle. Products fail; *systems* degrade.

49.5 Deployment, Minimally

The container that runs the same everywhere (Chapter 43's one-sentence Docker):

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY scoring_api.py churn_pipeline.joblib .
CMD ["uvicorn", "scoring_api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Where it runs is a Chapter 41 decision: a small VM, a container service, or a platform's managed endpoint — the artefact does not change. The CI habit from Chapter 43 applies verbatim: every push runs the model smoke test (fit on a fixed toy sample, score a fixed row, assert the score within tolerance) before anything deploys.

49.6 The Discipline of the Last Rung

The rung you choose is a maintenance contract. A memo costs nothing after Thursday; a dashboard costs a refresh schedule and a model; an API costs latency, uptime, and a rotation of on-call analysts. Before climbing, ask Chapter 2's question in product form: *how many people, how often, making which decision?* Soulfya's churn workbench served three people weekly — a Streamlit app, aggressively unglamorous, was the correct ceiling. The delivery-ETA scorer served thousands of app sessions daily — a containerised API earned itself. Both answers are right; the sin is not knowing which you are building.

Teaching Tip — Ship week: make the course capstone literally a shipped artefact — a live URL (Streamlit sharing or any free host) or a running container — graded with a rubric of product habits: does it state its model version? Does it degrade gracefully? Would a stranger understand the front page in thirty seconds? Students who have *deployed something* interview differently: they have war stories about CORS, caching, and cold starts, and war stories are what "experience" means.

Key Takeaways

- Products are rungs, not a summit: report → dashboard → app → API → decision service; climb to match the decision's frequency and audience, no further.
- Streamlit turns analysis into an interactive workbench in an afternoon — the highest value-per-line tool in this book for internal audiences.
- FastAPI gives validation, versioned routes, and generated documentation for free — and those defaults implement Chapter 43's traceability requirements.
- The four product habits: versioned-together artefacts, request logging (your drift dataset), documented contracts, graceful degradation.
- Deployment is a six-line container plus a CI smoke test; where it runs is a cloud economics decision.
- The last rung is a maintenance contract — size it to the audience, and let the boring tool win when the boring tool suffices.

Practice Lab

1. Build the churn explorer on your Part V scores; add a segment filter and a "download campaign CSV" button; deploy it (Streamlit sharing or a container) and send the link to a non-analyst for a critique.
2. Wrap your trained pipeline in the FastAPI scorer; call it with curl; then send an invalid feature value and read the 422 — write two sentences on why this protected your model.
3. Add `/v2/churn` with a second model; route 10% of traffic to it via a simple flag; log which version scored each request; you have built a minimal champion-challenger.
4. Break it on purpose: corrupt the model file and restart; implement the graceful fallback (default score + warning header); document the behaviour in the API's human page.
5. Write the rung justification memo for three artefacts your organisation needs — one per rung 2, 3, and 5 — with audience, frequency, and maintenance cost for each.
6. The ship-week rubric: score your own capstone artefact against Section 49.4's four habits; fix the lowest score before calling any project done.

Further Reading

- Streamlit and FastAPI official documentation (both free, both excellent)
- *Designing Machine Learning Systems* — Huyen, chapters 7–8 (serving and products)
- Chapter 43 for the monitoring this chapter's logs feed.

Chapter 50: Teaching Analytics — A Course Design and Instructor's Guide

Part IX — The Extended Curriculum: Special Topics and the Craft of Teaching

"If you cannot teach it simply, you do not understand it well enough. If you can teach it, you own it forever."

In this chapter you will learn

- How to turn this book into a 14-week course — sequencing, pacing, assessment, and outcomes.
- The lecture-lab-feedback rhythm that keeps a technical course alive.
- How to grade analytics work honestly: rubrics for code, memos, and presentations.
- How to run studio sessions, project reviews, and a capstone showcase.
- How to teach the tools you already know — SQL, Excel, Python, Power BI, Stata, SPSS — as one discipline, not six.
- How to keep your own teaching current without drowning in tools.

50.1 Why Teaching Is the Best Way to Learn

The Feynman technique is famous for a reason. When you teach a concept, you are forced to retrieve it, organise it, confront its edge cases, and answer for its assumptions. Every "dumb question" from a student is a free audit of your understanding.

From Your Toolkit — SPSS: You have already taught, even if you have not called it that. Walking a manager through a crosstab in SPSS, explaining a chi-square result, defending a regression table in a meeting — these are teaching events. This chapter scales that instinct into a discipline.

This chapter is written for three audiences at once: the instructor building a formal course, the analyst running internal upskilling, and the self-learner who needs a syllabus. The mechanics are the same; only the venue changes.

50.2 Course Outcomes: Design Backwards

Before pacing or slides, answer: **what will students be able to do at the end?** Write outcomes as observable behaviours, not topics covered.

Strong outcomes for this course:

1. Frame a business question as an analytics question with a stated decision, metric, and baseline.
2. Build a trustworthy dataset from messy sources — joins, cleaning, validation — and document its lineage.

3. Choose and defend a statistical test or model, including its assumptions and limitations.
4. Build a dashboard or data product that a non-analyst can use unaided.
5. Communicate findings as a decision recommendation with quantified uncertainty.
6. Audit their own work: leakage checks, sensitivity analyses, and honest error accounting.

Every lecture, lab, and assessment maps to at least one outcome. If a session supports none, cut it. That is the discipline of backwards design: outcomes → assessments → activities → content. Most courses are built in the opposite direction and feel like it.

50.3 Sequencing: Follow the Book's Arc

The course follows the book's structure, which was designed as a syllabus:

- **Part I (weeks 1-2):** Foundations. Questions before data; data maturity; the Vs; data types; the six-tool bridge.
- **Part II (weeks 3-5):** Data engineering. Joins, grain, pipelines, warehousing, pandas-to-SQL, cleaning, quality.
- **Part III (weeks 6-8):** Statistics. Descriptives, probability, testing, sampling, regression.
- **Part IV (weeks 8-9):** Visualization and BI. Craft, metric trees, dashboards, storytelling.
- **Part V (weeks 10-12):** Machine learning foundations. Framing, leakage, baselines, models, evaluation, ethics.
- **Part VI (weeks 13-14):** Advanced ML — clustering, recommenders, NLP, deep learning, LLMs — taught selectively for a first course.

A 14-week term covers Parts I-VI in depth and Parts VII-VIII by immersion: governance, Spark, and careers slot into the capstone fortnight, where they belong (you teach pipelines best when students are suffering a broken one). In a two-semester design, the second semester runs Part IX's special topics (Chapters 46-50), Part X's ten industry playbooks as case-method seminars, Part XI's frontier chapters as honours reading, and Part XII's delivery craft as role-play. Teach eighty chapter/appendix units as a full-year course: the arc spans Parts I-XII, from first principles to the business of the craft, and it is coherent end to end.

Teaching Tip — The bridge is the course. Students arrive knowing tools — SPSS from a stats service course, Excel from an internship, SQL from a bootcamp. The course's job is to connect those tools into one discipline. Never say "forget Excel"; say "here is what Excel was doing all along, and here is where it runs out." Respect for prior knowledge is not politeness; it is pedagogy.

50.4 The Weekly Rhythm: Lecture, Lab, Feedback

A week that works:

Slot	Length	Activity	Failure mode it prevents
Lecture	60–90 min	Concepts, live demos, toolkit bridges	Pure theory with no grip
Lab	90–120 min	Students work the chapter's Practice Lab in pairs	Watching demos without doing
Feedback	15–30 min	Instructor reviews 2–3 student artefacts live	Students repeating errors all term
Memo	Homework	One-page write-up of the lab's finding	Analysis that cannot be communicated

The feedback slot is the engine. Reviewing real student work publicly — kindly, specifically — teaches more than any slide, because every student recognises their own errors in someone else's artefact. Rotate whose work is reviewed so everyone is showcased and everyone is polished.

Teaching Tip — Live coding beats slides. Type the SQL. Make the syntax error. Debug it. Students learn that errors are the job, not the exception. Prepare a backup notebook for when the live demo truly dies, but let small failures happen on purpose: the recovery is the lesson.

50.5 Pacing: Where Students Struggle

The known pain points, and the responses that work:

Concept	The struggle	The response
Joins and grain (Ch. 6–8)	Fan-out feels arbitrary until it bites	The duplicated-orders lab: let them find the 3x revenue inflation themselves
Hypothesis testing (Ch. 14)	p-value as ritual, not reasoning	The coin-flip simulation: see 5% of fair coins "significant" at n=20
Leakage (Ch. 22)	Everyone nods, everyone leaks	The leaky-feature audit: give them a dataset with a planted leak
Baselines (Ch. 23)	Skipping straight to XGBoost	The league table: naive baseline first, model second, side by side
Interpretation (Ch. 25, 33)	Reading coefficients as causes	The confounded coffee example: same data, two stories

Budget extra weeks for joins and testing; steal them from dashboards and deep learning if you must. A student who can grain a table and read a p-value honestly can learn Plotly next week. The reverse is not true.

The second semester's pain points are different: case-method seminars (Part X) struggle until students stop summarising and start deciding ("what would you do Monday morning?"); frontier chapters (Part XI) struggle when taught as surveys — one chapter, one paper, one replication keeps depth honest; and the delivery role-plays (Part XII) feel theatrical until the first hostile-question round lands, after which nobody mocks them again.

50.6 Assessment: Grade the Artefacts

Grade what analysts produce, not what exams can measure. A scheme that has survived many cohorts:

Component	Weight	Artefact	Graded on
Weekly memos	25 %	One-page lab write-up	Decision framing, honest uncertainty, clarity
Midterm project	25 %	Cleaned dataset + EDA memo	Lineage, validation, quality checks
Capstone	40 %	End-to-end analysis + presentation	Outcome rubric below
Participation	10 %	Labs, reviews, showcase	Presence and contribution

Draw viva questions from each part's Practice Labs and Key Takeaways, and use them where students defend their capstone choices aloud. Oral defence is the only assessment that reliably catches borrowed work and missing understanding simultaneously.

The capstone rubric, shared with students on day one:

Criterion	Weight	Excellent looks like
Framing	20%	Decision question stated; stakeholder named; success metric chosen and defended
Data craft	20%	Lineage documented; validation gates; no leakage
Analysis	25%	Baseline → model → comparison; assumptions checked; sensitivity run
Communication	25%	Memo a manager reads; charts follow Ch. 17; executive summary passes the stranger test
Reproducibility	10%	A classmate can rerun it from the repo

For honours/second-semester students, extend the capstone with one Part X playbook applied to a local organisation, one Part XI method replicated on public data, or one Part XII engagement role-play — one extension, done deeply, beats three done thinly.

50.7 Running the Studio: Reviews and Showcases

Project reviews. At weeks 4, 8, and 12, students present work-in-progress for ten minutes and receive structured critique: two strengths, two risks, one recommendation. This mirrors real practice (Ch. 71's project reviews) and teaches the hardest professional skill — receiving critique without defending.

The showcase. The final session is public: students present capstones to a real audience — managers from local businesses, other faculty, family. Real audiences raise the stakes productively; students polish slides that would have stayed rough for classmates. Invite a working analyst to give one sentence of feedback per presenter: practitioners' praise is remembered for years.

From Your Toolkit — Power BI: the showcase is a dashboard demo, so coach it as one: one screen, one message, thirty seconds to the point. Students who have presented a Power BI report to a manager already know the genre; name that knowledge.

Slide kits for every part. Appendix I holds a starter slide deck for each of the twelve parts — twelve decks, 8–12 slides each, with the one diagram that anchors each part: the analytics question ladder (Part I), the star schema (Part II), the testing logic tree (Part III), the chart-choice matrix (Part IV), the modelling loop (Part V), the represent-learn-predict map (Part VI), the scale decision test (Part VII), the insight-to-impact chain (Part VIII), the teach-back cycle (Part IX), the playbook canvas (Part X), the frontier map (Part XI), and the engagement arc (Part XII). Adapt them; do not merely display them. If you prepare eighty chapter maps from the deck skeletons you will never wonder what to say.

50.8 Teaching the Tools as One Discipline

The six modules of your foundation are not six subjects. Taught as one discipline, they become a grammar:

- **SQL** is the language of *what happened* — selection, aggregation, joins, windows.
- **Excel** is the language of *business arithmetic* — the model of the business everyone shares.
- **Python** is the language of *anything, at any scale* — the glue that connects every other tool.
- **Power BI** is the language of *what everyone sees* — the interface between analysis and organisation.
- **Stata** is the language of *econometric caution* — the tool that asks "identified by what?"
- **SPSS** is the language of *classical statistics* — the hypothesis-testing ritual, done properly.

When you teach joins, show them in SQL, then Excel-VLOOKUP, then pandas-merge, then the Power BI relationship model — same concept, four syntaxes. When you teach testing, show SPSS's crosstab, Stata's `prtest`, and the Python simulation of the same question. Transfer, not coverage: students who see one idea in four tools stop believing tools are knowledge.

Teaching Tip — Retire the syllabus, not yourself. Tools churn; the grammar does not. When a student asks about a tool this book does not cover, the honest answer is: "Here is the grammar it implements; find which parts it changes." Model that answer and you have taught Chapter 47's final lesson — how to stay current — as a living practice.

50.9 Your Own Development as an Instructor

Three habits keep a course alive:

1. **Teach one new thing every term** — a new dataset, a new method, a new failure story from practice. The course that never changes is the course that dies.

2. **Keep a teaching log** — what worked, what died, the questions that stumped you. Chapter 45's career habit, applied to the classroom.
3. **Bring practice in** — guest practitioners, your own consulting war stories, the Appendices E and J case portfolio. Students can smell the difference between a course about analytics and a course from analytics.

50.10 A Closing Note from the Author

If you have taught through this book, you have given eighty chapters of grammar, practice, and judgement to people who will use them in ways you will never see. That is the compounding of teaching: every analyst you train multiplies the honesty and the craft of every decision they touch. This chapter closes the taught syllabus, but not the book: Part X walks the industries where your students will work, Part XI pushes to the frontier they will extend, and Part XII teaches the delivery craft they will practise. Teach on.

— *Tatenda Makuvaza, "The Big Data Analyst"*

Key Takeaways

- Design backwards: outcomes first, assessments second, activities third. If a session maps to no outcome, cut it.
- The weekly rhythm is lecture → lab → live feedback → memo; the feedback slot is the engine.
- Grade artefacts, not exams: memos, projects, defences. The viva catches what written work hides.
- Students struggle predictably — at joins, testing, leakage, baselines, and interpretation. Budget time there, and let the pain points teach.
- Teach the six tools as one grammar, shown in four syntaxes. Transfer, not coverage.
- Teach one new thing every term, keep a log, bring practice in — a course is a product with users, and it deserves the same craft.

Practice Lab

1. Write the six course outcomes for your own context (your team, your classroom, your self-study) as observable behaviours — then audit this book's chapters against them: which chapters serve which outcomes?
2. Design your 14-week pacing grid: chapters to weeks, labs to chapters, one review week, one showcase week. Mark the two weeks you expect joins and hypothesis testing to eat.
3. Draft the capstone rubric adapted to your audience, and write the one-page handout version students receive on day one.
4. Run one studio review — a colleague or classmate presents work-in-progress for ten minutes; you deliver two strengths, two risks, one recommendation; write down what the format caught that informal feedback would not.

5. Prepare a ten-minute live-coded lesson on one concept from this book, deliberately including one recoverable error; teach it, then log what the error taught.
6. Take the Part XII hostile-question protocol and run it against your own capstone: present to a friend who must ask the three hardest questions from Chapter 74. Survive, then repeat.

Further Reading

- *Understanding by Design* — Wiggins and McTighe (backwards design)
- *Teaching Naked* — José Antonio Bowen (technology outside the classroom)
- Appendix I (slide kits), Appendix J (case portfolio)

PART X

Analytics Across Industries: Domain Playbooks

Chapters 51 - 60

Chapter 51: Retail and E-Commerce — The Ndineka Fashion House Playbook

Part X — Analytics Across Industries: Domain Playbooks

"Retail analytics is one question in four costumes: what to stock, at what price, for whom, and how much is already dead."

In this chapter you will learn

- The retail playbook canvas: six slots that organise every retail analytics problem.
- Market-basket analysis: affinity, lift, and the difference between "bought together" and "buys more because".
- Markdown optimisation and price elasticity, honestly bounded.
- Customer lifetime value with the RFM bridge from your toolkit.
- The failure modes: the average customer, the dead-stock spiral, promo dependence.

51.1 The Retail Question

Ndineka Fashion House is a fictional Harare retailer: four stores, a growing online channel, 30,000 SKUs of seasonal clothing, and a founder — Ndineka herself — who describes her business in one sentence: "I buy six months before I sell, and I am wrong on purpose because being exactly right is impossible." That sentence is the entire retail problem. Inventory is frozen cash; the buy decision is made under uncertainty about weather, taste, and the economy; and the analyst's job is to shrink the wrongness, not to pretend it away.

Every retail analytics question reduces to one of four: **what to stock** (assortment), **at what price** (pricing and markdown), **for whom** (customers), and **how much is already dead** (inventory health). The playbook canvas organises them:

Slot	Ndineka's answer
Core question	Which lines, at what price, for which customers, bought how deep?
Unit of analysis	The SKU × store × week (and the customer, for the CLV layer)
Key metrics	Sell-through, gross margin, weeks of cover, dead-stock value, CLV
Data reality	POS transactions, seasonal buys, returns, a loyalty app, no true cost of returns
First project	The dead-stock ledger: name it, age it, price it
Failure mode	Promo dependence — margin spent to buy revenue

51.2 The Dead-Stock Ledger First

The highest-return first project in retail is almost always an inventory health report, because it requires no modelling and funds everything that follows. The discipline is aging: every SKU-store row gets weeks-on-shelf, sell-through rate, and a classification.

```
SELECT sku, store,
       weeks_on_shelf,
       units_sold / NULLIF(units_received, 0) AS sell_through,
       CASE
         WHEN weeks_on_shelf >= 26 AND sell_through < 0.30 THEN 'dead'
         WHEN weeks_on_shelf >= 12 AND sell_through < 0.50 THEN 'slow'
         ELSE 'healthy'
       END AS stock_class,
       retail_price * units_on_hand AS frozen_value
FROM inventory_snapshot
ORDER BY frozen_value DESC;
```

The `frozen_value` column is the meeting: when the leadership team sees that 22% of working capital is wearing a "dead" tag, markdown optimisation stops being a data science conversation and becomes a Tuesday agenda item.

51.3 Market-Basket Analysis

Baskets are the atom of retail behaviour. The classic measures, on the Soulfya's data you already know from Part II:

- **Support** — the share of baskets containing the pair. Rare pairs are noise.
- **Confidence** — given the anchor, how often the companion appears.
- **Lift** — confidence divided by the companion's base rate. Lift above 1 means the anchor *changes* the companion's probability.

```
pairs = (baskets.groupby(["anchor", "companion"])["basket_id"].nunique()
        .rename("both").reset_index())
pairs = pairs.merge(anchor_totals, on="anchor").merge(comp_totals, on="companion")
pairs["confidence"] = pairs["both"] / pairs["anchor_total"]
pairs["lift"] = pairs["confidence"] / (pairs["companion_total"] / n_baskets)
strong = pairs[(pairs["support"] >= 0.005) & (pairs["lift"] >= 1.5)]
```

The trap is reading lift as causation. Sadza-and-soft-drink has lift 1.4 because dinner is dinner — the pair reflects a meal occasion, not a promotion opportunity. The honest reading of basket analysis is *hypothesis generation*: every strong pair is a candidate for an experiment (Chapter 61's toolkit), never a layout change on its own.

51.4 Markdown and Price Elasticity

Markdown is not discounting; it is *managing the decay curve of a perishable asset* — fashion is perishable milk with better lighting. The analyst's contribution is elasticity bounded honestly. Fit demand (units sold) against price across comparable lines and weeks, but never as raw regression: confounders (season, stock depth, weather) ride along with every price cut. The defensible version is difference-based: compare similar items marked down at different times, or the same item across stores with staggered markdowns, and bound the elasticity estimate — "between -1.2 and -2.0" is an honest answer; "-1.61" is a costume.

Markdown optimisation, one paragraph: for each dead SKU, choose the smallest discount that clears stock before its terminal date (season end), given the elasticity bound and the salvage value. A -30% markdown that clears 80% of units beats a -50% markdown that clears 95% — the remaining 15% is worth more in the bargain bin than the margin given away on the first 80.

51.5 The Customer Layer: RFM to CLV

From Your Toolkit — Excel and SQL: you have built an RFM table before, even if you called it "sorting customers". Recency, frequency, monetary value is a pivot table in Excel (GROUP BY in SQL): every customer scored 1–5 on each axis, segments read off the grid — champions, at-risk, hibernating, gone. The bridge to CLV is Chapter 52's subject in full, but the retail version in one line: **CLV = margin per period × expected periods remaining**, with expected periods estimated from the retention curve of the customer's segment. Ndineka's champions (top RFM decile) generate 41% of margin — the number that justifies the loyalty programme's entire budget, and the number to check monthly.

51.6 Returns and the Long Tail

Two quiet killers. **Returns:** the online channel's return rate (Ndineka's is 28%, driven by sizing) is a data problem — fit notes, size charts, and the returns ledger joined back to product metadata turns "free shipping both ways" from a cost into a feedback loop that fixes the catalogue. **The long tail:** 30,000 SKUs means most SKUs sell rarely; models fit on averages will misprice the tail. Segment the assortment (core / seasonal / tail) and model each with its own honesty — the tail gets rules and monitoring, not deep learning.

51.7 Failure Modes

- **The average customer** — no one is average; averages misprice the tail and mis-target promotions. Segment first (Chapter 30), always.
- **The dead-stock spiral** — refusing to mark down because "the margin loss is visible and the cash release isn't". Show the ledger in cash terms.
- **Promo dependence** — every promotion trains customers to wait (Chapter 32's sleeping dogs, in retail form). Track full-price sell-through as a health metric, not just revenue.

- **Basket-lift layouts** — rearranging stores on untested affinities; Chapter 61's discipline applies.

Teaching Tip — The frozen-cash walk: take the class to any clothing store (or its website) and have each student pick three items, guess weeks-on-shelf and the coming markdown, and write the retailer's likely reason for the buy. Back in the lab, the dead-stock ledger feels like a familiar object, not a SQL exercise. The best retail exam question is one sentence: "Why is retail inventory perishable?"

Key Takeaways

- Retail is four questions: assortment, price, customers, inventory health — and the canvas keeps them straight.
- The dead-stock ledger is the highest-ROI first project: no models, immediate cash conversation.
- Basket analysis (support, confidence, lift) is hypothesis generation; lift is not causation, and layouts deserve experiments.
- Elasticity is a bound, not a number; markdown is perishable-asset decay management.
- RFM is the bridge from transactions to customers, and full-price sell-through guards against promo dependence.

Practice Lab

1. Build the dead-stock ledger on Soulfya's inventory snapshot (Appendix F generator): classify SKUs, sum frozen value, and write the one-page memo that makes the cash visible.
2. Compute support, confidence, and lift for all item pairs in a month of baskets; list the top ten by lift and annotate each with "occasion, substitution, or cause?" — then design the experiment that would settle one of them.
3. Estimate the elasticity bound for a markdown family using staggered-store prices; write the confidence interval and one paragraph on what confounds remain.
4. Build the RFM grid for Soulfya's Soul Circle members; identify the at-risk segment and size the win-back budget with an expected-value calculation.
5. Audit the long tail: what share of SKUs sold fewer than three units last quarter, and what rule (not model) would you apply to them?
6. The Ndineka memo: three recommendations — one assortment, one pricing, one customer — each with its metric, its baseline, and its reversal condition.

Further Reading

- *The New Science of Retailing* — Fisher and Raman
- *Why We Buy* — Paco Underhill (the physical analogue of basket analysis)

- Chapter 32 (uplift), Chapter 61 (causal discipline for pricing experiments)

Chapter 52: Marketing and Customer Analytics — The Zuva Mobile Playbook

Part X — Analytics Across Industries: Domain Playbooks

"Marketing without analytics is spending; marketing with analytics is still spending, but you know which half."

In this chapter you will learn

- The marketing playbook canvas, on a telecom's honest data reality.
- Acquisition analytics: CAC by channel, cohort curves, and the paid-growth trap.
- Churn, ARPU, and the CLV engine — the trio that values every subscriber.
- Uplift modelling for campaigns, revisited at telco scale.
- Attribution honesty: why last-click is a story, not a measurement.
- Failure modes: ROAS vanity, frequency fatigue, and the segment that cannot be saved.

52.1 The Marketing Question

Zuva Mobile is a fictional mobile network operator: 900,000 prepaid subscribers, 60,000 postpaid, a youth-heavy brand, and a marketing department that spends \$2.4 million a year across above-the-line media, agent commissions, digital, and sponsorships. The CFO's question is the one every marketing analytics practice is built to answer: **which spending changes customer behaviour, and which merely accompanies it?**

The canvas:

Slot	Zuva Mobile's answer
Core question	Which spend acquires, retains, and grows customers at positive lifetime value?
Unit of analysis	The subscriber × week; the campaign × cell
Key metrics	CAC, churn rate, ARPU, CLV, campaign incremental lift
Data reality	CDRs, recharges, campaign exposure logs with gaps, agent self-reports, no clean exposure truth
First project	The cohort retention curve by acquisition channel
Failure mode	Buying revenue with promotions and calling it growth

52.2 Acquisition: CAC and the Cohort Curve

Cost per acquisition by channel is arithmetic; it becomes analysis when paired with what happens after acquisition. The cohort curve is the method (Chapter 23's discipline applied to growth): group subscribers by acquisition month and channel, then plot retention and ARPU forward.

```
SELECT acquisition_cohort, channel,
       DATE_DIFF('week', acquisition_date, activity_week) AS weeks_in,
       COUNT(DISTINCT subscriber_id) AS active,
       ROUND(SUM(recharge_amount) / COUNT(DISTINCT subscriber_id), 2) AS arpu
FROM subscriber_activity
GROUP BY 1, 2, 3
ORDER BY 1, 2, 3;
```

The curve exposes what CAC alone hides: agent-acquired subscribers cost \$3.10 each and half are gone in six weeks; digital-acquired cost \$7.80 and hold 71% at week 26. **CAC without the cohort curve is a price tag on a mystery.** The channel that looks expensive per head is often the cheap one per *retained* head — and the metric that settles it is CAC amortised over surviving subscribers, or bluntly, payback weeks: months until cumulative margin covers acquisition cost.

52.3 The CLV Engine

The three numbers that value a subscriber base — churn, ARPU, CLV — form one engine:

- **Churn** (Chapter 27's survival view is the honest one): weekly active-churn for prepaid, where "active" must be *defined* (days since last recharge against the subscriber's own rhythm — a 30-day-absence means different things for a \$1-a-day user and a \$20-a-month user).
- **ARPU** (average revenue per user): revenue over subscriber-weeks, watched as a distribution, not an average — prepaid ARPU is bimodal.
- **CLV**: margin per period × expected periods remaining, where expected periods comes from the retention curve of the subscriber's segment, discounted if the horizon is long. The formula is one line; the craft is the segmentation that feeds it, because CLV computed on the whole base is a single number that describes no one.

From Your Toolkit — Power BI: the CLV engine is the classic three-page Power BI model: a cohort matrix visual (acquisition month × months-since, shaded by retention), a decomposition tree for ARPU, and a card row for payback weeks by channel. The toolkit bridge matters because the marketing team will *live* in this report; build it where they already work.

52.4 Campaigns: Uplift at Scale

Chapter 32 taught the four cells (persuadables, sure things, lost causes, sleeping dogs); telco scale makes the economics brutal. Zuva Mobile's airtime-bonus campaign reaches 300,000 subscribers per blast. Treating everyone wastes three-quarters of the spend and actively harms the sleeping dogs. The uplift discipline: run the campaign as a permanent 10% holdout (random, rotated monthly so no subscriber is forever untreated), model the *difference* in outcome between treated and untreated by segment, and target only segments whose uplift clears the cost line. The holdout is not lost revenue; it is the measurement budget, and it pays for itself the first time it kills a campaign that was buying revenue Zuva already had.

52.5 Attribution Honesty

Digital attribution is where marketing analytics goes to feel scientific. The honest position: **last-click attribution is a convention, not a measurement** — it assigns the conversion to the final touch, which systematically over-credits search and retargeting (the closers) and under-credits the media that created demand (the openers). The alternatives are all imperfect: multi-touch models are assumptions arranged in a funnel; media-mix modelling (Chapter 62's regression discipline at weekly granularity) is slow but honest about offline; and geo-experiments — splitting regions into treatment and control — are the closest to truth a big brand can buy. Zuva Mobile's practical stack: last-click for tactical optimisation within a channel, geo-experiments for the big budget questions, and the humility to say "unattributed" out loud.

52.6 Brand and Survey Analytics

Surveys are marketing's second dataset: aided/unaided awareness, consideration, and intent, sampled properly (Chapter 15) and tracked quarterly. The craft is refusing the vanity read: awareness that does not move consideration is not working, and intent questions need the stated-vs-revealed caveat. **From Your Toolkit — SPSS:** the tracking study is where your SPSS crosstab-and-chi-square training applies directly — significance tests on awareness by region and age band, with the sampling weights respected (Stata's `svy` discipline, same idea).

52.7 Failure Modes

- **ROAS vanity** — return on ad spend computed without a holdout: revenue that would have arrived anyway, booked as marketing's achievement.
- **Frequency fatigue** — reaching the same subscribers weekly until they churn to escape the campaign; watch opt-outs per exposure.
- **The unsaveable segment** — churn models that concentrate spend on the highest *risk* rather than the highest *uplift* (Chapter 32 again: some churners are lost causes; spend on the persuadables).
- **The average subscriber** — bimodal prepaid ARPU averaged into meaninglessness.

Teaching Tip — The holdout debate: stage it. Assign half the class to defend "every impression should be monetised, holdouts waste reach" and half to defend "unmeasured spend is unmanaged spend". Ten minutes, then show a real campaign's incremental lift beside its last-click ROAS. The gap between the two numbers is the whole section, seen rather than told.

Key Takeaways

- CAC is a price tag; the cohort retention curve is the product. Never quote one without the other.

- CLV is one formula over three honest inputs: segmented churn curves, ARPU as a distribution, and expected periods from the retention curve.
- Campaigns run on uplift with permanent holdouts — the holdout is the measurement budget, not lost revenue.
- Attribution conventions are not measurements: last-click for tactics, geo-experiments for truth, and "unattributed" as a respectable answer.
- Target uplift, not risk; watch frequency fatigue; and never average a bimodal base.

Practice Lab

1. Build the cohort retention matrix (acquisition month $\times$ weeks-in) for Zuva Mobile's data; identify the channel whose CAC looks cheapest per head and most expensive per survivor.
2. Compute payback weeks by channel; write the investment memo: which channel gets next year's incremental \$500,000, with the reversal condition stated.
3. Fit the CLV engine on segmented retention curves; produce the base valuation under three churn scenarios (base, +2 points, -2 points) and comment on the swing.
4. Design the uplift campaign: which segments get treated, what the holdout is, and what the success metric is before launch. Then compute the campaign's incremental profit from the (planted) results in Appendix F's campaign dataset.
5. Run the attribution audit: take a month of exposure and conversion logs, compute last-click ROAS, and list every assumption it makes in one table; mark which ones a geo-experiment would test.
6. The Zuva memo: the marketing budget is cut 15% — what do you cut, what do you protect, and what measurement survives the cut?

Further Reading

- *Marketing Analytics* — Wayne Winston (the operations-research view)
- Chapter 32 (uplift), Chapter 15 (sampling, for the survey layer), Chapter 61 (causal inference for media mix)
- Zuva Mobile returns in Chapter 65 (Kumba's language services on Zuva's zero-rated bundle) and Appendix J

Chapter 53: Financial Services — The ZuvaPay Playbook

Part X — Analytics Across Industries: Domain Playbooks

"In lending, the model is not the product; the loss function is the product."

In this chapter you will learn

- The financial services canvas: risk, fraud, and regulation as one analytical system.
- Credit scoring the modern way, with the classic scorecard as its interpretable core.
- Class imbalance done honestly: cost-based thresholds for fraud.
- Explainability as law, not preference — and what a regulator actually asks.
- Population stability and drift monitoring as a *prudential* habit.
- Failure modes: score gaming, threshold drift, and the excluded applicant.

53.1 The Financial Services Question

ZuvaPay is a fictional payments-and-credit fintech: 1.1 million wallet users, 40,000 merchant points, and a fast-growing nano-loan product — \$10 to \$150, 30-day terms, approved in seconds from wallet behaviour. Its analytical life is dominated by three questions: **who will repay** (credit risk), **who is stealing** (fraud), and **can we prove it** (regulation). The twist that makes this playbook distinct from every other chapter in Part X: the *cost of being wrong is asymmetric and regulated* — a declined good customer costs \$4 of margin; a funded fraudster costs \$150 of principal; and a model the regulator cannot understand is a model you cannot deploy, whatever its AUC.

Slot	ZuvaPay's answer
Core question	Approve, price, and detect — at what thresholds, provable to whom?
Unit of analysis	The application (credit); the transaction (fraud)
Key metrics	Default rate, loss rate, approval rate, false-positive cost of fraud, model stability (PSI)
Data reality	Wallet histories, device signals, bureau thin-files for most applicants, adversarial fraud
First project	Vintage curves on the existing loan book
Failure mode	Accuracy that cannot survive an audit

53.2 Credit Risk: Vintage Curves First

Before any model, the vintage analysis: group loans by origination month, track cumulative default through days-past-due windows (30/60/90), and read the curves. The vintage chart is the lending business's ECG — it shows whether the book is improving cohort by cohort, and it calibrates everyone (credit committee included) to the actual default base rates before a single score exists.

The scoring stack is then a Chapter 23-25 problem with domain clothing: features from wallet history (top-up rhythm, spend volatility, airtime borrowing, merchant mix), a binary target (90+ days past due within term), and a *business* metric — profit at threshold, not AUC. The classic scorecard remains the deployed core in most regulated lenders, and its craft is worth knowing:

```
# Weight-of-evidence binning: the scorecard's interpretability engine
woe = (df.groupby("bin")["target"].agg(["sum", "count"]))
woe["good"] = woe["count"] - woe["sum"]
woe["woe"] = np.log(
    (woe["good"] / max(woe["good"].sum(), 1))
    / (woe["sum"] / max(woe["sum"].sum(), 1))
)
```

Each feature becomes a small table of bins and weights; the score is an addition; and "your recharge regularity contributed -12 points" is an explanation a call-centre agent can read aloud. **From Your Toolkit — SPSS:** this is the direct descendant of the SPSS scorecard tradition you met in training — the same binning logic, now fitted in Python and monitored like a system.

53.3 Fraud: Imbalance and the Cost Line

Fraud detection is the hardest imbalance problem in practice: 0.3% fraud prevalence, an adversary who adapts, and a threshold that is a *business decision* wearing a technical costume. The honest method:

1. Baseline rules first (velocity, device reuse, geo-inconsistency) — they catch the blunt fraud and they are explainable.
2. A model for the residue (gradient boosting on behavioural sequence features), evaluated on **precision at the review-queue size** the operations team can actually staff, not on accuracy or even AUC.
3. The threshold set from the cost matrix: review is worth it when $\text{expected fraud caught} \times \text{loss size} > \text{review cost} \times \text{false-positive rate} \times \text{queue size}$. At ZuvaPay's numbers (\$150 average loss, \$2.20 per review, 30 minutes of customer friction), the queue that clears this line is 1.8% of transactions — and *that* number, not the model, is the deliverable.

53.4 Explainability as Law

The regulatory layer is not optional garnish. A ZuvaPay decline triggers, in most markets, a right to explanation: the principal reasons, in human language, in a regulated format. The practical stack is Chapter 33's: reason codes from the scorecard bins (native), SHAP-style attributions for the model layer (Chapter 33), and a documented mapping from every feature to its data lineage (Chapter 42's governance, enforced by the regulator rather than chosen by taste). The discipline to internalise: **explanation artefacts are part of the model artefact**, versioned together, tested together — an explanation that lags the model is a compliance incident waiting for its auditor.

53.5 Monitoring: PSI and the Champion-Challenger Rhythm

Credit models decay as the population and the economy move. The standard vigilance is the **population stability index** on every feature and the score itself, monthly:

```
SELECT score_band,
       COUNT(*) * 1.0 / SUM(COUNT(*)) OVER () AS share_now,
       expected_share,
       ROUND((share_now - expected_share)
             * LN(share_now / NULLIF(expected_share, 0)), 4) AS psi_part
FROM scored_applications
GROUP BY score_band, expected_share;
```

PSI above 0.25 on a feature or the score is the alarm: the world the model learned has moved. The response is Chapter 43's champion-challenger: refit as challenger, shadow-score live, swap only when the challenger beats the champion on recent data — with the swap itself documented, because the regulator will ask.

53.6 Failure Modes

- **Score gaming** — applicants (and agents) reverse-engineer the scorecard and manufacture the signals; monitor feature distributions for spikes at bin edges.
- **Threshold drift** — the model is stable, the queue is not: a review queue allowed to grow silently changes the product's economics and its customer experience.
- **The excluded applicant** — thin-file applicants score low by absence of data, not absence of worth; the fair-lending audit (Chapter 29's disparate-impact discipline) is a prudential duty, and alternative-data features must justify themselves against it.
- **Accuracy without auditability** — the 0.87-AUC deep model that cannot ship; in regulated credit, the explainable 0.84 wins the right to exist.

Teaching Tip — The decline letter: give students a scored applicant (features, bins, points) and have them write the regulated decline letter — principal reasons, plain language, the applicant's rights. Nothing exposes a shallow understanding of a scorecard faster than trying to explain one to the person it rejected. Pair it with the mirror exercise: the letter the *falsely declined* applicant deserves.

Key Takeaways

- Lending analytics starts with vintage curves, not models — the book's ECG before its engine.
- The scorecard's binning-and-addition design is not nostalgia; it is explainability engineered into the artefact.
- Fraud thresholds are business decisions: staff the queue from the cost line, and evaluate on precision-at-queue-size.
- Explanations are part of the model artefact — versioned, tested, regulated.
- PSI monthly, champion-challenger always; and the fair-lending audit is not optional.

Practice Lab

1. Build vintage curves on ZuvaPay's loan book (Appendix F generator); identify the deteriorating cohort months and hypothesise (with evidence) why.
2. Fit a scorecard: WOE-bin five features, fit a logistic regression on the WOEs, produce points, and write the decline letter for three named applicants.
3. Set the fraud queue: given the cost matrix and the model's precision-recall curve, choose the queue size, defend it in dollars, and show what the queue costs in customer friction.
4. Compute monthly PSI on the score and top features; find the feature that broke 0.25 and write the incident note.
5. Run the fair-lending audit: approval rates and score distributions by applicant segment (urban/rural, age band); document any gap and propose the investigation.
6. The credit-committee memo: raise approval rate 5 points without raising loss rate — what lever do you pull, what would prove you wrong, and what does the regulator need to see?

Further Reading

- *Credit Risk Scorecards* — Siddiqi (the scorecard bible)
- Chapter 23 (evaluation and cost), Chapter 29 (fairness), Chapter 33 (explanation), Chapter 43 (monitoring)
- Appendix J Case 2 for the leakage post-mortem pattern that applies to credit models verbatim

Chapter 54: Healthcare Analytics — The Batanai Health Trust Playbook

Part X — Analytics Across Industries: Domain Playbooks

"In healthcare the error costs are not symmetric, the ethics are not optional, and the chart is someone's mother."

In this chapter you will learn

- The healthcare canvas: outcomes, flow, and cost under a duty of care.
- Readmission risk: the canonical clinical prediction problem, done honestly.
- Patient flow and staffing: queueing and simulation for wards and clinics.
- Survey and epidemiology analytics with sampling weights respected.
- Privacy beyond compliance: consent, minimum necessary, and the harm test.
- Failure modes: the risk score as a gate, Goodhart on clinical targets, and the average patient.

54.1 The Healthcare Question

Batanai Health Trust is a fictional non-profit health network: three clinics and a 60-bed district hospital, 140,000 patients on file, donor-funded, and measured by the Ministry and the donors on a scorecard it did not choose. Its analytical questions: **who needs care before they return deteriorating** (risk), **how do we flow patients through scarce beds and staff** (operations), **what is actually happening in the population** (epidemiology), and **what can we honestly claim to our funders** (evaluation). The distinctive constraint is the duty of care: an error here is not a lost sale; it is a person.

Slot	Batanai's answer
Core question	Which patients need which intervention, when — and does it work?
Unit of analysis	The patient-episode; the clinic-day; the catchment sector
Key metrics	Readmission rate, time-to-treatment, bed occupancy, programme yield, cost per outcome
Data reality	An EMR with coding gaps, paper registers at two clinics, staff logs, DHIS2 reports to the Ministry
First project	The missed-appointment and readmission register, unified
Failure mode	A model that gates care instead of guiding it

54.2 The Unified Register First

The first project is deliberately unglamorous: one patient-level register unifying EMR extracts, the clinics' paper registers (photographed, keyed, and *validated* — Chapter 11's discipline, because a register nobody trusts is a rumour with columns), and follow-up call logs. The register's columns are the analytics agenda: patient key, episode dates, diagnoses (coded, with an "uncoded" bucket that is itself a finding), discharge destination, appointment dates, attendance, and outcomes. Every later model, dashboard, and donor claim stands on this register — which is why it is a governance artefact (Chapter 42) with a named steward, not a spreadsheet on someone's laptop.

54.3 Readmission Risk, Honestly

The canonical problem: predict 30-day readmission, so that discharge planning and follow-up can concentrate where risk is highest. The honest version has four features that distinguish it from the Kaggle costume:

1. **The baseline is clinical, not algorithmic.** Nurses already know the highest-risk patients by diagnosis, age, and social circumstance. The model must beat clinical judgement (or, more realistically, *augment* it by surfacing non-obvious risk — the young patient with a chaotic visit history).
2. **The target is defined by operations, not purity.** "Readmission to any facility within 30 days of discharge" — and the register must be able to see transfers, or the target is silently wrong.
3. **Calibration beats discrimination.** A risk score that says 20% must mean 20%, because the intervention (a follow-up call, a review slot) is rationed by threshold; miscalibrated risk rations it wrong (Chapter 25's calibration discipline, now clinical).
4. **The intervention defines the metric.** The point is not AUC; it is whether flagged patients who received the follow-up bundle readmit less — an uplift question (Chapter 32) wearing scrubs. Deploy with a randomised holdout of flagged patients from day one; it is both ethical and the only proof that works.

54.4 Flow, Beds, and Staffing

The operations layer is Chapter 66's queueing and Chapter 67's simulation, in a hospital's clothing. The ward is a queue: arrivals (admissions by hour and day), servers (beds and nurses), and a waiting discipline with clinical priority. Batanai's maternity ward runs at 92% peak occupancy — past the ρ -cliff where Chapter 66's waiting times explode — and the analyst's contribution is the queueing arithmetic that turns "we feel busy" into "at 92% utilisation your expected wait is four hours; at 80% it is ninety minutes; here is the staffing plan that buys the difference". The simulation then tests the plan against reality: elective-schedule changes, a flu week, one nurse absent — the p90 day, not the average day, is what the ward manager lives.

54.5 Population and Survey Analytics

The epidemiology layer answers "what is happening in the catchment": prevalence estimates from clinic data (biased toward the sick who attend — the classic selection problem, Chapter 15), corrected by community surveys with proper sampling. **From Your Toolkit — Stata:** this is Stata's home ground — `svyset` with the sampling design, `svy: proportion` for prevalence with design-corrected confidence intervals, and `svy: logistic` for the risk-factor models. The same discipline in Python (`statsmodels` with survey weights) gives the same answers; what must never happen is the unweighted crosstab presented as a population estimate — the single most common published error in programme reporting.

54.6 Privacy Beyond Compliance

Health data is the hardest privacy case: re-identification is easy (rare diagnosis + suburb + age = one person), consent was often gathered pre-analysis, and the harm from leakage is direct. The practitioner's floor, beyond every regulation: **minimum necessary** (the analysis extract carries only the columns the question needs — the full EMR never leaves the clinical system), **the harm test** (before any release, ask what the worst reader could do with it), and **aggregate-only reporting** for anything leaving the Trust — with small-cell suppression (no cell below 5) as the default rule. Chapter 68's formal privacy engineering is the frontier; this section is the floor beneath it.

54.7 Failure Modes

- **The risk score as a gate** — a model intended to *guide* discharge planning quietly becomes the rule that denies the follow-up call to the "low-risk"; monitor what decisions the score actually feeds.
- **Goodhart on clinical targets** — time-to-treatment targets met by a triage queue that starts *after* registration; every operational metric needs its companion audit (Chapter 45's discipline, clinically applied).
- **The average patient** — clinical subgroups (age bands, comorbidities, distance) differ enough that one model or one threshold mis-serves them; stratify evaluation, not just training.
- **The donor-pleasing estimate** — unweighted clinic data reported as population prevalence; the error of Section 54.5, published.

Teaching Tip — The chart is someone's mother: run the ethics round before the lab. Give students a de-identified extract and the local newspaper's court report page; thirty minutes to attempt re-identification of one patient. The exercise lands harder than any lecture on privacy — and it motivates small-cell suppression, minimum necessary, and aggregate-only reporting as *craft*, not compliance theatre.

Key Takeaways

- Start with the unified register: every model, dashboard, and donor claim stands on it; it is a governance artefact with a steward.
- Readmission models must beat or augment clinical judgement, calibrate (not just discriminate), and be evaluated as an intervention with a randomised holdout.
- Queueing arithmetic and simulation turn "we feel busy" into staffing plans tested against the p90 day.
- Survey analytics respects the design: unweighted clinic data is not population prevalence.
- The privacy floor is minimum necessary, the harm test, and small-cell suppression — Chapter 68 is the frontier, not the floor.
- Watch what decisions each score feeds: gates are the failure mode of guides.

Practice Lab

1. Build the unified register's validation report on the Batanai extract (Appendix F generator): duplicate patient keys, uncoded diagnoses, appointment dates after discharge, and the paper-register reconciliation table.
2. Fit the readmission model; evaluate against the clinical baseline, report calibration by decile, and write the deployment note that includes the randomised holdout design.
3. Queueing arithmetic: given arrival and service rates by shift, compute expected waits at current staffing and at two alternatives; present the staffing memo with the p90 simulation results beside the means.
4. The survey correction: estimate prevalence from clinic data, then re-estimate with the community survey's design weights; write the two sentences a donor report needs explaining the difference.
5. Run the re-identification exercise on your own extract; document the three easiest attacks and the specific controls that block each.
6. The Goodhart audit: pick one Ministry indicator, describe how it could be met while care worsens, and design the companion metric that catches it.

Further Reading

- *Clinical Prediction Models* — Steyerberg (the honest textbook of the genre)
- Chapter 32 (uplift as intervention evaluation), Chapter 63 (survival curves for time-to-readmission), Chapter 66–67 (queueing and simulation)
- Appendix J Case 4 for the policy-evaluation pattern that donor claims should follow

Chapter 55: Agriculture and Food Security — The Chikafu Foods and NGA Playbook

Part X — Analytics Across Industries: Domain Playbooks

"Farming is a weather hedge wearing overalls; food security analytics is the hedge book."

In this chapter you will learn

- The agriculture canvas: yield, logistics, and markets as one food-system problem.
- Yield forecasting that combines agronomy, weather, and remote sensing — with honest error bars.
- Grain logistics and the buffer stock: seasonality, storage economics, and the import decision.
- Smallholder data: satellite plus surveys, and the bias in every convenient source.
- The co-operative evaluation pattern: Sable Fields and the missing-data discipline.
- Failure modes: the average farm, the planting-date fallacy, and precision without coverage.

55.1 The Food-System Question

Chikafu Foods is a fictional grain miller and food processor: it buys maize and small grains from 40,000 smallholders and three commercial estates, mills for retail, and holds a supply contract with the **National Grain Authority (NGA)** — the fictional state buyer that runs the strategic reserve. Their analytical questions chain together: **how much will grow** (yield forecast), **where and when will it move** (logistics), **what should we pay and stock** (markets and the buffer), and **what did the programmes actually do** (evaluation). Agriculture is the playbook where the *system* is the unit: a miller's forecast error becomes the NGA's storage decision becomes the consumer's price — and the analyst who sees only their own link optimises the wrong thing.

Slot	Chikafu / NGA's answer
Core question	How much grain, where, when — and what should the reserve do about it?
Unit of analysis	The field/plot for yield; the depot-week for logistics; the district-season for food security
Key metrics	Yield (t/ha), forecast error at lead time, depot stock weeks, price seasonality, food-security coverage
Data reality	Remote sensing pixels, sparse weather stations, farmer registers with staleness, market price boards
First project	The yield forecast with published, scored error — plus the error history
Failure mode	Confident averages: the mean yield that describes no farm and the reserve that trusts it

55.2 Yield Forecasting with Honest Error Bars

The yield forecast is the sector's headline model, and its craft is layering information sources with their uncertainties:

- **Agronomic structure first:** planted area $\times$ trend yield, adjusted for season start (planting dates are a strong signal — late onset means lower yield potential, and the adjustment is bounded by agronomy, not fit).
- **Weather on top:** rainfall accumulation and distribution by dekad (ten-day period) from the sparse station network plus gridded products; distribution matters — three well-spaced rains beat one deluge.
- **Remote sensing:** vegetation indices (NDVI-type) from public satellites give a canopy-health read per pixel, aggregated to district — the pixel sees every field, including the ones nobody reported.
- **Ground truth:** crop-cut surveys on sampled plots — small, expensive, and the only anchor that keeps the satellite honest.

The forecast is published with the *history of its errors*: "district maize yield, 1.9-2.3 t/ha, past-April error ± 0.4 ". A forecast without its scored error history is a rumour with decimals — and the NGA's storage decisions are exactly the kind of decision (Chapter 61's language) that deserves an interval, not a point.

```
# Layered yield forecast: agronomic base, weather adjustment, NDVI residual
base = planted_area * trend_yield           # agronomic anchor
weather_adj = 1.0 + (-0.35) * late_onset_frac # bounded by agronomy
ndvi_adj = 1.0 + 0.5 * (ndvi_z - ndvi_clim_z) # canopy anomaly
forecast = base * weather_adj * ndvi_adj
err = forecast - crop_cut_yield              # scored against ground truth
```

55.3 The Buffer Stock and the Logistics Layer

The NGA's reserve problem is seasonal by construction: buy at harvest when prices sag, release in the hungry months when prices spike, hold enough to smooth but not so much that storage and losses eat the budget. The analytics are Chapter 28's forecasting (price seasonality with its shocks), Chapter 47's optimisation (depot siting and movement minimisation — grain is heavy and fuel is dear), and one accounting discipline: **carryover cost per tonne per month** (storage + interest + shrinkage), because the release decision is "sell now versus hold", and that number is the whole trade. The failure to fear is the reserve that becomes a hoard: bought for price smoothing, held for prestige, financing itself out of existence.

55.4 Smallholder Data: Satellite and Survey

The sector's hardest data problem: the average farm is 1.5 hectares, transactions are informal, and every convenient source is biased. Farmer registers overcount active farmers (dead entries never leave); purchases undercount sales to informal traders; satellite sees canopy but not crop (maize and weeds are both green). The craft is triangulation with explicit bias accounting — satellite area as the coverage layer, register+survey as the behaviour layer, and the gap between them *reported* as a finding, not patched silently. **From Your Toolkit — Stata:** the crop-cut and household survey analysis is Stata territory — design-based estimates, panel regressions for programme effects, and the robust standard errors that keep small-sample honesty — with Chapter 15's missing-data discipline standing behind it.

55.5 The Evaluation Pattern: Sable Fields

The co-operative evaluation of Appendix J Case 5 (Sable Fields Co-operative: 2,400 farmers, a four-year productivity grant, and a year of missing records) is the sector's canonical evaluation shape: treatment that was not randomised, data with gaps that are informative, outcomes complicated by exit. The pattern — quantify the missingness, choose the strategy, bound the estimate, model the exits with survival curves (Chapter 63), and publish the gap in the first paragraph — is the discipline every agriculture programme report owes its funders. Agriculture is where evaluation honesty is most tested, because the funders want transformation stories and the data usually offers bridges.

55.6 Failure Modes

- **The average farm** — mean yield over a right-skewed distribution of 1.5-hectare plots; median and deciles are the sector's honest descriptors.
- **The planting-date fallacy** — late planting correlates with poorer farmers, so "late planters yield less" is partly poverty, not delay; the covariate structure must be respected before advising on timing.
- **Precision without coverage** — a beautiful model on the three commercial estates' clean data, extrapolated to 40,000 smallholders it never saw; the coverage audit belongs in every agriculture report.
- **The prestige reserve** — Section 55.3's hoard; the buffer stock needs its own dashboard, with carryover cost and release triggers, before it needs a forecast.

Teaching Tip — The two-map exercise: hand students two district maps side by side — satellite greenness and crop-cut yield — and ask what the differences mean. The discussion discovers every section of this chapter: coverage versus ground truth, canopy versus crop, and why the reserve needs both maps plus a survey. It is the whole playbook in one visual, and it teaches the humility of remote sensing better than any lecture.

Key Takeaways

- Agriculture is a system: yield, logistics, and markets chain together, and optimising one link in isolation mistimes the whole.
- Yield forecasts layer agronomy, weather, and remote sensing — and publish their scored error history or count as rumours.
- The buffer stock is a smoothing instrument with a carrying cost; give it triggers and a dashboard, not prestige.
- Every convenient smallholder source is biased; triangulate and report the gaps instead of patching them silently.
- Programme evaluations follow the Sable Fields pattern: missingness quantified, estimates bounded, exits modelled, honesty in the first paragraph.

Practice Lab

1. Build the layered district yield forecast on the season's data; score it against crop-cut plots, and write the release note with the interval and the error history attached.
2. Price seasonality: decompose five years of market maize prices into trend, seasonal, and shock components; recommend the reserve's buy window and the release trigger.
3. The carryover audit: compute storage, interest, and shrinkage per tonne per month for each depot; identify the depot where holding grain is destroying value.
4. Coverage audit: compare register-reported planted area against satellite-derived area by district; map the gap and write the two hypotheses it most plausibly supports.
5. Design the Sable Fields successor evaluation: a grant renewal where you control the design from day one — specify the randomisation (or its honest alternative), the measurement plan, and the pre-registered analysis.
6. The food-security memo: the forecast comes in 12% below trend — what does Chikafu do, what does the NGA do, and what does each need to know that the other holds?

Further Reading

- *Agricultural Production Economics* — the yield-response foundations
- Appendix J Case 5 (Sable Fields) for the full evaluation; Chapter 63 (survival for exits); Chapter 68 (privacy for farmer registers)
- FAO's seasonal forecast methodology papers — the best public example of error-scored forecasting

Chapter 56: Energy Analytics — The Sable Energy Playbook

Part X — Analytics Across Industries: Domain Playbooks

"An electricity system has no warehouse: every kilowatt is consumed the instant it is made, forever."

In this chapter you will learn

- The energy canvas: demand, supply, and money under a zero-inventory constraint.
- Load forecasting: the model that runs the company, from daily shape to tariff-sensitive peaks.
- Prepaid meters and collections analytics: revenue assurance as a first-class product.
- Losses: technical versus non-technical, and the anomaly discipline that tells them apart.
- Tariff and subsidy analysis with elasticity respected — the policy layer.
- Failure modes: the average day, the duck curve, and the meter that lies.

56.1 The Energy Question

Sable Energy is a fictional national distribution utility: 780,000 metered connections, a peak demand that has doubled in a decade, prepaid meters in most homes, and a generation mix it partly controls (two coal units, one hydro contract, growing solar) and partly prays over (rainfall for the hydro, evening for the solar). The defining constraint makes energy unique among this part's playbooks: **electricity cannot be stored at scale**, so supply must meet demand every minute — which makes forecasting not a reporting function but the operational spine. The analytical questions: **what will demand be** (load forecast), **is everyone paying** (collections and losses), **where is the power going that no one paid for** (loss analytics), and **what should it cost** (tariffs and subsidies).

Slot	Sable Energy's answer
Core question	How much power, when; who is paying; where are the losses?
Unit of analysis	The connection × 30-minute interval; the feeder-day; the tariff band
Key metrics	Peak demand (MW) and timing, forecast MAPE at day-ahead, collection ratio, loss % (technical vs nontechnical), load factor
Data reality	Smart-meter interval data (partial), prepaid vending logs, SCADA feeds, feeder sensors, theft and tamper records
First project	The day-ahead load forecast with a published error, by region
Failure mode	Planning the average day and being destroyed by the peak

56.2 Load Forecasting: The Model That Runs the Company

Demand is the most structured series in this book: strong daily shape (morning rise, evening peak), weekly shape (weekday/weekend), seasonal shape (winter heating, summer cooling), and event structure (paydays, public holidays, a national football final). The craft is treating those structures as features, then letting the model handle the residuals:

```
features = pd.DataFrame({
    "hour":      ts.index.hour,
    "dow":      ts.index.dayofweek,
    "is_weekend": (ts.index.dayofweek >= 5).astype(int),
    "temp_c":    weather.reindex(ts.index)["temp"],
    "payday_0to2": payday_proximity(ts.index),    # 0,1,2 days after payday
    "lag_1d":    ts["mw"].shift(24 * 2),          # same interval yesterday
    "lag_1w":    ts["mw"].shift(24 * 2 * 7),      # same interval last week
})
```

The evaluation is Chapter 28's seasonal-naïve discipline (the league table, always) and the metric that matters operationally: **day-ahead MAPE at the evening peak**, because a 3% average error that is 8% at 18:30 is a utility that buys emergency power at the worst prices. The forecast feeds the unit-commitment decision (which generators run tomorrow), so its error has a price tag — and Chapter 61's counterfactual language applies: the value of a better forecast is the avoided cost of the mistakes, computable and reportable.

56.3 The Prepaid Layer: Collections and Vending

Prepaid metering turns the revenue question inside-out: instead of billing after consumption, the utility collects *before* it — and analytics shifts from receivables to vending behaviour. The questions are behavioural and operational: purchase frequency and size by segment (a household's vending rhythm is its energy fingerprint), disconnection-and-reconnection patterns, and the *timing* load — paydays, school fees season, harvest income in rural feeders. **From Your Toolkit — Excel and Power BI:** the vending analytics layer is a classic Excel-to-Power-BI product: purchase registers aggregated to connection-week, a decomposition view for seasonality, and a collections dashboard the commercial team lives in. The distinctive energy lesson for the analyst: in prepaid systems, **demand data doubles as payment-capability data**, and the two must be read together — a feeder whose consumption falls while its vending rises is telling you something about metering, not weather.

56.4 Losses: Technical and Non-Technical

Losses are the gap between energy bought (or generated) and energy sold, and the analytics exist to split that gap in two: **technical losses** (physics — resistance in lines and transformers, computable from network models and loading) and **non-technical losses** (theft, tampering, metering error, billing gaps — the human share). The method is Chapter 46's anomaly discipline at feeder granularity:

- Energy balance per feeder-day: energy in (substation) versus energy vended plus estimated unmetered use — the residual is the loss.
- Profile the residuals: technical losses rise smoothly with load squared; tampering leaves step changes and consumption-collapse signatures that physics cannot explain.
- Rank and investigate: feeders ranked by unexplained residual per connection, crossed with tamper-inspection outcomes, produce the inspection schedule — and the *audit* of whether inspections are working.

The quiet trap: an aggressive loss-reduction campaign can chase the physics. The split — modelled technical loss versus residual — is what keeps the campaign aimed at the human share.

56.5 Tariffs, Subsidies, and the Elasticity Layer

The policy layer is where energy analytics meets Chapter 61 head-on: tariffs change behaviour, and a tariff modelled with zero elasticity is a plan for a world that does not exist. The pattern is Appendix J Case 4's (Harare Metro Water) transposed: cross-subsidy designs (lifeline blocks funded by higher bands) evaluated under elasticity estimated from historical tariff shocks, with revenue-neutrality checked as a *range*, never a point. Sable Energy's version: the lifeline block (first 50 kWh cheaper) whose true cost depends on how many high-band customers respond to their price rise by installing solar — the elasticity term, wearing panels.

56.6 Failure Modes

- **Planning the average day** — capacity is sized and committed against peaks; every average-based report understates the risk by construction.
- **The duck curve** — solar shifts net demand's shape (deep midday trough, steep evening ramp); a forecast trained on gross consumption miscommits the evening units exactly when they are dear.
- **The meter that lies** — smart-meter data is sensor data: gaps, drift, and stuck registers are findings to model (Chapter 11's validation gates, per interval), not background noise to average through.
- **Loss theatre** — reporting total losses without the technical split, then celebrating reductions that were physics all along.

Teaching Tip — The zero-inventory hour: open the class by asking what a warehouse is for, then deleting it from the electricity system. Everything in the chapter follows from that one absence: forecasting becomes operations, storage becomes the most expensive product on earth (batteries), and the evening peak becomes the moment the company lives or dies. Students who feel the constraint stop treating load forecasting as "just a time series".

Key Takeaways

- Electricity's zero-inventory constraint makes the load forecast the operational spine: evaluate at day-ahead peak MAPE, and price the forecast's errors.
- Prepaid systems shift analytics from receivables to vending behaviour — and consumption data doubles as payment-capability data.
- Loss analytics exists to split physics from people: model technical loss, then investigate the residual, and audit the investigations.
- Tariff and subsidy analysis respects elasticity — revenue neutrality is a range, and the lifeline block has a solar-panel term.
- Every average is a trap: peaks, ramps, and p90 days are the utility's real calendar.

Practice Lab

1. Build the day-ahead load forecast for one region (Appendix F generator supplies interval demand and weather); run the league table including seasonal-naïve, and report MAPE overall and at the evening peak.
2. Price the forecast: given balancing costs, translate the forecast error distribution into an expected cost, and the 2-point MAPE improvement into its avoided-cost value.
3. The vending layer: build connection-week vending aggregates, find the payday signature, and identify three feeders whose consumption-vending divergence suggests metering problems.
4. The loss split: compute feeder-day residuals, estimate the technical loss model, rank feeders by unexplained residual per connection, and design the inspection experiment that tests whether the ranking works.
5. The tariff memo: evaluate the lifeline design under elasticity bounds from the last tariff shock; show the revenue range and name the political question the range hides.
6. The resilience test: simulate the p10 hydro-rainfall year against your demand forecast — what breaks first, and what would you pre-negotiate?

Further Reading

- *Electricity Economics* — the tariff and peak-pricing foundations
- Chapter 28 (forecasting), Chapter 46 (anomaly detection for losses), Chapter 61 (elasticity and counterfactuals)
- Appendix J Case 4 for the cross-subsidy evaluation pattern this chapter borrows

Chapter 57: Government and Public Sector Analytics — The Harare Metro Playbook

Part X — Analytics Across Industries: Domain Playbooks

"Public data answers to everyone at once: the minister, the auditor, the taxpayer, and the person in the queue."

In this chapter you will learn

- The public-sector canvas: services, evidence, and scrutiny under permanent visibility.
- Programme evaluation culture: the DiD discipline and the null result that must be published.
- Operational analytics for public services: queues, registers, and the trust problem.
- Open data and privacy: publication as an engineering problem.
- Procurement analytics: the anomaly layer where public money leaks.
- Failure modes: Goodhart's law at national scale and the report nobody reads.

57.1 The Public-Sector Question

Harare Metro is the fictional metropolitan authority you met in Appendix J Case 4 (the water tariff) — but its analytical life is broader than one tariff: refuse collection, water and sanitation, road maintenance, licensing, and the social programmes it administers on behalf of national government. The public-sector pattern differs from every other playbook in Part X on three axes: **many masters** (the political principal changes, the permanent service endures, the public owns the data), **full scrutiny** (an auditor, a journalist, or an opposition councillor may read any number you publish), and **the null-result duty** (programmes must be evaluated whether or not they worked — a cancelled finding is a public wrong).

Slot	Harare Metro's answer
Core question	Which services work, for whom, at what cost — provably?
Unit of analysis	The service interaction (permit, collection, connection); the ward-month; the programme-beneficiary
Key metrics	Service level (collections/permits per target), unit cost, programme coverage and leakage, citizen wait times
Data reality	Legacy systems, paper registers, ward-level politics in every count, periodic national surveys
First project	The service-level baseline: one service, measured honestly, published
Failure mode	Indicators that become targets (Goodhart) and evaluation that becomes advocacy

57.2 The Service Baseline and the Trust Problem

The first project in any public analytics practice is establishing a *baseline anyone can check*: for one service — refuse collection is the classic — the fleet's GPS tracks, the ward collection schedule, and a complaint log, reconciled into a single published number: "86% of scheduled collections achieved, by ward, this quarter". The trust problem is the real opponent: public-sector data has been politicised before, and the analyst's craft is *showing the work* — methodology published, data extractable, numbers reconcilable by any journalist with a spreadsheet. Chapter 42's lineage discipline is not a nice-to-have here; it is the difference between a dashboard and a press release. **From Your Toolkit — Power BI**: the public dashboard is a Power BI (or open-source equivalent) product with one unusual requirement: every published figure must carry its provenance link, because the audience includes adversaries by design.

57.3 Programme Evaluation Culture

Chapter 61's methods belong here at full strength: difference-in-differences, synthetic control, and the interrupted time series are the public sector's workhorses, because randomisation is usually politically impossible. The Metro's discipline, learned the hard way:

1. **Pre-register the analysis** — the question, the comparison, and the success metric, dated before the data arrive. In politics, the temptation to redefine success afterwards is structural, and pre-registration is the structural answer.
2. **Respect the comparison's honesty** — the ward next door is not a control group until you show it trends in parallel (DiD's assumption, checked and displayed, per Chapter 61).
3. **Publish the nulls** — the street-lighting programme that did not move crime statistics is a finding the next budget needs; suppressing it converts an error into a tradition.
4. **Bound the estimate** — "reduced reported complaints by $15\% \pm 8$, with reporting artefacts unquantified" is the honest register; point estimates without intervals are lobbying with extra steps.

57.4 Operational Analytics: Queues and Registers

The service layer is Chapter 66's queueing in civic clothing: the licensing office is a queue (the Wq arithmetic that turned "we feel busy" into staffing plans in Chapter 54 works verbatim here), and the register is the product — a permit tracked from application to issuance, with the stage-level timestamps that expose where the month of delay actually lives. The public-sector twist is *adversarial demand*: the queue includes people who will pay to skip it, so process analytics (where the bottlenecks are) must be paired with integrity analytics (who skips, how often, at which stage) — Section 57.5's toolkit applied inward.

57.5 Procurement Analytics

Public procurement is where the money leaks, and the anomaly discipline (Chapter 46) has its highest civic value here: contracts clustered just below the tender threshold, single-bidder patterns by supplier, price drift against market indices, and the "split purchase" signature (one requirement, many small awards). The craft is the same three-layer stack as fraud: rules that catch the blunt patterns (explainable, auditable, publishable), a model for the residue, and a *human investigation* layer — because in the public sector, a flagged case that survives review must be presentable to an audit committee, and explainability is the law of the genre. Chapter 53's decline-letter discipline transposes: every flag carries its reasons, in language a non-technical councillor can repeat.

57.6 Open Data and Privacy as Engineering

Publication is the public sector's default and its hardest engineering problem: aggregate enough to protect (Chapter 54's small-cell suppression), document enough to verify (methodology, extract dates, known gaps), and version enough to trust (yesterday's number must still be reconstructable when the auditor asks — Chapter 42 again). The rare-data floor: administrative data on citizens carries the same harm test as health data (Chapter 54), and Chapter 68's formal techniques are the far end of the same road. The practitioner's rule: **publish the aggregate, protect the row, and write down which is which.**

57.7 Failure Modes

- **Goodhart at national scale** — collections-per-ward targets met by redefining "scheduled"; every indicator needs its companion audit metric (Chapter 45's pairing, now with an audit committee in the room).
- **Evaluation as advocacy** — the programme evaluation written to survive a cabinet meeting rather than an audit; pre-registration and published nulls are the antidotes.
- **The dashboard nobody uses** — the beautiful expenditure dashboard the councillors never open, because it answers no question anyone holds; Chapter 19's audience discipline is civic, not corporate.
- **The extract that becomes the record** — a one-off data pull, exported and quoted, drifting from the live system; version and date everything.

Teaching Tip — The adversarial reading: have students publish (in class) a one-page service baseline, then assign a third of the room to attack it as the opposition councillor: What is hidden? What is redefined? What would you FOA-request? The defending team learns more about methodology documentation in twenty minutes of adversarial reading than in a week of lectures — and it is exactly the muscle the job requires.

Key Takeaways

- Public analytics has many masters and permanent scrutiny: show the work — lineage, methodology, extractable data — because the audience includes adversaries by design.
- The first project is one honest, published service baseline; trust is the deliverable as much as the number.
- Evaluation culture: pre-register, check the parallel-trends assumption, publish the nulls, and bound every estimate.
- Procurement anomaly analytics is civic work: explainable flags, auditable reasons, presentable cases.
- Publish the aggregate, protect the row, and write down which is which.

Practice Lab

1. Build the refuse-collection service baseline from the fleet GPS and ward schedules (Appendix F generator); publish the methodology page and the reconciliation table, then run the adversarial reading exercise on a classmate's baseline.
2. DiD practice: evaluate the street-lighting expansion on ward-level incident data; display the parallel-trends check honestly and write the finding — including the null if that is what the data says.
3. Queueing at the licensing office: given arrival and service logs, compute the stage-level delays and the Wq arithmetic for two staffing options; write the memo with the p90 wait, not the mean.
4. Procurement anomalies: run the threshold-clustering and split-purchase rules on three years of awards; produce three flags with written reasons, each presentable to a non-technical committee.
5. The open-data release: take a service register and produce the publishable aggregate — suppression applied, methodology written, version dated — plus the internal note on what was withheld and why.
6. The indicator redesign: pick one current civic target, describe its Goodhart failure mode, and propose the paired metric that closes it.

Further Reading

- *Public Sector Analytics* literature: the What Works centres (the UK's movement is the reference model)
- Chapter 61 (causal inference), Chapter 42 (lineage), Chapter 46 (anomaly detection), Chapter 68 (formal privacy)
- Appendix J Case 4 (the tariff evaluation) — this playbook's deepest worked case

Chapter 58: Manufacturing Analytics — The Sabvura Industries Playbook

Part X — Analytics Across Industries: Domain Playbooks

"The factory already knows; the data's job is to get the knowledge out before the shift ends."

In this chapter you will learn

- The manufacturing canvas: throughput, quality, and uptime as one OEE story.
- Overall Equipment Effectiveness: the three losses and the availability × performance × quality arithmetic.
- Statistical process control: control charts that separate signal from shop-floor noise.
- Predictive maintenance: the survival-model discipline for machines that age.
- From forecast to line: production scheduling under Chapter 47's optimisation.
- Failure modes: local OEE, the cherry-picked chart, and the sensor swamp.

58.1 The Manufacturing Question

Sabvura Industries is a fictional garment manufacturer: three production lines (cutting, sewing, finishing), 410 operators, an export contract with a European buyer whose audits arrive unannounced, and a factory manager who keeps a whiteboard she calls "the truth" — hourly counts, defects, and downtime, written by hand. The analytical questions: **are the lines producing what they could** (OEE), **is quality stable or drifting** (SPC), **which machines will fail and when** (maintenance), and **what should we make next week** (scheduling). The distinctive feature of manufacturing data: it is generated *on the line, in real time, by machines and operators who will tell you the truth if the system makes truth-telling easy* — and the analyst's first job is usually to make the whiteboard digital without killing its honesty.

Slot	Sabvura's answer
Core question	Throughput, quality, uptime — where do the hours go?
Unit of analysis	The machine-hour; the lot/batch; the operator-shift
Key metrics	OEE and its three components, defect rate by defect class, mean time between failures, schedule adherence
Data reality	Machine counters, operator log sheets, QC inspections by lot, maintenance records in prose
First project	The downtime Pareto, machine-logged and reconciled
Failure mode	Local optimisation: a line that hits OEE by pushing defects downstream

58.2 OEE: The Three Losses

Overall Equipment Effectiveness is the sector's headline metric because it multiplies the three ways a shift is lost:

- **Availability** — run time ÷ planned time. Losses: breakdowns, changeovers, waiting for material.
- **Performance** — actual output ÷ output at rated speed. Losses: micro-stops, slow cycles.
- **Quality** — good units ÷ total units. Losses: defects and rework.

The arithmetic is honest and humbling: a world-class 85% OEE is $0.90 \times 0.95 \times 0.995$ — three decent numbers multiplied into an excellence few reach; Sabvura's finishing line runs 58%, which is three mediocre numbers multiplied into a management conversation. The craft is decomposition: OEE as a single number is a scoreboard, OEE *split into its three components by line and shift* is a diagnosis, and the downtime Pareto is where the first improvement project always lives.

```
SELECT line, shift_date, shift,
       SUM(run_minutes) / SUM(planned_minutes)           AS availability,
       SUM(actual_units) / NULLIF(SUM(rated_units), 0)    AS performance,
       SUM(good_units) / NULLIF(SUM(actual_units), 0)     AS quality,
       SUM(run_minutes) / SUM(planned_minutes)
       * SUM(actual_units) / NULLIF(SUM(rated_units), 0)
       * SUM(good_units) / NULLIF(SUM(actual_units), 0) AS oee
FROM line_counters
GROUP BY line, shift_date, shift;
```

58.3 Statistical Process Control

SPC is Chapter 14's inference, industrialised: a control chart is a hypothesis test drawn repeatedly through time, separating common-cause noise (leave it alone; tampering adds variance) from special-cause signals (find the reason, now). The discipline that survives every software fashion:

```
def xbar_chart(measurements, subgroup=5, sigma_k=3):
    import numpy as np
    groups = [measurements[i:i+subgroup] for i in range(0, len(measurements), subgroup)]
    means = np.array([g.mean() for g in groups])
    cl, sigma = means.mean(), np.array([g.std(ddof=1) for g in groups]).mean()
    ucl, lcl = cl + sigma_k * sigma / np.sqrt(subgroup), cl - sigma_k * sigma /
    np.sqrt(subgroup)
    out = [g for g, m in zip(groups, means) if m > ucl or m < lcl]
    return cl, lcl, ucl, out # out-of-control subgroups, each with its hours
```

The rule that gives SPC its power: **respond to signals, leave noise alone**. The classic manufacturing sin is adjusting a process that was stable — adding variance while feeling busy. The analyst's contribution is annotating the chart (which signal coincided with the fabric-batch change, the operator changeover, the humidity spike) so the investigation starts warm.

58.4 Predictive Maintenance

Machines age, and the honest model of aging is Chapter 63's survival analysis: each machine's hazard of failure as a function of age, run-hours, and condition signals (vibration, temperature, error codes). The discipline matters more than the model: survival curves give *probabilities over time windows* ("30% failure hazard in the next 200 run-hours"), not the false-precision "fails Thursday"; maintenance scheduling is then Chapter 47's optimisation (schedule the intervention to minimise expected downtime cost, given the hazard and the parts lead time). The cautionary tale every plant lives: the model that flags a failure risk the technicians already schedule around — predictive maintenance's value is over and above *experienced maintenance planning*, and proving that requires the same holdout discipline as any model (Chapter 23's league table, greasy edition).

From Your Toolkit — Excel: the maintenance backlog and the MTBF league table live happily in Excel pivot tables, and that is the right first home — the maintenance planners already work there. The survival model informs the planners; it does not replace them, and the interface is a table they trust.

58.5 From Forecast to Line

The scheduling layer closes the loop with the demand side: Chapter 28's order forecasts become Chapter 47's constrained schedule — lines, changeovers (whose cost in minutes is the real currency), operator rosters, and the export buyer's audit windows. The craft is making constraints explicit rather than implied: the intern's optimisation from Appendix J Case 3 failed not in its arithmetic but in its *unstated* constraint (the loading rule); the same lesson walks this factory's floor. A schedule no one can explain is a schedule the floor will quietly override — explainability is operational, not cosmetic.

58.6 Failure Modes

- **Local OEE** — the cutting line hits its target by passing borderline fabric to sewing; OEE must be measured at the *system* level, or the target destroys the plant.
- **The cherry-picked chart** — the control chart shown only in its stable months; keep the full run and the annotations, always.
- **The sensor swamp** — instrumenting everything and analysing nothing: start from the decision (which changeover? which intervention?) and instrument backwards from it.
- **The whiteboard's revenge** — the new digital system that operators feed garbage because truth-telling became harder than the paper it replaced; watch the reconciliation gap, and fix the form before blaming the floor.

Teaching Tip — The paper OEE: run one class session entirely on a paper log sheet: students record a simulated line's hour (planned, run, units, defects, reasons for stops) by hand, then compute OEE and the Pareto themselves. The lesson is double: they internalise the arithmetic *and* they feel how data quality is won or lost at the point of entry — the whiteboard manager's problem, lived for an hour.

Key Takeaways

- OEE is a scoreboard; its three components by line and shift are the diagnosis, and the downtime Pareto is the first project.
- SPC is repeated inference: respond to special-cause signals, leave common-cause noise alone — tampering is the industrial original sin.
- Predictive maintenance is survival curves plus scheduling optimisation, and must beat experienced planners in a fair holdout to count.
- Constraints belong in the model, not in the lore; a schedule the floor cannot explain is a schedule it will override.
- Data quality is won at the point of entry: make truth-telling easier than the paper it replaced.

Practice Lab

1. Compute OEE and its components by line and shift from the line counters (Appendix F generator); build the downtime Pareto and name the first improvement project with its expected minutes.
2. Build the X-bar chart for seam strength by line; annotate the out-of-control subgroups against the batch and operator logs; write the two investigation memos the chart demands.
3. Fit the survival model for the cutting machines (age, run-hours, vibration); produce each machine's 200-hour hazard window, and the maintenance schedule that minimises expected downtime cost.
4. The league-table test: does your survival model beat the planners' existing PM calendar in a backtest? Design the fair comparison and report the result honestly.
5. The scheduling round: given the order book, changeover matrix, and rosters, produce next week's schedule — then write the one-page explanation the floor supervisor reads aloud at the shift briefing.
6. The reconciliation audit: compare the digital counters against the whiteboard's week; quantify the gap by reason code, and propose the form change that closes the largest one.

Further Reading

- *Out of the Crisis* — W. Edwards Deming (SPC and the tampering doctrine)
- *OEE for Operators* — Nakajima (the source text, short and complete)

- Chapter 47 (scheduling optimisation), Chapter 63 (survival analysis), Chapter 14 (the inference beneath SPC)

Chapter 59: Transport and Logistics — The Kunaka Logistics Playbook

Part X — Analytics Across Industries: Domain Playbooks

"Logistics is the mathematics of promises: an ETA is a contract signed by a queue."

In this chapter you will learn

- The logistics canvas: routing, reliability, fleet health, and the cold chain.
- The constraint discipline of routing optimisation — and why the optimiser's world is not the world.
- ETA prediction: the regression that respects the road.
- Fleet analytics: utilisation, maintenance, and the cost per delivered case.
- IoT data engineering for cold-chain compliance: the temperature excursion ledger.
- Failure modes: average speed, the unmodelled constraint, and the dashboard that lies to the customer.

59.1 The Logistics Question

Kunaka Logistics is the fictional cold-chain operator you met in Appendix J Case 3: one depot, 62 retail drop points, 14 vehicles, and a dairy client whose product fails quality control if any drop arrives more than four hours after loading. Its analytical questions: **which stops on which vehicle in which order** (routing), **when will it actually arrive** (ETA), **what does each vehicle really cost** (fleet economics), and **did the cold chain hold** (compliance). The distinctive feature: logistics is the discipline of *constraints wearing costumes* — time windows, temperature ramps, loading order, driver hours — and the recurring failure of logistics analytics is building a beautiful model of a world without them.

Slot	Kunaka's answer
Core question	Which routes, in what order, at what reliability and cost?
Unit of analysis	The stop; the trip; the vehicle-day
Key metrics	Cost per delivered case, on-time %, distance per drop, temperature excursions, fleet utilisation
Data reality	GPS traces, telematics temperature logs, drive-time matrices, driver manifests on paper, traffic by hour
First project	The on-time baseline and the excursion ledger
Failure mode	Optimising distance in a world that runs on time

59.2 Routing: The Constraint Discipline

Vehicle routing is a canonical optimisation problem — and a canonical warning. Case 3's story is the sector in miniature: an 18% distance saving that became 11% once drive-times replaced distances and the four-hour constraint was enforced, then lost two routes entirely to the loading-order rule nobody wrote down. The practitioner's method:

1. **Write the constraints as a list, with owners.** Time windows, capacity, cold-chain ramps, driver hours, the customer's receiving dock that closes at 16:00. Each constraint gets a name and a source — the union rep's sentence about loading order is data.
2. **Solve with the simplest method you can explain.** Cluster-then-sequence (group stops by geography, then order each cluster greedily by time) is usually within a few percent of the optimum and can be explained to a driver — which is worth more than the missing percent.
3. **Validate by simulation, not assertion** (Chapter 67): Monte Carlo the plan under demand and traffic variance; read the p10 day. The plan that survives the p10 day is the plan; the rest is a diagram.
4. **Pilot on one route for two weeks** before signing the network. The pilot is where the unmodelled constraints introduce themselves.

59.3 ETA Prediction

The customer-facing layer is a regression problem with one unusual property: **its error has a sign that matters**. An ETA five minutes early is a promise kept; five minutes late is a promise broken, and the asymmetric cost belongs in the loss function — predict the *quantile* that makes lateness rare enough to match the service promise (Chapter 25's quantile thinking, delivered). The features are the road's honest structure: distance and traffic by hour, stop durations learned from GPS dwell times, the dock queue at the receiving end, and the trip's own history at that hour.

```
model = GradientBoostingRegressor(loss="quantile", alpha=0.85) # 85th percentile
model.fit(features, arrival_deltas_minutes) # positive = late
# The promise: "arrives by X" where X is the 85th-percentile arrival.
```

From Your Toolkit — Excel: the ETA *audit* is a pivot table the operations team can own: promised versus actual arrival by route and hour, the lateness rate, and the worst three stops — the analysis that keeps the model honest lives in the tool the dispatcher already has open.

59.4 Fleet Economics

The unit that disciplines everything is **cost per delivered case** — fuel, driver, maintenance, depreciation, and the cold-chain energy, divided by cases delivered — because it exposes the trade the fleet runs every day: an under-filled van at 40% capacity has a beautiful cost-per-kilometre and a terrible cost-per-case. The analytics: utilisation by day and route (the half-empty Tuesday run), maintenance cost per vehicle against age and odometer (Chapter 63's survival hazard, feeding the replace-or-repair decision), and the fixed-versus-variable split that answers the lease question. The quiet discipline: allocate costs to trips honestly (including the depot overheads), or the route-level profitability ranking is fiction with decimals.

59.5 The Cold-Chain Ledger

Cold-chain compliance is an IoT data engineering problem before it is an analytics one: temperature sensors stream intervals; excursions must be detected, attributed (which stop's door opening?), and reported to the dairy client within the contract's window. The artefact is the **excursion ledger** — every trip's temperature trace, every breach flagged with the stop, duration, and peak, and the reconciliation against the quality standard. The engineering is Chapter 9's pipeline discipline (validation gates on sensor data — stuck readings, gaps, clock drift) with Chapter 46's anomaly logic on top: the sensor that reports a perfectly flat 4.0°C for three hours is not a healthy freezer; it is a broken sensor, and the ledger must treat it as a data incident, not a compliance pass.

59.6 Failure Modes

- **Average speed** — routing on mean drive-times schedules the fleet for a world without the 16:00 truck; drive-time *distributions* by hour are the honest input.
- **The unmodelled constraint** — Case 3's lesson: constraints live in drivers' heads and unions' rules until they are written as data.
- **The dashboard that lies to the customer** — an on-time metric defined on departure rather than arrival; every customer-facing metric must name the clock it uses.
- **Sensor silence as health** — the flat line is an incident; validation gates make silence loud.

Teaching Tip — The drivers' meeting: have students present a route redesign to a panel playing drivers, dispatchers, and the union rep (Case 3's cast). The panel's first question is always a constraint the model missed — and the lesson, that the person closest to the work knows the model's missing variable, lands as lived experience rather than a maxim. Rotate roles; the next cohort's modellers are this cohort's drivers.

Key Takeaways

- Constraints are the logistics model: list them with owners, solve with methods you can explain, validate by simulation on the p10 day, and pilot before signing.
- ETA models predict service, not averages: fit the quantile that matches the promise, and audit promised-versus-actual in the dispatcher's own tool.
- Cost per delivered case disciplines the fleet; allocate overheads honestly or the route ranking is fiction.
- The cold-chain ledger treats sensor silence as an incident — validation gates first, compliance second.
- The person closest to the work knows the missing constraint; ask before optimising.

Practice Lab

1. Build the on-time baseline and excursion ledger from the telematics extract (Appendix F generator); report on-time % by route and the three worst stops, naming the clock each metric uses.
2. Refit the Case 3 routing with drive-time distributions and the loading constraint; report the honest saving and the two assumptions you most want validated in the pilot.
3. Fit the ETA quantile model; choose the service quantile, and show the lateness rate it implies against the customer's contract threshold.
4. The fleet economics table: cost per delivered case by vehicle and route; identify the under-filled run and the replace-or-repair candidate from the maintenance hazard curve.
5. The sensor audit: run the validation gates on a month of temperature logs; document the flat-line sensors, the clock drift, and what the compliance number becomes once they are excluded.
6. Run the drivers' meeting on your own redesign; log every constraint the panel adds, and revise the model and its explanation in one page.

Further Reading

- *Vehicle Routing* — Toth and Vigo (the reference; read Chapter 1 and the heuristics chapters)
- Appendix J Case 3 (the van in the wrong place) — this playbook's full worked case
- Chapter 47 (optimisation), Chapter 67 (simulation), Chapter 25 (quantiles for service promises)

Chapter 60: Media and Sport Analytics — The Nhaka FC and Kumba Playbook

Part X — Analytics Across Industries: Domain Playbooks

"Sport measures everything and wins nothing by measuring; media measures nothing and calls it reach. Both are wrong the same way."

In this chapter you will learn

- The two-headed canvas: performance analytics (sport) and attention analytics (media).
- Sport: expected goals in miniature, recruitment value, and injury risk as a survival problem.
- Media: the metrics that pay — attention, churn, and content decay.
- Kumba's frontier: analytics and language technology for low-resource markets.
- The storytelling bridge: how analytics becomes broadcast.
- Failure modes: Goodhart ruins the game, and reach is not revenue.

60.1 The Two-Headed Question

This playbook pairs two industries that share one underlying commodity — **attention** — and one shared failure: measuring what is easy instead of what matters. **Nhaka FC** is a fictional Premier Soccer League club: 28 professional players, an academy, modest revenue, and a technical director who wants to stop signing players "because the chairman saw them once". **Kumba** is a fictional media and language-technology company: streaming radio and news in Shona and Ndebele, a small advertising business, and a language-services division whose frontier work — Chapter 65's low-resource NLP — is watched from well beyond Harare. The canvas, twice:

Slot	Nhaka FC's answer	Kumba's answer
Core question	Which players, which style, which risks — at what wage?	Which content, for whom, at what attention value?
Unit of analysis	The match event; the player-season; the session	The listener-session; the content item; the campaign impression
Key metrics	Expected goals for/against, points per wage, availability rate	Listening hours, subscriber churn, attention per content dollar
Data reality	Event-coded matches, GPS training loads, sparse injury records	Streaming logs, app telemetry, no panel ratings, Shona/Ndebele content untitled
First project	The recruitment value ledger	The content decay curve
Failure mode	Metrics that ruin the playing style	Reach that never becomes revenue

60.2 Sport: Expected Goals in Miniature

The idea that changed football analytics: shots are not equal, so count the *quality* of chances. A minimal expected-goals (xG) model — the shot's outcome probability given location, body part, and assist type — is a logistic regression you can fit in an afternoon and defend in a pub:

```
import statsmodels.formula.api as smf
model = smf.logit("goal ~ distance + angle_rad + header + fast_break",
                  data=shots).fit()
shots["xg"] = model.predict(shots)
# A season's story: goals minus xG = finishing (or luck) above chance creation.
```

The honest uses: xG over a season separates *chance creation* from *finishing luck* (a striker 6 goals above xG is due a regression the wage negotiation should know about); xG against exposes a defence that has been bailed out by its goalkeeper; and the rolling xG difference is the earliest signal that the team's style has changed. The honest caveats: sample sizes are small (a season is ~350 shots), the model is only as good as the event coding, and — the classic — xG is a *scouting* tool, not a tactic: a team that optimises shot-taking to maximise xG becomes predictable (Section 60.6's Goodhart, in boots).

60.3 Recruitment and Injury

Recruitment value is the club's moneyball ledger: contribution (xG+xA per 90, availability-weighted) per wage dollar, compared honestly against the *replacement* — the academy player who costs a stipend. The craft is the comparison set: a winger scouted against the whole league looks average; against age-mates in the same tactical role, he is a bargain or a fraud. **Injury analytics** is Chapter 63's survival discipline in tracksuit: hazard of injury as a function of training load (GPS distance, high-intensity efforts), congestion (matches per 14 days), and history — used to *manage exposure* (rotate before the hazard spikes), never to promise that a player will stay fit. Availability is the quiet metric: the squad's expected available minutes is worth as many points as any signing.

60.4 Media: Attention That Pays

Kumba's analytics stand on one honest distinction: **reach is not attention, and attention is not revenue**. Reach counts devices; attention measures minutes; revenue needs *minutes that advertisers or subscribers will pay for*. The core artefacts:

- **The content decay curve:** listening by content item against minutes-since-publication, by format — news decays in hours, drama serials in days, and the shape (not the peak) tells the schedule what to commission next.
- **The churn layer:** Chapter 27's retention curves on subscriber data — which programmes appear in the listening histories of those who stay? (Correlation, honestly labelled; the causal test is Chapter 61's — stagger a schedule change across regions and watch.)

- **The attention ledger:** listening hours per content dollar, by format and language — the commissioning meeting's one table.

From Your Toolkit — Power BI: the attention dashboard is a genre you know: cohort retention curves, a decay chart per format, and the commissioning table on the front page. The bridge: this is the same retention analytics as Zuva Mobile (Chapter 52) with listening minutes in the ARPU seat.

60.5 Kumba's Frontier: Language Technology

Kumba's deepest analytics problem is Chapter 65's subject and this part's bridge to Part XI: content in Shona and Ndebele is poorly served by tools trained on the internet's English — transcription, search, and recommendation underperform exactly where Kumba's audience lives. The practical frontier: speech and text models adapted to low-resource languages (transfer learning from related languages, SentencePiece tokenisation on Shona corpora), metadata generated in-language (the untitled archive is unsearchable in any language), and — the quiet craft — *evaluation sets built by hand* because no benchmark exists. The strategic lesson for every analyst: where the tooling is thin, the data you can build becomes the moat.

60.6 The Storytelling Bridge and the Shared Failure

Analytics in both industries becomes valuable when it becomes *story*: the broadcast graphic (xG rolling difference at minute 70), the commentary line, the schedule insight a producer can pitch. Chapter 74's structures apply verbatim: one message, evidence, the decision. And both industries share the terminal failure mode — **Goodhart**: the player who stops shooting from anywhere because low-xG shots dent the dashboard; the newsroom that chases clicks into trivia. The practitioner's guard is the paired metric (Chapter 45): every performance metric travels with its health companion — xG with unpredictability, clicks with subscriber retention — and the pair is reviewed together, or the metric eventually ruins the thing it measured.

Teaching Tip — Code your own match: give students a raw match event file (20 minutes of an amateur match, coded from video) and have them compute xG, then explain it to a panel playing a sceptical coach, a chairman, and a journalist. The three explanations must differ — the coach wants tactics, the chairman wants money, the journalist wants a sentence — and the exercise teaches the last mile of this whole part: the same analysis, told three ways, or it was never done.

Key Takeaways

- Sport and media both measure attention; both fail by measuring the easy proxy.
- xG is a small, defensible model with big honest uses — chance creation versus finishing luck — and it is a scouting tool, not a tactic.
- Recruitment value is contribution per wage against the honest replacement; availability is the quiet signing.

- Reach is not attention, attention is not revenue: decay curves, churn curves, and the attention per content dollar ledger are the media desk.
- Where tooling is thin — low-resource languages — the data you build is the moat.
- Pair every metric with its health companion, or Goodhart ruins the game and the newsroom alike.

Practice Lab

1. Fit the minimal xG model on the shot data (Appendix F generator); report each striker's goals minus xG and write the wage-negotiation paragraph that regression-to-the-mean deserves.
2. Build the recruitment ledger: contribution per 90, availability-weighted, per wage dollar — against the whole league and against age-mates; note where the two comparisons disagree and which you trust.
3. Fit the injury hazard model on training loads and congestion; produce next month's rotation plan and its expected available minutes.
4. Build the content decay curves by format and language; write the commissioning memo — what to renew, what to retire, what to schedule at 18:30.
5. The churn layer: retention curves for subscribers with and without each flagship programme in their history; label the correlation honestly and design the staggered test that would settle it.
6. Code 20 minutes of a match (any match, your own events file) and produce the three explanations: coach, chairman, journalist.

Further Reading

- *The Numbers Game* — Anderson and Sally (football analytics' best entry point)
- *Soccernomics* — Kuper and Szymanski (recruitment economics)
- Chapter 63 (survival for injuries), Chapter 65 (low-resource NLP — Kumba's deep end), Chapter 74 (the three-way storytelling)

PART XI

The Frontier: Advanced Methods and Emerging Practice

Chapters 61 – 70

Chapter 61: Causal Inference — The Counterfactual Discipline

Part XI — The Frontier: Advanced Methods and Emerging Practice

"The question correlation cannot answer is the only question the business is asking: what would have happened anyway?"

In this chapter you will learn

- Potential outcomes: the one idea under every causal method.
- The credibility ladder: randomisation, natural experiments, adjustment — in order of trust.
- Difference-in-differences, and the parallel-trends check you must show.
- Instrumental variables and synthetic control, with their failure modes.
- Sensitivity analysis as a first-class result, not an appendix.
- The National Street Levy case: one policy, four estimates, and the 2.4x spread that honesty demands.

61.1 The Counterfactual Question

Every chapter before this one has asked "what does the data show?" This chapter asks the harder question underneath every business request: **what would have happened without the thing we're studying?** Soulfya's ran a promotion (Case 1); members who received it visited 31% more. The number is a fact; the question is whether the promotion *caused* it — and the answer lives in the visits of a customer who does not exist: the same customer, un-promoted, in the same quarter.

That ghost is the **potential outcome**, and it is the single idea behind every method in this chapter. We never see both outcomes for the same unit (the fundamental problem of causal inference), so every method is a discipline for constructing a believable ghost: a control group, a counterfactual trend, a natural experiment. **From Your Toolkit — Stata:** this is the question your econometrics training kept asking — "identified by what?" — and this chapter is the field guide to the answers.

61.2 The Credibility Ladder

Order your trust like this, and say the order out loud in every write-up:

1. **Randomised experiment** — the ghost is the control arm, different only by luck. Chapter 67's discipline.
2. **Natural experiment** — assignment to treatment happens by forces that look like luck: a threshold, a cutoff, a strike, a boundary. DiD, regression discontinuity, instrumental variables.

3. **Adjustment** — statistical controls on observational data: regression, matching, weighting. Necessary, everywhere, and the least trustworthy — because the confounders you did not measure do not politely announce themselves.

The professional failure is inverting the ladder: presenting an adjusted observational estimate with experimental confidence. The honest practitioner states the rung before the result.

61.3 Difference-in-Differences

The workhorse. Two groups, two periods: the treated group's *change* minus the control group's *change* — the second difference removes both the group's permanent level and the period's common shock.

```
import statsmodels.formula.api as smf
did = smf.ols("visits ~ treated * post + member_features",
              data=panel).fit(cov_type="cluster",
                              cov_kwds={"groups": panel["member_id"]})
effect = did.params["treated:post"]      # the DiD estimate
```

The assumption that must be **shown, not asserted**: parallel trends — before the treatment, the two groups moved together. Plot the pre-period lines; if they diverged before the treatment, the method is decorating confounding with arithmetic. Appendix J Case 4's tariff elasticity is this method at full scale, including the messy part real natural experiments refuse to hide.

61.4 Instrumental Variables

When treatment is entangled with the outcome (farms that adopt a programme are the farms that would have done well anyway), you need a variable that moves the treatment but has no other path to the outcome — rainfall moves fertiliser adoption but (arguably) affects yield only through farming inputs. The craft is the two tests you can run (the instrument strongly predicts treatment; the estimate is stable across specifications) and the one you cannot (exclusion — the instrument has no back door), which is why IV results always carry their "arguably" in the text. Weak instruments produce loud, wrong answers: check the first-stage F before trusting the second stage.

61.5 Synthetic Control

For the one treated unit — a district, a province, a flagship store — build the ghost as a weighted blend of untreated neighbours, chosen so the blend tracked the treated unit's past. When the levy hits one province, the synthetic province (weighted combination of its neighbours) continues the old trend, and the gap is the effect. The craft: donor pools that exclude contaminated units, weights that stay interpretable, and placebo checks — run the method on a never-treated unit and see how big a "effect" it fabricates.

61.6 The Case: The National Street Levy

The fictional National Street Levy funds road repairs with a 2% levy on fuel in four pilot provinces. Three years in, the ministry's question: did it reduce vehicle operating costs? The honest analysis produced four estimates, not one:

Method	Comparison	Estimate
Naive before-after	Pilot provinces, pre vs post	-18.4%
DiD	Pilot vs matched non-pilot provinces	-9.1%
DiD + economic controls	Adds fuel-price and freight shifts	-7.6%
Synthetic control	Blended counterfactual provinces	-7.1%

The naive estimate is more than double the most-defensible one — a **2.4x swing across specifications**, and the swing is the finding. The report's headline: "vehicle operating costs fell 7-9% in pilot provinces, with the naive 18% attributable mostly to the national freight downturn that hit both pilot and control provinces." The ministry's press office wanted 18%; the analysis delivered an interval with its reasons — and Chapter 76's audit would have found the 18% indefensible in a week.

61.7 Sensitivity as the Result

The discipline that separates this chapter from a textbook: **the sensitivity analysis is the result**. Report the estimate under every defensible specification, show the spread, and name what drives it. A finding that survives every reasonable specification is a finding; a finding that lives only in one model is a coincidence with a standard error. The one-page robustness table — methods down the rows, choices across the columns, the estimate repeated and its range highlighted — is the frontier chapter's signature artefact, and the one Part XII's memos are built to carry.

Teaching Tip — The anyway report: have students write two reports from the same data: the advocacy report (one estimate, the biggest) and the anyway report (the range, with the assumptions named). Reading them side by side teaches more about analytical honesty than any lecture on causality — and the anyway report is, invariably, the more persuasive document.

Key Takeaways

- Causality is the business question: what would have happened anyway? Every method is a way of building the ghost.
- Trust in order: randomisation, natural experiments, adjustment — and state the rung before the result.
- DiD's parallel trends must be displayed, not asserted; IV's exclusion is argued, never proven; synthetic control needs its placebos.
- The National Street Levy's 2.4x spread across specifications is the norm, not the exception — report the range.

- Sensitivity analysis is the result, not the appendix: one-page robustness table, every time.

Practice Lab

1. Re-run Case 1's promotion analysis as a DiD (email vs non-email members, pre vs post); display the parallel-trends plot and write the paragraph a sceptic deserves.
2. Estimate the street-lighting effect (Chapter 57's data) three ways: before-after, DiD, DiD with controls; build the robustness table and highlight the range.
3. Build a synthetic control for one pilot province's operating costs; run the placebo test on a never-treated province and report the fake effect it fabricates.
4. Find the weak instrument: simulate a first stage with $F = 4$ and watch the second-stage estimate explode; then strengthen the instrument and watch it recover.
5. The anyway report: take any published claim (a news story citing a study) and write the anyway version — what comparison was made, what ghost was assumed, and the range the honest estimate would carry.
6. The Soulfya's menu redesign: design the experiment the founder should have run — randomise by outlet-week, define the metric, pre-register the analysis, and state the sample size needed.

Further Reading

- *Causal Inference: The Mixtape* — Cunningham (free online; the field guide)
- *Mostly Harmless Econometrics* — Angrist and Pischke (the IV chapters)
- Appendix J Cases 1 and 4; Chapter 67 (where the top rung of the ladder is engineered); Chapter 76 (the audit that tests this chapter's habits)

Chapter 62: Geospatial Analytics — Where the Data Lives

Part XI — The Frontier: Advanced Methods and Emerging Practice

"Every row has an address; most analysis pretends it doesn't."

In this chapter you will learn

- Spatial data's two geometries (points and polygons) and the spatial join.
- Distance-based features: buffers, drive times, and nearest-neighbour logic.
- Moran's I: measuring clustering honestly, with a permutation test.
- The Soulflya's fourth-outlet siting study: demand surfaces, cannibalisation, and the gravity model.
- The modifiable areal unit problem and the ecological fallacy.
- Failure modes: the heat map that lies, and choropleths of population.

62.1 Space Is a Column You Forgot

Soulflya's delivery data has a latitude and longitude on every order — two columns the dashboards ignore. Turn them on and the questions multiply: where do orders come from, how far is too far, which neighbourhoods churn on delivery times, and where should the fourth outlet go? Geospatial analytics is not a new discipline so much as a missing feature: **place**, with its own joins, its own statistics, and its own ways to lie.

The two geometries that cover 90% of practice: **points** (customers, orders, outlets, towers) and **polygons** (suburbs, wards, districts, trade areas). The core operation is the **spatial join**: which polygon contains each point, or which point is nearest each point — and once you can do that, place flows into every model you already know.

```
import geopandas as gpd
orders = gpd.GeoDataFrame(orders,
    geometry=gpd.points_from_xy(orders["lon"], orders["lat"]), crs="EPSG:4326")
wards = gpd.read_file("wards.geojson")
orders_by_ward = gpd.sjoin(orders, wards, how="left", predicate="within")
```

From Your Toolkit — Power BI: every map visual you have built was a spatial join the tool did silently — points coloured by a measure, with the polygon boundaries implied. The bridge: when the map is not enough (drive times, buffers, clustering tests), the same data goes to `geopandas`, and the result comes back as a column the dashboard can show.

62.2 Distance Features: Buffers and Drive Times

Straight-line distance is where spatial features begin and where they mislead. The craft: **buffers** (everything within 2 km of an outlet — but a river is not 2 km of road), **drive times** (the honest measure, from a routing service or a road network graph), and **nearest-X** (distance to the nearest competitor, computed once, reused everywhere). Soulfya's delivery data shows the classic result: orders fall off sharply past 15 minutes of drive time, not past any straight-line threshold — the river and the highway decide the trade area, not the compass.

62.3 Moran's I: Is It Clustered, or Does It Just Look Clustered?

Humans see clusters in noise; Moran's I is the test that keeps you honest. The statistic measures whether near things are more alike than far things — spatial autocorrelation — and it is one ratio:

$$I = (n / S_0) * \sum(i,j: w_{ij} * (x_i - \bar{x})(x_j - \bar{x})) / \sum(i: (x_i - \bar{x})^2)$$

Values near +1 mean clustered (similar values adjacent), near 0 mean scattered, negative means dissimilar values adjacent. The interpretation rule that makes it honest: **never read the raw I without its permutation test** — shuffle the values across the map a thousand times, and see how often chance produces an I this large. In the Soulfya's outlet-siting lab, delivery-frequency by suburb yields $I = 0.31$ with a permutation p of 0.004: the clustering is real, and the hot spots in the north and east are where the analysis should look.

62.4 Siting the Fourth Outlet

The part's running study: Azu wants a fourth outlet, and the analytics earn their fee by turning a hunch ("Borrowdale feels busy") into a siting shortlist. The method, in layers:

1. **The demand surface:** kernel density of delivery orders and loyalty visits, weighted by basket value — where does money live on the map?
2. **The competition and cannibalisation layer:** a simple gravity model — expected patronage falls with drive time and rises with outlet attractiveness — to estimate how much of the new outlet's revenue comes from the existing three. A site that "wins" mostly by stealing Avondale's customers is a smaller decision than it looks.
3. **The constraint layer:** rent per square metre, parking, power reliability, staff transport — the Chapter 59 lesson that constraints are the model.
4. **The shortlist memo:** three sites, expected incremental revenue (net of cannibalisation) with intervals, and the pre-committed measurement plan (a control geography for the post-opening evaluation, Chapter 61's ghost).

62.5 Two Classic Traps

- **The modifiable areal unit problem (MAUP)** — aggregate the same points by suburb and the pattern says one thing; by ward and it says another; by 1-km grid and a third. There is no "true" aggregation — so report the one that matches the decision (trade areas for siting; administrative units for compliance) and check that the finding survives the alternatives.
- **The ecological fallacy** — a suburb whose average income is high is not a suburb of rich households; averages over areas describe areas, never the people in them. The analyst's rule: **aggregate for planning, individual data for people-level claims**, and never slide between them silently.

62.6 Failure Modes

- **The heat map that lies** — kernel density of *orders* shows where your current customers are, not where demand is (the whole city orders less from a badly-placed outlet); normalise by population or households before calling anything a demand map.
- **Choropleths of population** — more people means more everything; map *rates*, not counts, or the map is a population map wearing a costume.
- **Precision navels** — coordinates to six decimal places from a phone's GPS are accurate to a city block on a good day; match the precision of the geography to the precision of the decision.
- **The missing base map moment** — analysis that never checks whether the "empty" area is empty because of data gaps (no delivery coverage = no orders recorded), not absence of demand.

Teaching Tip — Two maps, one truth: hand students the same point data and ask for the trade-area map twice — once as straight-line buffers, once as drive-time polygons. The two maps disagree about every border suburb, and the disagreement is the lecture: geography is a model too, and the road network is part of your data whether you loaded it or not.

Key Takeaways

- Space is a feature: spatial joins and distance features bring place into every model you already know.
- Straight-line distance misleads; drive time is the honest currency of trade areas and delivery.
- Moran's I plus its permutation test separates real clustering from pattern-hungry eyes.
- Siting is layered: demand surface, cannibalisation (gravity), constraints, and a pre-committed evaluation plan.
- Aggregate for planning, individual data for people; and always check which aggregation is doing the talking (MAUP).

Practice Lab

1. Build the demand surface for Soulfya's delivery orders; map it per 1,000 households, and mark where the raw-count map and the normalised map disagree most.
2. Compute nearest-competitor and drive-time-to-outlet features for every member; add them to the churn model from Part V and report whether they earn their place.
3. Compute Moran's I for delivery frequency by suburb; run the permutation test and write the two-sentence interpretation a manager can repeat. (The worked arithmetic, for the check: the sum of cross-products is 502.5.)
4. The siting study: build the gravity model, score three candidate sites on incremental revenue net of cannibalisation, and write the shortlist memo with intervals.
5. The MAUP audit: aggregate the same orders by suburb, ward, and 1-km grid; find one finding that changes sign across aggregations and explain what the decision-maker should be told.
6. The missing-base-map check: identify suburbs with zero orders, and determine — from coverage polygons and population — whether they are low demand or no coverage.

Further Reading

- *Geographic Data Science with Python* — Rey, Arribas-Bel and Wolf (free online)
- `geopandas` and `libpysal` documentation and galleries
- Chapter 61 (the evaluation plan every siting deserves), Chapter 59 (constraints as data), Chapter 57 (wards and the public-sector map)

Chapter 63: Survival Analysis — Time Until the Event

Part XI — The Frontier: Advanced Methods and Emerging Practice

"Did they churn?" is the wrong question. "When — and while we watched?" is the truth."

In this chapter you will learn

- Why churn flags lie, and censoring is the honest alternative.
- The Kaplan-Meier estimator: the retention curve, done properly.
- Reading survival curves: medians, quantiles, and the danger of the average.
- Cox regression: hazards, covariates, and the proportional assumption you must check.
- Where survival belongs: churn, machines, patients, farmers, loans.
- Failure modes: immortal time, competing risks, and averaging hazards.

63.1 The Wrong Column

Every churn model you have met (Chapters 27 and 32) ended in a flag: churned, yes or no, at the data freeze. The flag is a convenience that costs the truth three ways: it throws away *when*, it treats a member who left in week 1 the same as one who left in week 51, and it silently drops the members who are simply not yet gone. The honest framing is survival analysis: model **time until the event**, with the event defined in advance and the un-evented members **censored** — observed until they were observed, no longer.

The Soul Circle makes the stakes concrete. "Churned in 60 days" hides the curve's shape: a steep early drop (the trial-and-leave members), a long plateau (the habit members), and a late sag (the defected-to-competitor members). Retention campaigns, CLV arithmetic, and the win-back budget all depend on the shape — not the flag.

63.2 Kaplan-Meier: The Retention Curve, Done Properly

The estimator is one multiplication done in order: at each event time, multiply the survival probability by the share of those still at risk who survived the moment.

```
from lifelines import KaplanMeierFitter
kmf = KaplanMeierFitter()
kmf.fit(durations=members["weeks_active"],
        event_observed=members["churned"])
kmf.plot_survival_function()          # the retention curve, with bands
print(kmf.survival_function_at_times(15))  # S(15)
print(kmf.median_survival_time_)          # the honest "typical lifetime"
```

Reading discipline, in three rules. **Report the quantiles, not the mean** — survival is skewed, and the mean of a censored distribution is a guess about the unobserved future wearing a formula. In the Soul Circle cohort, $S(15) = 0.389$: 38.9% of members remain active at week 15, and the median lifetime falls in the interval (9, 15] — stated as an interval, because that is the resolution the data supports. **Read the curve, not a point** — the early cliff and the late sag are different businesses. **Respect the bands** — where few members remain at risk, the confidence interval widens and the curve becomes suggestion; stop narrating before the data runs out.

63.3 Cox Regression: Hazards with Covariates

The Kaplan-Meier describes one group; the Cox model asks what moves the curve: each covariate's exponentiated coefficient is a **hazard ratio** — how much faster (or slower) the event comes, multiplicatively, for that characteristic.

```
from lifelines import CoxPHFitter
cph = CoxPHFitter()
cph.fit(member_covariates, duration_col="weeks_active",
        event_col="churned")
print(cph.summary[["coef", "exp(coef)", "p"]].sort_values("p"))
# exp(coef) = 1.62 on delivery_share: heavy delivery users churn 62% faster.
```

The assumption to check, always: **proportional hazards** — that the covariate's effect is constant over time. Plot the scaled Schoenfeld residuals; when a covariate's effect fades or flips (a welcome-offer that delays churn for eight weeks and then stops working), the single hazard ratio is an average of two different stories — and the two stories are the finding.

63.4 Where Survival Belongs

The method migrates further than any other in this part:

- **Members and subscribers** — churn curves, CLV inputs (Chapter 52's expected periods, now honest).
- **Machines** — time to failure as a function of age and condition (Chapter 58's maintenance hazard).
- **Patients** — time to readmission (Chapter 54); time to recovery; the curves stratified by ward.
- **Farmers** — time to co-operative exit (Appendix J Case 5's converging curves: the grant *delayed* exit, a different finding from preventing it).
- **Loans** — time to default, which beats a 30-day flag for the same reason churn flags lie (Chapter 53's vintage curves are survival's cousin).
- **Employees** — time to attrition, in the HR analytics you will be asked for eventually.

From Your Toolkit — SPSS and Stata: survival analysis is the classical stats packages' home turf — SPSS's Kaplan-Meier dialogs and Stata's `stset/stcox` taught this method to a generation before Python wrapped it. The bridge is exact: the same estimator, the same log-rank test, now with `lifelines` on the other side of the click.

63.5 Competing Risks, Briefly

Sometimes the event you model is not the only way out. A member can churn — or move cities, or be absorbed into a household account; a patient can be readmitted — or die. Treating every exit as your event biases the curve; treating every other exit as censored biases it differently. The honest-lite discipline: define the competing events, report their shares, and where they are material, model the cumulative incidence for each cause rather than one Kaplan-Meier for all exits combined.

63.6 Failure Modes

- **Immortal time** — members must be alive (active) to receive the "treatment" (the loyalty tier, the clinic programme); the window between enrolment and treatment is time they could not have churned, and counting it inflates the programme's effect. Align the clock at the treatment, always.
- **Averaging hazards** — one hazard ratio over a covariate whose effect changes sign (early retention, late churn) reports a confident nothing.
- **The censored-as-gone error** — treating members lost to follow-up as churned; censoring means "we stopped watching", which is information about the watcher, not the watched.
- **Reading past the data** — narrating the curve's tail where three members remain at risk and the confidence band is a road.

Teaching Tip — Draw the day-30 cliff: give students a raw activity table and ask them to sketch the retention curve *before* computing it. The sketches disagree wildly; then Kaplan-Meier settles it in one line, and the class sees the estimator as the resolution of an argument they were just having. It is the fastest lesson in why the method exists.

Key Takeaways

- Churn flags discard timing and censoring; survival analysis keeps both — and the shape of the curve is the business.
- Kaplan-Meier with confidence bands; report quantiles as intervals ($S(15) = 0.389$, median in (9, 15]), never a false-precision mean.
- Cox gives hazard ratios; proportional hazards must be checked, and its violations are findings.
- The method migrates everywhere: members, machines, patients, farmers, loans, employees.
- Align the clock at treatment (immortal time), define competing risks, and stop reading where the data runs out.

Practice Lab

1. Build the Soul Circle survival curve; report $S(15)$, the median interval, and the early-cliff/plateau/late-sag reading in three sentences a marketing manager can repeat.
2. Fit the Cox model on member covariates; check proportional hazards; find the covariate whose effect changes over time and write the two stories it averages.
3. Stratify the curves by acquisition channel (Chapter 52's cohort logic, deepened): do channels differ in *shape* or only in *level*? Test with the log-rank statistic.
4. The machines: fit time-to-failure on Sabvura's maintenance data (Appendix F); produce the 200-hour hazard window per machine and compare with Chapter 58's schedule.
5. The competing-risks audit: define the Soul Circle's non-churn exits, report their shares, and recompute the churn curve as cumulative incidence; note where the naive curve lied.
6. The immortal-time hunt: take Case 5's grant data and show how the naive comparison (treatment defined at grant end) inflates the effect; align the clock at grant start and report both.

Further Reading

- *Applied Survival Analysis* — Hosmer, Lemeshow and May
- `lifelines` documentation (the best tutorial in the Python ecosystem)
- Chapters 52, 54, 58 (the domains this method serves); Appendix J Case 5 (converging curves, and what they mean for a grant renewal)

Chapter 64: Advanced Forecasting — Hierarchies, Honest Intervals, and Intermittent Demand

Part XI — The Frontier: Advanced Methods and Emerging Practice

"A forecast is a promise with a range; most organisations only hear the point."

In this chapter you will learn

- The reconciliation problem: why the outlet forecasts must add to the chain forecast.
- Bottom-up, top-down, and optimal (MinT) reconciliation — and the one-line idea behind each.
- Bootstrap prediction intervals: uncertainty that survives the real world.
- Croston's method and its successors: forecasting what is ordered rarely.
- Rolling-origin evaluation: the league table, season by season.
- Failure modes: the single-number forecast, the unbalanced hierarchy, and intervals nobody believes.

64.1 The Reconciliation Problem

Soulfya's forecasts the chain, each outlet, and each outlet's categories — and the first week the numbers meet in one spreadsheet, they disagree: the outlet forecasts sum to 112% of the chain forecast, and the category forecasts disagree with both. The organisation has three options, and every organisation with a hierarchy (chain → outlet → category → item) faces them: live with the inconsistency, force one level and lose the others' information, or **reconcile** — adjust all levels so they agree, each level informed by the others.

That is the reconciliation problem, and it is solved not by better point models but by a projection: whatever each level says, move the whole set onto the space of forecasts that sum correctly, weighting by how much you trust each level.

64.2 Bottom-Up, Top-Down, MinT

- **Bottom-up** — forecast the leaves, add them up. Respects local information; amplifies leaf noise; blind to the chain-level signal (the promotion, the macro shock).
- **Top-down** — forecast the top, split by historical shares. Stable; silent about every local pattern (the outlet with a new neighbour).
- **Optimal reconciliation (MinT)** — forecast every level independently, then combine them with weights that reflect each level's error structure, so the result sums correctly and borrows strength across levels. The intuition in one line: **the**

best estimate of any node is a weighted blend of its own forecast and the reconciled forecasts around it — the same shrinkage instinct as Chapter 34's hierarchical means, applied to a forecast tree.

```
# MinT, conceptually: reconcile with an identity-ish covariance
# (full MinT estimates the covariance; the shrinkage variant is robust)
import numpy as np
S = np.array([[1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 1, 0],
              [1, 1, 1, 0]])      # S: which leaves make which nodes
yhat = np.array([210, 180, 150, 600]) # independent base forecasts
P = np.diag([14, 12, 11, 40])      # each level's error variance
reconciled = yhat - P @ S.T @ np.linalg.inv(S @ P @ S.T) @ (S.T.T @ yhat - yhat)
```

In the Soufyia's case, reconciliation moves the chain forecast 4% below the naive sum — because the outlet-level models, each blind to the others, had collectively double-counted the promotion. The hierarchy is a measurement instrument, not just a reporting structure.

64.3 Honest Intervals: The Bootstrap

Point forecasts are half the product; the interval is the half that gets used (stock, staffing, cash). The textbook interval assumes normal, symmetric errors — and restaurant demand is neither. The bootstrap discipline: refit (or re-weight) many times, and read the distribution of the forecasts themselves.

```
boot = []
for b in range(1000):
    sample = residuals[np.random.randint(0, len(residuals), len(residuals))]
    boot.append(point_forecast + np.cumsum(sample[:h])) # h-step paths
lo, hi = np.percentile(np.array(boot), [10, 90], axis=0)
```

The intervals that matter are **asymmetric and widen honestly** — the p90 demand for Saturday dinner is further above the point than the p10 is below it, and the stocking decision lives in that asymmetry. If a forecast product shows one number, it is a rumour; if it shows an honest range, it is a decision instrument.

From Your Toolkit — Excel: the forecast sheet everyone knows produces a line; the bootstrap produces a fan — and the fan is buildable in a spreadsheet (a data table of 200 resampled paths is a classic Excel Monte Carlo). The bridge: the interval is not a specialist add-on; it is the part of the forecast your operations team actually needs.

64.4 Croston: Forecasting What Rarely Happens

Most forecasting assumes demand arrives every period. Spare parts, slow menu items, and intermittent delivery routes do not: weeks of zeros, then an order of 14. Averaging produces "0.8 units per week" — a number that is wrong every week. Croston's method separates the two questions — *how often* (a smooth estimate of the inter-arrival interval) and *how big* (a smooth estimate of the size when it comes) — and the forecast is size-over-interval. Its successors (SBA's bias correction; the more recent intermittent-demand models) improve the edges; the discipline is the separation. The Soulfya's kitchen applies it to slow-moving packaging SKUs, where it cut stock-outs at *lower* inventory — the rare case where the fancy method wins on both axes at once.

64.5 Rolling-Origin Evaluation

One train/test split flatters whatever period you chose. The honest league table (Chapter 28's discipline, extended) rolls the origin: fit to everything through March, forecast April; through April, forecast May; and so on across seasons — so the league table is earned across a year of changing conditions, including the promotions and the shocks.

```
from sklearn.metrics import mean_absolute_error
def mase(actual, forecast, naive):
    return (mean_absolute_error(actual, forecast)
            / mean_absolute_error(actual, naive))
# rolling table: seasonal-naive, ETS, ARIMA, reconciliation – every month, MASE
```

The rule that keeps it honest: **the seasonal-naive baseline appears in every table**, and any method that cannot beat it on the organisation's real loss (stock-out cost, not MAPE) is not deployed, however elegant its mathematics.

64.6 Failure Modes

- **The single-number forecast** — the point estimate without an interval, delivered to a team that will make a stock decision with it.
- **The unbalanced hierarchy** — reconciling chain and outlets but not categories, so the disagreement just moves to the level you ignored.
- **Intervals nobody believes** — bands so narrow they are broken weekly (fit to calm periods) or so wide they are useless (fit to the shock year); the bootstrap on rolling residuals is the cure for both.
- **Averaging intermittent demand** — the 0.8-units-per-week trap; separate frequency and size.

Teaching Tip — Make the sum work: give students three levels of a hierarchy with deliberately inconsistent forecasts and one instruction: "the board will see all three; make them agree, and be able to defend every change." The debate that follows — bottom-up versus top-down versus blend — teaches MinT's motivation better than the matrix algebra ever will, and the reconciliation formula then lands as the resolution of an argument they personally had.

Key Takeaways

- Hierarchies must reconcile: forecast every level, then blend so they agree — MinT formalises what a good analyst does by hand.
- The interval is the product: bootstrap the paths, report asymmetric p10/p90, and let the stocking decision live in the fan.
- Intermittent demand needs Croston's separation: how often, and how big.
- League tables roll: seasonal-naive in every table, evaluated on the organisation's real loss.
- A forecast without a range is a rumour; a range without a baseline is unfalsifiable.

Practice Lab

1. Build the three-level hierarchy for Soulfya's (chain, outlets, categories); reconcile bottom-up, top-down, and MinT-style; report how much each level moved and why.
2. Bootstrap the Saturday-dinner forecast: 1,000 resampled paths, the p10/p50/p90 fan, and the stocking decision the asymmetry implies.
3. Fit Croston (and SBA) on the slow-moving packaging SKUs; compare against the moving average on stock-out rate *and* inventory value — the two-axis win.
4. Run the rolling-origin league: seasonal-naive, ETS, and your model across twelve months; report MASE per month and name the month your model lost and why.
5. The promotion double-count: show how independent outlet forecasts sum to more than the chain signal, and how reconciliation resolves it; write the memo the ops meeting needs.
6. Interval audit: take any forecast your organisation produces; backtest its stated interval over the last year — how often did reality fall outside it? (For an honest 80% interval: about one week in five.)

Further Reading

- *Forecasting: Principles and Practice* — Hyndman and Athanasopoulos (free online; the hierarchical chapters)
- Hyndman et al., "Optimal forecast reconciliation for hierarchical and grouped time series"
- Chapter 28 (the foundations this part extends), Chapter 67 (simulation that consumes these fans), Chapter 58 (where intermittent demand bites)

Chapter 65: Low-Resource Language Analytics — Kumba and the Languages the Internet Forgot

Part XI — The Frontier: Advanced Methods and Emerging Practice

"The models were trained on the internet's languages; your customers were not born on the internet."

In this chapter you will learn

- What "low-resource" means, and why Shona and Ndebele analytics underperform English — by construction.
- Tokenisation and subword vocabularies: why SentencePiece beats word-level for agglutinative languages.
- Transfer learning: borrowing structure from high-resource neighbours.
- The evaluation set you must build by hand — and why it becomes the moat.
- Kumba's stack: transcription, search, and moderation in-language.
- Ethics: data sovereignty, consent, and who owns a corpus.

65.1 The Problem

Kumba (Chapter 60's media company) streams news and radio in Shona and Ndebele to an audience the internet's tools barely see. Off-the-shelf speech recognition mangles Shona names; search works only for users who type English queries; sentiment models return confident nonsense. The reason is not magic: modern language tools are statistical reflections of their training data, and the internet holds perhaps a thousand times more English than Shona. Low-resource does not mean the language is poor in structure — it means the *data infrastructure* is thin: few corpora, few annotators, few benchmarks, few tools that ship with the language preloaded.

The strategic framing every analyst should internalise: **where the tooling is thin, the data you can build becomes the moat**. Kumba's competitive asset is not a model; it is ten thousand hours of transcribed Shona audio and the eval sets no competitor owns.

65.2 Tokenisation: Words Are Not the Unit

English tokenisation (split on spaces) fails hard on agglutinative languages, where one word carries what English spends a sentence on — a Shona verb absorbs subject, tense, object, and mood into a single token. A word-level vocabulary explodes into millions of mostly-unseen entries. The fix is **subword tokenisation**: learn a vocabulary of frequent pieces (SentencePiece, BPE) sized to the corpus, so every unseen word decomposes into known parts.

```
import sentencepiece as spm
spm.SentencePieceTrainer.train(
    input="shona_corpus.txt", model_prefix="shona_sp",
    vocab_size=8000, model_type="bpe",
    character_coverage=1.0)          # every character must be reachable
sp = spm.SentencePieceProcessor(model_file="shona_sp.model")
print(sp.encode("ndakamuida muno", out_type=str))
# ['_nda', 'ka', 'mu', 'ida', '_muno'] - morphemes, not words
```

The craft decisions: vocabulary size matched to corpus size (8k for a million lines, not 50k), `character_coverage=1.0` so diacritics never become unknowns, and **one tokeniser per language** — a Shona-Ndebele hybrid vocabulary serves neither.

65.3 Transfer: Borrowing Structure

You cannot train a large language model from scratch on thin data — but you do not have to. The transfer ladder, cheapest first:

1. **Continue-pretraining** an existing multilingual model on your corpus (further pretraining on Shona text).
2. **Fine-tune** on a small labelled task set (classification, tagging) with aggressive regularisation and early stopping.
3. **Adapters / parameter-efficient tuning** — freeze the base, train a small inserted module; hundreds of thousands of parameters instead of hundreds of millions, which is what a labelled set of 3,000 sentences can actually support.
4. **Related-language transfer** — structure learned on languages with larger corpora and shared roots moves better than structure learned on unrelated giants.

The honesty clause: transfer raises the floor but does not close the gap, and the gap is measurable — which is Section 65.4's entire point.

65.4 The Evaluation Set You Build By Hand

No benchmark exists, so you construct one: 500 sentences annotated for the task (intent, sentiment, named entities), stratified across dialects, registers (radio Shona, street Shona, formal news), and the hard cases (code-switching — the sentence that drifts into English mid-thought — which is not noise but the *dominant register* of real usage).

The discipline that makes it a moat rather than a chore: annotate twice (two speakers, adjudicated disagreements), freeze the set, never train on it, and score every model change against it. **The eval set is the product**: models are replaceable, the measurement instrument is not. When Kumba's transcription vendor claims "30%

better", the claim dies or survives on the frozen set — and the set improves only by deliberate, versioned additions.

65.5 Kumba's Stack

- **Transcription:** a fine-tuned speech model on ten thousand labelled minutes; the eval set is 200 hand-checked hours covering accents, music beds, and code-switching. Word error rate on the eval set, published internally every release.
- **Search:** the untitled archive (Chapter 60) titled in-language — subword tokenisation means a query matches across morphological variants; embeddings fine-tuned on listener behaviour so "murimi" finds the farming programme the listener cannot name.
- **Moderation:** comment and call-in triage in-language — the English model's confidence was the danger (confident nonsense, silently applied); the fine-tuned model is calibrated to admit what it cannot read.

From Your Toolkit — Python: this is the one frontier where only Python reaches — your SQL moves the data, your Power BI monitors the pipelines, but the language work is a Python notebook discipline, end to end. The six-tool grammar holds: the new tool implements an old idea — representation, learned from data — and the bridge names it.

65.6 Ethics: Whose Corpus Is It?

The corpus is communities' speech, gathered under consent that must be specific and revocable; the annotators are speakers whose expertise deserves more than piecework rates; and the models built on a community's language should be auditable by that community. Data sovereignty in practice: provenance recorded per item (Chapter 42's lineage, applied to audio), opt-out honoured by retraining, and the eval sets documented so the community's dialects are visible in the measurement, not averaged into the standard register.

65.7 Failure Modes

- **The English benchmark halo** — "our model is state of the art" on an English-adjacent eval, deployed on Shona; the only number that matters is the one on your frozen set.
- **Tokeniser mismatch** — training data tokenised one way, serving traffic another; the vocabulary must version with the model, together.
- **The code-switching blind spot** — models trained on "pure" text meet real speech mid-drift; stratify the eval set for it or miss the dominant register.
- **Confident nonsense** — the calibrated-to-English confidence scores applied in-language; recalibrate on the in-language eval set before any automated decision.

Teaching Tip — The untranslated search box: put a search interface in front of the class and have Shona-speaking students query it while everyone watches the results. Two minutes of watching real queries fail — then succeed after subword tokenisation — teaches the whole chapter's motivation with no slides at all. Students who have watched a language be ignored by software never forget whose problem this is.

Key Takeaways

- Low-resource is a data-infrastructure condition, not a property of the language — and the corpus you build is the moat.
- Subword tokenisation (one per language, character coverage 1.0) replaces the word-level vocabulary that explodes.
- Transfer the structure (continue-pretrain, fine-tune, adapters), then measure the remaining gap — do not claim it closed.
- The hand-built, frozen, dual-annotated evaluation set is the product; every claim dies or survives on it.
- Consent, provenance, and community auditability are the ethics floor for speech corpora.

Practice Lab

1. Train a SentencePiece model on a Shona (or any low-resource) corpus at two vocabulary sizes; measure out-of-vocabulary rate on held-out text and justify your chosen size.
2. Fine-tune a multilingual model on a 2,000-sentence sentiment task; evaluate on the frozen hand-built set and report the gap to its English performance honestly.
3. Build the eval set: 200 sentences, two annotators, adjudication log, stratification across registers including code-switching; write the versioned documentation page.
4. The search audit: index the archive with word-level and subword tokenisation; run 20 real listener queries against both; report the recall difference.
5. The calibration fix: take the English-calibrated confidence scores, recalibrate on your in-language set, and show the decision-threshold difference it causes.
6. The sovereignty memo: for a corpus of community speech, write the provenance, consent, and opt-out plan — and the paragraph you would show the community's elders.

Further Reading

- Masakhane's published papers (the reference community for African-language NLP)
- *Natural Language Processing with Transformers* — Tunstall, von Wille and Wolf (free online)

- Chapter 60 (Kumba's media business), Chapter 42 (lineage for corpora), Chapter 68 (privacy, when the data is people's voices)

Chapter 66: Simulation I — Queueing and the Utilisation Cliff

Part XI — The Frontier: Advanced Methods and Emerging Practice

"Busy feels like a staffing problem; it is almost always a mathematics problem."

In this chapter you will learn

- The three numbers of every queue: arrival rate, service rate, utilisation.
- The utilisation cliff: why waits explode as you approach 100% busy.
- Little's Law and the queue arithmetic you can do on a napkin.
- Variability's surcharge: Kingman's formula, and why averages lie at the counter.
- Pools versus separate queues, and the staffing memo that settles it.
- Failure modes: the utilisation target that kills, and the average wait nobody lives.

66.1 The Queue You Already Own

Soulfya's Avondale outlet on a Saturday: one till, a steady stream of customers, and a manager who believes the answer is "more staff, obviously". The analytics answer is a piece of arithmetic that says *when* more staff matter, *how much*, and what the current wait is costing — before anyone is hired. Queueing theory is the mathematics of waiting lines, and it is the highest-return-per-formula chapter in this book for anyone who touches operations: restaurants, clinics (Chapter 54), licensing offices (Chapter 57), delivery dispatch, support desks.

66.2 Three Numbers

- **λ (lambda)** — the arrival rate: customers per minute, orders per hour. Estimated from timestamps, and *as a distribution* — Saturday arrivals are not a smooth river; they clump.
- **μ (mu)** — the service rate: customers served per minute per server, from the service-time distribution (and its variance, which matters as much as its mean).
- **ρ (rho)** — the utilisation: $\lambda / (\text{number of servers} \times \mu)$. The share of capacity that is busy. ρ must stay below 1; the interesting question is how far below.

Every queue conversation is three numbers, and the discipline of the chapter is extracting them from the timestamps you already log.

66.3 The Cliff

The single most valuable table in operational analytics — expected waiting time in queue (W_q) for a single-server queue at increasing utilisation, in units of mean service time ($M/M/1$, $W_q = \rho/(\mu - \lambda)$):

Utilisation ρ	Wq in service times	At Soulfya's (mean service 3.0 min)
0.60	1.5	4.5 min (a calm counter)
0.80	4.0	12 min (the line you photograph)
0.90	9.0	27 min (walk-outs begin)
0.95	19.0	57 min (the counter is a rumour)

Read the shape, not the numbers: from 60% to 80% busy, waits double; from 80% to 90%, they double again; the last five points of utilisation cost more than the first fifty. The Saturday problem at Avondale is not "busy"; it is **$\rho = 0.9$ on one till** — and the fix is a second till at peak, which drops ρ to 0.45 for the queue that matters, at a fraction of a full-time hire.

```
def mm1_wq(lam, mu):
    rho = lam / mu
    return rho / (mu - lam) if rho < 1 else float("inf")
# Two tills, pooled (M/M/2): waits fall faster than the naive halving
# because idle minutes on either till cover the other's clumps.
```

66.4 Little's Law and Kingman's Surcharge

Little's Law — $L = \lambda W$ — ties the queue in the room to the wait per customer: see 9 people waiting and you know the wait without a stopwatch. It is the napkin arithmetic of operations (Chapter 54's ward manager, Chapter 57's licensing office), and it is exactly true for any stable queue, however messy.

Kingman's formula is the honest upgrade to the table above: waits grow with *variability* as well as utilisation —

```
Wq ≈ (ca^2 + cs^2)/2 × ρ/(1-ρ) × service_time
```

— where ca is arrival variability and cs is service-time variability. The management implications write themselves: **cut variability, not just add capacity**. A limited menu at peak (smaller cs), batch preparation before the rush, and smoothing arrival clumps (pre-orders, timed pickup slots) all shorten the line without a single new hire — and Chapter 67's simulation exists to test exactly these moves before the Saturday they land.

66.5 Pools versus Separate Queues

One queue feeding several servers beats one line per server — the pooled queue never idles a server while another line grows, and the customer experience (one fair line) is the one people rage about less. The exceptions are real: servers with different speeds (the fast lane behind the slow till), setup differences (the delivery-only counter), and the political economy of priority (clinical triage, Chapter 54). The analytics answer is a simulation run, not a principle — but the prior is pooled until proven otherwise.

66.6 The Staffing Memo

The chapter's deliverable, the one that earns the fee: current λ and μ by hour (from timestamps), current ρ and W_q , two staffing scenarios, and the cost line — expected walk-outs $\times$ basket value against wage minutes, at the p90 Saturday, not the average one. At Avondale: the second till at peak hours costs \$38 per Saturday in wage minutes and removes an estimated 6 walk-outs at a \$19 average basket — \$114 recovered, a 3:1 return before counting the goodwill, which is the part the review scores (Chapter 60's attention logic) eventually bill.

From Your Toolkit — Excel: the entire chapter runs in a spreadsheet — W_q and Little's Law are one cell each, the scenario table is a data table, and the Saturday simulation of Chapter 67 can live beside it. Queueing arithmetic was born for the napkin and the spreadsheet; the manager you are advising already trusts both.

66.7 Failure Modes

- **The utilisation target that kills** — "keep staff 85% utilised" sounds efficient and, at the counter, is a queue policy: 85% on one till is a 20-minute line. Set targets on waits, not utilisation — capacity is the cost, the wait is the harm.
- **The average wait nobody lives** — mean W_q across the day hides the 12:30 spike that drives every review; report by hour, and defend the worst hour.
- **Arrivals as a rate** — the clumps are the message; a Poisson assumption you never check against the clumpy reality (church crowds, payday, the bus arrival) under-states the peak every time.
- **Serving the queue, not the customer** — optimising till throughput while the actual wait is at the kitchen (the system has two queues in series; the long one is the constraint).

Teaching Tip — Live the cliff: run the class as a queue. One "server" rolling dice (mean service 30 seconds), "customers" arriving per a clumpy schedule — first at $\rho = 0.5$, then $\rho = 0.9$. Students time their own waits, plot them, and *feel* the non-linearity before you show the table. No one who has stood in the 0.9 round argues for the 85% utilisation target again.

Key Takeaways

- Every queue is three numbers: λ , μ , ρ — extractable from timestamps you already log.
- Waits explode non-linearly as utilisation approaches 1: the last 5% of busy costs more than the first 50%.
- Little's Law ($L = \lambda W$) is napkin arithmetic; Kingman adds the variability surcharge — cut variation, not just add capacity.
- Pooled queues beat separate lines until proven otherwise; simulation settles the exceptions.

- Staff against the p90 hour, target waits not utilisation, and always find the *longer* queue in the system.

Practice Lab

1. Estimate λ and μ by hour from Soulfya's till timestamps (Appendix F); compute ρ and W_q per hour; find the cliff hours and the utilisation target hiding in the current roster.
2. The staffing memo: two scenarios, wage cost versus walk-out cost, at the p90 hour; write the one-page version the owner signs.
3. Variability audit: measure arrival clumping (variance/mean of counts) and service-time variance; use Kingman to rank three interventions — second till, limited menu, pre-order slots.
4. Pool test: simulate one shared queue versus two tills with separate lines under the measured Saturday pattern; report the wait distribution, not the mean.
5. The series trap: measure the kitchen queue and the till queue separately; identify which one binds at peak and recompute the staffing memo against the true constraint.
6. Little's Law check: count the queue at random moments (L) and the average wait from timestamps (W); verify $L = \lambda W$ and explain any gap as measurement error or non-stationarity.

Further Reading

- *Queueing Methods for Services and Manufacturing* — Hall (the practitioner's reference)
- Little's original paper (1961) — one page, permanent value
- Chapter 67 (simulation that tests these moves), Chapter 54 and 57 (the clinic and the licensing office)

Chapter 67: Simulation II and Experimentation at Scale — Monte Carlo, Bandits, and CUPED

Part XI — The Frontier: Advanced Methods and Emerging Practice

"When the maths gets honest, you stop solving the model and start living in it."

In this chapter you will learn

- Monte Carlo as the analyst's default instrument: the kitchen that runs 10,000 Saturdays.
- Tornado charts and scenario design: finding the variables that own the outcome.
- Online experiments: the A/B discipline, A/A tests, and guardrail metrics.
- Sequential tests and bandits: stopping early without lying to yourself.
- CUPED: cutting noise with pre-period data — the variance-reduction trick behind modern platforms.
- The measurement infrastructure: holdouts, logging, and the decision that owns every test.

67.1 The Monte Carlo Kitchen

The Soulfya's Saturday margin is not a number; it is a system — covers that vary with weather and payday, average basket that drifts with the menu mix, two tills whose waits decide walk-outs (Chapter 66), and a kitchen whose speed degrades when the tickets stack. The deterministic spreadsheet says Saturday earns \$340. The Monte Carlo says more:

```
import numpy as np
runs = 10000
covers = np.maximum(80, np.random.normal(260, 60, runs))
basket = np.random.normal(19.0, 3.2, runs) * (1 + 0.05*np.random.binomial(1,.3,runs))
walkout = np.random.binomial(np.maximum(covers-240, 0), 0.18)
margin = (covers - walkout) * basket * 0.62 - 620
print("mean %.0f | p10 %.0f | P(margin < 0) = %.1f%%"
      % (margin.mean(), np.percentile(margin, 10),
        100*(margin < 0).mean()))
```

The output that changes decisions: **mean \$302, p10 \$170** — and the probability of a loss-making Saturday it makes visible. The mean alone said \$340 and hid the tail; the distribution prices the risk of the new oven, the buffer the cash plan needs, and which Saturday to run the promotion on. Monte Carlo's creed: any number you would put in a memo whose inputs are uncertain deserves a distribution.

67.2 The Tornado: Which Uncertainty Owns the Outcome

Vary each input across its honest range, hold the rest at base, and rank the swings: the tornado chart for the Saturday model shows covers ($\pm\$110$) and basket ($\pm\70) dwarfing kitchen cost ($\pm\$20$). The management lesson writes itself: **measure what moves the outcome** — better cover forecasts (Chapter 64) and price-mix analytics (Chapter 51) pay; renegotiating the paprika supplier does not. Every Monte Carlo ends in a tornado, or it was just a very slow calculator.

67.3 Experiments: The Gold Standard, Engineered

Chapter 61 put randomisation at the top of the credibility ladder; this section engineers it. The running discipline at scale:

1. **One decision per test** — name the decision, the metric, and the owner *before* launch (a test without a decision attached is telemetry).
2. **A/A tests as calibration** — run offer-A versus offer-A monthly; your false-positive rate is *measured*, not assumed, and every "we found a 4% lift" is read against it.
3. **Guardrails** — the metrics that must not move (complaint rate, refund rate, p95 latency); a win that breaks a guardrail is a loss wearing a banner.
4. **Power, computed before** — Chapter 14's discipline at platform scale: the sample size the effect you care about actually needs, or the honest "this test cannot answer that".

67.4 Stopping Early Without Lying: Sequential Tests and Bandits

The classic sin: peeking daily and stopping at the first significant day — which turns a 5% test into a 30% false-positive machine. The honest alternatives: **sequential tests** (spend-the-error-budget designs that permit early looks at pre-set checkpoints) and **bandits** — when the cost of the *loser* is real (every customer shown the worse page), the multi-armed bandit shifts traffic toward the winner as it learns, trading a little certainty for a lot of regret avoided. The rule of thumb that survives contact with business: **fixed-horizon tests for decisions you must certify; bandits for optimisations you can keep running.**

67.5 CUPED: Cutting the Noise

The trick behind modern experimentation platforms: outcomes are noisy, but you often know each user's behaviour *before* the experiment — last quarter's orders, pre-period engagement. **CUPED** (controlled experiment using pre-experiment data) adjusts each user's outcome by their pre-period baseline, and the noise falls with the correlation:

```
theta = np.cov(y, y_pre)[0, 1] / np.var(y_pre)
y_cuped = y - theta * (y_pre - y_pre.mean())
# treatment effect unchanged; standard error falls from 1.64 to 0.27 on the
# Soulfya's delivery-time test – the difference 2.67 detected becomes 2.43
```

| # (shrunk toward its pre-period luck), and detected in days, not weeks.

The numbers to internalise from the Soulfya's delivery-time experiment: unadjusted standard error 1.64 minutes, CUPED-adjusted **0.27** — a six-fold noise cut from a single covariate — and the estimated effect moves from 2.67 to 2.43 minutes as each user's pre-experiment luck is subtracted. That is the quiet power and the quiet danger: CUPED makes small effects visible, and it will also *change* your estimate; report both, and pre-register which is primary.

67.6 The Measurement Infrastructure

Experiments at scale are infrastructure, not heroics: **holdouts** that persist (Chapter 52's rotated 10%), **logging** that lands every exposure and outcome in one queryable place (Chapter 43's drift dataset doing double duty), **a registry** — one row per test: hypothesis, owner, start, end, decision, and the metric movement it produced (Chapter 75's analysis log, platform edition). The registry's dividend: the organisation's true experiment win-rate, the number that tells you whether the idea pipeline is good or merely busy.

From Your Toolkit — Excel and Python: the Monte Carlo kitchen was born in a spreadsheet (Chapter 64's fan is the same discipline), and the CUPED arithmetic fits in one; the platform that runs thousands of tests is Python and SQL. The bridge holds as always: same ideas, different scale.

67.7 Failure Modes

- **The slow calculator** — a Monte Carlo without a tornado, or with input ranges nobody believes: garbage distributions, confidently rendered.
- **The peek** — daily significance checks on a fixed-horizon test; use sequential designs or stop looking.
- **The metric that moved by accident** — fifty metrics, one "significant", a launch celebrated; the pre-registered primary metric is the vaccine.
- **Twelve tests at once** — interactions unmodelled, winners confounding each other; the registry and a simple exposure map keep the portfolio honest.

Teaching Tip — A/A day: dedicate one lab to an A/A test — two identical experiences, "analysed" until something looks significant. Students find the "win" every time, and then the coin-drop: it cannot be real. No lecture on multiple comparisons survives A/A day with anything left to prove.

Key Takeaways

- Any memo number with uncertain inputs deserves a distribution: Monte Carlo it, and mean \$302 / p10 \$170 beats \$340-every-time.
- End every simulation with a tornado: measure what moves the outcome.
- Experiments are engineered: one decision per test, A/A calibration, guardrails, and power computed before launch.

- Sequential tests and bandits let you act early honestly; CUPED cuts noise six-fold with pre-period covariates — and moves your estimate, so report both.
- The registry is the dividend: holdouts, logging, one row per test, and the organisation's true win-rate.

Practice Lab

1. Build the Monte Carlo kitchen on Appendix F's Saturday data; report mean, p10, and P(loss); then build the tornado and name the two variables that own the outcome.
2. Run an A/A test on simulated delivery times: 50 metrics, 20 daily peeks; count the false "wins" and write the lab report your future self needs before the first real launch.
3. Implement CUPED on the delivery-time experiment: verify the SE fall ($1.64 \rightarrow 0.27$), the effect shrink ($2.67 \rightarrow 2.43$), and write the two-column table both numbers deserve.
4. Design the promotion test that Chapter 61 said Soulfya's should have run: power calculation, pre-registered primary metric, guardrails, and the decision rule — all on one page.
5. The bandit simulation: three menu specials, unknown appeal, Thompson sampling versus an equal-split A/B; plot cumulative reward and regret over 30 days.
6. The registry: design the one-row-per-test schema (SQL) for an organisation running dozens of concurrent experiments; add the exposure map that catches interactions.

Further Reading

- *Trustworthy Online Controlled Experiments* — Kohavi, Tang and Xu (the bible; CUPED chapter especially)
- *Algorithms to Live By* — Christian and Griffiths (the explore/exploit chapter, for the intuition)
- Chapter 61 (the ladder this chapter engineers), Chapter 66 (the kitchen's queues), Chapter 43 (the logging everything reads)

Chapter 68: Privacy Engineering — Sharing Data Without Sharing People

Part XI — The Frontier: Advanced Methods and Emerging Practice

"Anonymised" is a promise, not a method; differential privacy is a method.

In this chapter you will learn

- Why anonymisation keeps failing: the re-identification graveyard and the matching game.
- k-anonymity and its cracks: homogeneity and background knowledge.
- Differential privacy's one idea: calibrate noise to the question, with a budget.
- The numbers to feel: what $\epsilon = 1$ versus $\epsilon = 0.1$ does to a count.
- Local versus central DP, and where each belongs.
- The practical programme: minimisation, tiered access, aggregation, and audits.

68.1 From the Floor to the Frontier

Chapter 54 set the privacy floor for the hardest case (health data): minimum necessary, the harm test, small-cell suppression. This chapter builds the floor into a discipline, because the floor keeps failing in a specific, repeatable way: **someone "anonymises" a dataset by deleting names, and a motivated stranger re-identifies it anyway.**

The graveyard is well-populated: the medical records distinguished by the trio of attributes in the classic studies; the "anonymous" browsing histories matched to public social media; the mobility traces where four spatio-temporal points identify most individuals. The pattern every case shares: **the attacker brings their own data** — a voter roll, a social graph, a newspaper's court reports — and matches it to yours. The lesson is structural, not procedural: privacy that depends on what the attacker *doesn't know* is not privacy; it is a waiting list.

68.2 k-anonymity and Its Cracks

The first formal answer: ensure every record is indistinguishable from at least $k-1$ others on every combination of quasi-identifiers (age band, suburb, admission date). It is a real discipline — generalise and suppress until the crowd is thick. But two cracks: **homogeneity** (all k records share the sensitive value, so the crowd protects nothing) and **background knowledge** (the attacker's extra data thins the crowd from outside). l-diversity and t-closeness patch the cracks and open others; the family's honest summary is that k-anonymity is a strong *process* discipline (it forces you to think in crowds) with an unfixable *guarantee* problem (the guarantee depends on the attacker's ignorance).

68.3 Differential Privacy: Calibrated Noise

Differential privacy flips the guarantee: instead of promising the attacker knows little (unprovable), promise that **any single person's presence changes the output by at most a bounded amount, no matter what else the attacker knows**. The mechanism for a count is almost insultingly simple — add random noise scaled to $1/\epsilon$:

```
def dp_count(rows, predicate, epsilon):
    true_n = sum(1 for r in rows if predicate(r))
    scale = 1.0 / epsilon
    return true_n + np.random.laplace(0, scale)
```

And the budget ϵ (epsilon) is the honesty dial — smaller ϵ , more privacy, more noise. The numbers to feel, on a query for a rare subgroup:

Setting	Noise (std $\approx 1.4/\epsilon$)	A true count of 20 comes back as...
$\epsilon = 1.0$	sd ≈ 1.4	18-22 (usually usable)
$\epsilon = 0.1$	sd ≈ 14	6-34 (the signal drowns)

The trade is now explicit and quantitative: **stronger privacy buys noisier answers, and the noise is a price you can compute before you pay it**. The second, subtler idea is the **budget**: each query spends ϵ , and the spend composes — an analyst asking 50 questions is 50 leaks, so the system metering the budget (not the analyst) is the architecture. This is why real deployments wrap a **privacy accountant** around the query layer: per-user, per-day budgets, and answers that degrade loudly rather than silently.

68.4 Local versus Central

- **Central DP** — the curator holds raw data and adds noise to answers. Accurate; requires trusting the curator completely.
- **Local DP** — each user randomises *before* sending (the "coin flip" protocols: answer honestly with probability p , randomly otherwise). No curator to trust; much more noise for the same ϵ — the price of removing trust.

The mapping to products is direct: central DP for a statistics office publishing tables (the census pattern), local DP for telemetry where the collector should never see the raw answer (browser vendors' usage stats). **From Your Toolkit — SQL**: the practical bridge from where you already work — most "DP" work in an organisation starts as *aggregation discipline in the query layer*: pre-built aggregate views, suppression rules, and query logs, which is exactly the plumbing an accountant needs when real budgets arrive.

68.5 The Practical Programme

For the practitioner who will not deploy a DP system this quarter but holds data this afternoon:

1. **Minimise at extract** — the analysis extract carries only the columns the question needs (Chapter 54's floor, now policy).

2. **Tier the access** — raw data to named stewards; aggregates to everyone; row-level access logged and justified, with the log itself audited.
3. **Aggregate by default, suppress by rule** — no cell below 5, complements suppressed with it; make the rule code, not judgement.
4. **Meter the mosaic** — log query patterns per analyst; the re-identification attack is *differential* (many small queries), and the audit that catches it is differential too.
5. **Test like an attacker** — the red-team extract exercise (Chapter 54's lab) run annually; what the strongest internal attacker can join is the real k.
6. **Say what you did** — every release states its protection method and its limits; "anonymised" alone is a word, not a method.

68.6 Failure Modes

- **The zip-code trap** — deleting names and keeping the combination that identifies (birth date + suburb + gender); quasi-identifiers are defined by the attacker's join keys, not your column names.
- **Spending the budget in slices** — "just one more query" fifty times; composition is the attack's best friend and the accountant's whole job.
- **Synthetic data as absolutism** — synthetic datasets generated without formal guarantees leak the rare rows they overfit; synthesis is a modelling discipline (Chapter 61's honesty), not a privacy spell.
- **Utility theatre in reverse** — choosing $\epsilon = 10$ to keep answers clean, which is privacy's equivalent of $p = 0.2$; publish ϵ with every release or the number means nothing.

Teaching Tip — The noisy counter: run a live vote ("who has ever broken a promise?") with the local-DP coin flip, then estimate the true rate as a class. Students watch the honest answer emerge from everyone's noisy one — and the follow-up ("what if we ask each of you 20 questions?") teaches composition as budget arithmetic they have personally spent. Privacy engineering stops being compliance the moment you have felt the dial.

Key Takeaways

- Anonymisation fails because attackers bring their own data; guarantees that depend on their ignorance are waiting lists.
- k-anonymity is good process with an unfixable guarantee; use it as a crowd-thinking discipline, not a promise.
- Differential privacy prices the trade: noise scales $1/\epsilon$ (sd ≈ 1.4 at $\epsilon = 1$; ≈ 14 at $\epsilon = 0.1$), and the budget composes — the accountant must own it.
- Central DP for trusted curators; local DP where trust is what you are removing.
- The programme you can run now: minimise, tier, aggregate with rules, meter the mosaic, red-team annually, and state the method — never just "anonymised".

Practice Lab

1. The re-identification exercise: take a "de-identified" extract and the local voter roll; find three join keys and name the k your extract actually delivers.
2. Implement the DP counter; for a true count of 20, plot the distribution of answers at $\epsilon = 1$ and $\epsilon = 0.1$; write the one-paragraph explanation a data owner can repeat about the dial.
3. The budget meter: build the query-layer accountant (per-analyst daily ϵ , logged); show how 50 small queries exhaust it and what the system does next.
4. Local DP vote: run the coin-flip protocol in class or in code; estimate the true rate with confidence intervals; then double the question count and watch certainty degrade.
5. The suppression rule as code: implement no-cell-below-5 with complement suppression in SQL; document the rule on the release's methodology page.
6. The synthesis audit: generate a synthetic dataset from a rare-heavy table; measure which rows are memorised, and write the honest limits paragraph that ships with it.

Further Reading

- *The Algorithmic Foundations of Differential Privacy* — Dwork and Roth (free online)
- OpenDP and Google's DP library documentation
- Chapter 54 (the floor), Chapter 42 (lineage — privacy's memory), Chapter 68's siblings: 61 (honesty under uncertainty) and 75 (documentation that states the method)

Chapter 69: Analytics Engineering — The Warehouse as a Product

Part XI — The Frontier: Advanced Methods and Emerging Practice

"Your warehouse has users, a roadmap, and bugs; only one of those is on the roadmap."

In this chapter you will learn

- The analytics engineer: the role between the data engineer and the analyst.
- The layer cake: staging, intermediate, marts — and why the order is load-bearing.
- Tests as gates: schema tests, data tests, and the contract with downstream users.
- The workflow: git, pull requests, environments, and CI for queries.
- Naming, grain, and documentation as product decisions.
- Failure modes: the tribal-knowledge warehouse and the notebook that became production.

69.1 The Missing Role

Somewhere between the data engineer (pipes, platforms, uptime) and the analyst (questions, models, decisions) a gap opened, and into it fell everything neither owned: the metric that differs by outlet, the table nobody remembers building, the column named `final_v2_REAL`. The **analytics engineer** is the role that closes the gap: the person who treats the transformation layer — raw data to analysis-ready tables — as a **software product with users**, where the users are analysts, dashboards, and every executive who quotes a number.

Soulfya's data platform by Part VIII had three dashboards quoting three different "revenues" — the fan-out bug of Chapter 6, fossilised into architecture. The analytics engineering discipline is the permanent fix: one definition, versioned, tested, documented, owned.

69.2 The Layer Cake

The discipline's shape, in layers, each with one job:

- **Staging** (raw → cleaned): rename, type, deduplicate, apply the data card. One staging model per source table; no business logic; never bypassed.
- **Intermediate** (cleaned → shaped): joins, business logic, the fiddly middle where fan-out lives and dies. Consumed by marts, never by dashboards.
- **Marts** (shaped → consumed): the star schemas and wide tables of Chapter 8, named for their users — `finance_*`, `marketing_*`, `ops_*` — each with a stated grain and an owner.

The order is load-bearing: every layer consumes only the layer beneath, and every exception (the "quick" dashboard reading raw tables) is the technical debt that Chapter 43's incidents are made of.

```
-- models/marts/ops/outlet_daily.sql (a mart model, dbt-style)
select o.outlet_id, d.date_day,
       sum(f.net_amount)      as revenue,
       count(distinct f.order_id) as orders,
       sum(f.net_amount) / nullif(count(distinct f.order_id), 0) as avg_basket
from {{ ref('stg_orders') }} f
join {{ ref('stg_outlets') }} o using (outlet_id)
join {{ ref('stg_dates') }} d using (date_day)
group by 1, 2
```

From Your Toolkit — SQL: this chapter is SQL's homecoming. The language you learned first becomes the *product language*: versioned in git, reviewed in pull requests, tested in CI, and documented in the warehouse itself. Everything Part II taught as craft becomes, here, infrastructure.

69.3 Tests as Gates

Two families, both code:

- **Schema tests** — the shape of the data: not null, unique, accepted values, referential integrity (every order's outlet exists). Cheap, universal, and they catch the source system's 3 a.m. schema change before the dashboards do.
- **Data tests** — the meaning of the data: revenue non-negative, order counts within 3x of the 28-day median, no outlet-day rows for closed outlets. These encode institutional knowledge that used to live in one analyst's head, and fail loudly when the world changes.

```
models:
  - name: outlet_daily
    columns:
      - name: outlet_id
        tests: [not_null, relationships: {to: ref('stg_outlets'), field: outlet_id}]
      - name: date_day
        tests: [not_null]
    tests:
      - unique_combination: {columns: [outlet_id, date_day]}
      - expression_is_true: {expression: "revenue >= 0"}
```

The discipline that makes tests real: **failures page someone** (Chapter 43's monitoring, applied to tables), and the failure's fix is a pull request, not a silent patch.

69.4 Contracts and Docs

The mart's contract with its users, stated in the model file: grain (one row per outlet-day), freshness (by 06:00), owner (a name, not a team email), and the metric definitions (revenue = net of refunds, including delivery, in USD). The docs generate the catalogue: the page an analyst reads *before* the query, and the page Chapter 76's audit starts from. A metric without a definition page is a rumour with a column name.

69.5 The Workflow

- **Git for everything** — branches, pull requests, and review for transformation code; the query that feeds the board is production software and lives like it.
- **Environments** — dev (your sandbox), staging (the copy), prod (the one the dashboards read); the pipeline promotes, people do not hotfix.
- **CI on every PR** — build the changed models, run the tests, check the docs; a broken definition dies in review, not on a Monday dashboard.
- **Modularity over cleverness** — small models, `ref()`-linked, each testable; the 400-line CTE-monolith is the notebook that became production, wearing a nicer shirt.

69.6 The Craft Decisions

Two conventions carry most of the value. **Naming is documentation:** `fct/dim_` prefixes (facts and dimensions, Chapter 8's star schema made visible), past-tense staging (`stg_orders`, `cleaned`), `no_v2`, `no_final` — version control means never renaming to `final_REAL`. **Grain is stated, not discovered:** every model's file begins with its grain, one sentence, and the reviewer enforces it — because the fan-out bug of Chapter 6 was always a grain statement somebody skipped.

69.7 Failure Modes

- **The tribal-knowledge warehouse** — 4,000 tables, 12 documented, the map in three analysts' heads; the audit question "why does this number differ?" takes a week and a séance.
- **The notebook that became production** — the analysis script someone scheduled in cron, untested, undocumented, now load-bearing; the migration path is the layer cake, one notebook at a time.
- **Test theatre** — hundreds of not-nulls, zero meaning-tests; coverage counted, confidence unearned.
- **The undocumented metric** — `revenue` in two marts, one gross, one net, both believed; the contract and the catalogue are the cure.

Teaching Tip — The broken pipeline drill: hand students a deliberately broken warehouse (a source column renamed, a fan-out reintroduced, a silent null flood) and a user complaint ("the board number is wrong"). The hunt — tests failing, lineage graph, the PR that fixes it — teaches the entire chapter as a lived incident, and the retrospective writes the runbook for them.

Key Takeaways

- The analytics engineer treats transformation as a product: layers, tests, contracts, docs, and users who are analysts.
- The layer cake — staging, intermediate, marts — with each layer consuming only the one beneath; exceptions are debt.

- Tests are code that pages: schema for shape, data for meaning, and the fix is a pull request.
- Naming is documentation, grain is stated in the file, and a metric without a definition page is a rumour.
- Git, PRs, environments, CI — the query behind the board deck is production software and lives like it.

Practice Lab

1. Draw Soulfya's current warehouse as the layer cake; find the three violations (dashboards reading raw, marts reading marts) and write the migration plan, one model at a time.
2. Convert the outlet-daily query into a tested model: five schema tests, three data tests, and the grain statement at the top of the file.
3. Write the contract page for the two "revenue" metrics that caused the three-dashboards problem; define each, name the owner, and mark the deprecated one.
4. Set up CI for the transformation repo: on every PR, build changed models, run tests, and fail on any error; document the workflow for a new analyst.
5. The naming audit: list every `_final`, `_v2`, and `tmp_` table in a warehouse you can access; write the renaming PR and its communication plan.
6. The broken pipeline drill: run it on a classmate's build; write the runbook you wish you had when the board number was wrong.

Further Reading

- dbt's documentation and blog (the discipline's home turf)
- *The Data Warehouse Toolkit* — Kimball and Ross (the dimensional foundations)
- Chapter 8 (star schemas — this chapter is its infrastructure), Chapter 11 (validation — here made permanent), Chapter 43 (the monitoring that pages on test failure)

Chapter 70: Staying Current — A Reading Discipline and Ten Predictions

Part XI — The Frontier: Advanced Methods and Emerging Practice

"The tools will churn; the grammar will not; your job is to know which one a new thing is."

In this chapter you will learn

- The reading discipline: a weekly rhythm that survives a full-time job.
- The triage question: grammar or fashion — and the test that tells them apart.
- Learning by replication: the one-paper-per-week frontier habit.
- Communities, teaching, and the compounding of being slightly early.
- Ten predictions to track — the book's wager on the next five years.
- Failure modes: tool collector syndrome and the frozen practitioner.

70.1 The Churn Is the Job

The half-life of a specific tool is short; the half-life of the discipline is long. You trained on SPSS; the industry moved to Python; Python's libraries reshuffle yearly; and underneath it all, the join, the hypothesis test, the baseline, and the memo have not moved in decades. Staying current is therefore not tool-chasing — it is a **triage discipline**: separating the rare changes to the grammar from the constant churn of fashion, and spending your scarce learning hours accordingly.

70.2 The Reading Rhythm

A rhythm that survives a full-time job (Chapter 45's habit, made specific):

- **Weekly (90 minutes)** — two newsletters you trust (one technical, one industry), skimmed with the triage question; one paper or post *read*, not skimmed.
- **Monthly (half a day)** — the release notes of your core stack, actually read; one tutorial *run*, not watched.
- **Quarterly (a weekend)** — one replication: pick a paper or a new method, rebuild its smallest claim on data you know; the frontier discipline of Part XI's teaching guide.
- **Annually** — the audit: what did you learn this year that changed how you work? (Not "what did you skim?") If the list is empty, the rhythm has become theatre.

The subscriptions matter less than the **decisions they trigger**: every reading session ends with one line — *act*, *watch*, or *drop*. "Watch" items go on a list with a review date; the list is pruned quarterly without mercy.

70.3 The Triage Question

When a new thing announces itself, ask: **what part of the grammar does it change?**

- It changes the grammar if it changes *how questions are answered*: the transition from file-based data to query engines (DuckDB), from hand-built features to learned representations, from point estimates to causal or distributional thinking. These are rare — a few per decade — and worth a deep investment.
- It is fashion if it changes *who or where, but not what*: a new BI tool with the same chart grammar, a new AutoML wrapper around the same models, a new notebook skin. These are learned in a week when needed, never before.

The test that separates them: **can you name a question you could not answer well last year that this answers better?** If yes, replicate the answer on data you know — grammar candidate confirmed or killed in a weekend. If no, file under watch and move on.

From Your Toolkit — all six tools: this is where the six-tool grammar pays its final dividend. SQL, Excel, Python, Power BI, Stata, SPSS — every new tool you will ever meet implements some subset of that grammar, and your fluency in the grammar is what makes a week of learning sufficient. The practitioner who learned *dashboards* can adopt any BI tool; the practitioner who learned *one BI tool* must start over every time.

70.4 Communities and Teaching

You cannot read your way to the frontier alone; the frontier is a conversation. The disciplines: one community you *belong* to (a local meet-up, a professional group, an open-source project), one you *contribute* to (answers, review notes, a small library), and the teaching habit of Chapter 50 — because explaining this year's grammar to someone else is the only reliable test that you learned it. Being slightly early — one replication ahead of your market's demand — compounds quietly: Chapter 80's value ledger has a line for it.

70.5 Ten Predictions

The book's wager — scored annually, in public, as your model of how to hold opinions:

1. **SQL returns as the analytical lingua franca** — engines like DuckDB make "just query the files" the default; the notebook generation learns window functions.
2. **Causal inference becomes a default expectation** — "what would have happened anyway?" moves from the frontier chapter to the standard business question, as experimentation culture spreads.
3. **LLMs become the analyst's pair, not the analyst** — draft memos, first-pass code, synthesis of documentation; the judgement layer (this book) stays human.
4. **The analytics engineer role consolidates** — the layer cake of Chapter 69 becomes a standard organisational function with its own ladder.

5. **Small models hold the middle** — fine-tuned small models (Chapter 65's discipline) beat frontier models on cost, latency, and sovereignty for most production tasks.
6. **Differential privacy gets regulated into deployment** — a privacy budget becomes a compliance artefact, as Chapter 68's accountants ship in mainstream platforms.
7. **Synthetic data grows up** — from demo trick to governed technique, with formal guarantees and honest limits pages.
8. **Local-first analytics persists** — constrained environments (Chapter 77's offline-first world) drive tooling that treats connectivity as a bonus, not a requirement.
9. **Geospatial goes default** — every warehouse grows geometry types, as Chapter 62's missing feature becomes a column everyone has.
10. **The craft premium widens** — as generation gets cheap, the practitioners who price and deliver *judgement* (Part XII) command the premium; the ones who price tool operation do not.

Scorecard discipline: each prediction gets a verdict every year — *hit, miss, or too early* — and the misses are the interesting ones. Predictions are how you force yourself to have falsifiable opinions.

70.6 Failure Modes

- **Tool collector syndrome** — twenty "watch" projects, no replications; the GitHub stars as a substitute for the grammar.
- **The frozen practitioner** — the opposite failure: the stack of five years ago, defended with increasing energy; the triage question asked of nothing, ever.
- **The skimming treadmill** — 100% newsletter coverage, 0% decisions triggered; the rhythm's one-line rule exists to kill this.
- **Prediction without a scorecard** — opinions held publicly but never scored; hold them like Chapter 61 holds estimates, with the range and the reversal condition.

Teaching Tip — The tool autopsy: assign each student a dead tool (or a dying one) — a BI platform, a library, an entire paradigm — and have them present: what grammar did it change, what did it get right, why did it die, and what absorbed its ideas? Nothing teaches triage faster than an autopsy, and nothing inoculates against fashion harder than watching a "revolutionary" become a footnote that still influences everything.

Key Takeaways

- Staying current is triage: grammar changes get deep investment, fashion gets a week when needed — and the grammar you already own makes that possible.
- The rhythm: weekly reads with a decision, monthly runs, quarterly replications, annual audit.

- The test: name the question this answers better, then replicate it on data you know.
- Communities and teaching compound; being one replication early is a career asset.
- Ten predictions, scored annually in public — falsifiable opinions, held honestly.

Practice Lab

1. Build your reading system: choose the two newsletters, the paper cadence, and the watch-list with review dates; run it for a month and write the audit of what changed.
2. Run the triage test on the last three tools you heard about: for each, the question it answers better, the grammar-or-fashion verdict, and the act/watch/drop decision.
3. One replication: pick a paper from this part (CUPED, MinT, Croston, or a Masakhane model), rebuild its smallest claim on Appendix F data, and write the two-page replication note.
4. Join and contribute: one community joined, one contribution made (an answer, a review, a fix); write the paragraph on what teaching taught you.
5. Score the ten predictions for this year: hit, miss, or too early — with one sentence of evidence each; then write your own prediction to add to next year's card.
6. The tool autopsy: research one dead analytics tool and present its grammar legacy in five slides; name which of its ideas you use weekly without knowing their origin.

Further Reading

- Your two newsletters — chosen, triaged, and actually read
- Papers With Code (the replication habit's home)
- Chapter 50 (teaching as the test), Chapter 45 (the career habit), Chapter 80 (where being early gets priced)

PART XII

The Practitioner's Path: Projects, Clients and the Craft of Delivery

Chapters 71 - 80

Chapter 71: The Anatomy of an Analytics Project

Part XII — The Practitioner's Path: Projects, Clients and the Craft of Delivery

"Projects fail at the beginning, at the middle, and at the end — in that order of preventability."

In this chapter you will learn

- The engagement arc: scope, plan, analyse, deliver — and the gate between each.
- The analysis plan: one page that prevents a month of drift.
- The stranger test: the quality gate every artefact passes before it ships.
- Mid-project reviews: the structured checkpoint that catches drift early.
- The deliverable ladder: memo, deck, dashboard, handover — choosing the right rung.
- Failure modes: scope creep, the endless exploration, and the vanishing analyst.

71.1 The Arc

Every analytics engagement — internal or consulting, two weeks or two quarters — runs the same four phases, and the phases are separated by **gates**: explicit moments where the work is reviewed against its plan before it continues.

Phase	Output	Gate to next phase
Scope	The brief: question, decision, data, owner, date	Sponsor signs the one-pager
Plan	The analysis plan: methods, data pulls, artefacts, timeline	Plan reviewed against the brief
Analyse	Findings, models, checked assumptions	Mid-project reviews at 1/3 and 2/3
Deliver	Memo/deck/dashboard, handover, measurement plan	Stranger test + sponsor read-out

The arc's discipline is that **the gates are real**: skipping the scope gate is how three-week projects become three-month explorations; skipping the review gates is how projects deliver a beautiful answer to a question nobody asked.

71.2 The Analysis Plan

One page, written before the analysis, containing:

- **The question and the decision it serves** (Chapter 1's discipline, in writing)
- **The metric and its baseline** — what number, compared to what
- **Data sources and known gaps** — the data card, referenced

- **Methods, in order** — and the order is the argument: descriptive first, causal where the question demands, models where they pay
- **Artefacts and owners** — memo, dashboard, model card; who writes each
- **Timeline with review dates** — the mid-project reviews booked at the start, not found at the end

The plan's value is not prescience — plans change — but **drift detection**: when week three's work does not serve the plan, either the work or the plan is wrong, and the plan makes the conflict visible. Appendix J's five cases all include their analysis plans for exactly this reason: the plan is where the evaluator looks first.

71.3 The Stranger Test

The quality gate for every artefact: **hand it to a stranger — someone with the relevant job but no context — and ask them to state the decision it recommends and why**. If they cannot, in under a minute for the executive summary, the artefact is not done. The test catches the practitioner's occupational disease: the curse of knowledge, where the context in your head substitutes for the context on the page. Run it before the sponsor sees anything; the stranger's confusion is cheap, the sponsor's is expensive.

71.4 Mid-Project Reviews

At roughly one-third and two-thirds of the timeline, a structured checkpoint (the same format as Chapter 50's studio reviews, professional edition):

1. **Two strengths** — what is working and should be protected.
2. **Two risks** — what could sink the project, stated with evidence.
3. **One recommendation** — the single change that most improves the outcome.

The review's rule: the analyst presents *work in progress, including the ugly parts* — because the review's entire value is catching drift while it is still cheap. Receiving critique without defending (Chapter 50's hardest skill) is a professional requirement here, not a classroom exercise: the review is against the project, not the person.

71.5 The Deliverable Ladder

Chapter 49's product ladder, delivery-side: the **memo** (the default — Chapter 74's structures), the **deck** (when the room is the medium), the **dashboard** (when the question repeats), the **handover** (when the artefact must live without you: code, docs, the analysis log, the owner named). The delivery failure is stopping at the wrong rung — a dashboard for a one-time decision, a memo for a weekly ritual — and the discipline is Chapter 49's question: *how many people, how often, making which decision?*

From Your Toolkit — Excel: the analysis plan, the review schedule, and the deliverable checklist run happily in a spreadsheet — and they should, because the project's operational artifacts must live where the team works, not in a tool the project outlives.

71.6 Failure Modes

- **Scope creep** — the brief that quietly grows ("while you're in there..."); the gate is the sponsor's signature on the one-pager, and every addition is a re-scope conversation, not a favour.
- **The endless exploration** — "one more data source" for six weeks; the plan's timeline is the alarm, and the review is where it rings.
- **The vanishing analyst** — heads-down for a month, emerging with a 60-page report; the no-surprises rule (Chapter 74) exists because silence reads as risk.
- **The unfinished finish** — findings delivered, nothing measured; every project ends with its measurement plan: what will be watched, by whom, for how long, to learn whether the recommendation worked.

Teaching Tip — The plan autopsy: give students a failed project (any real one, names removed, or Appendix J Case 3's intern) and have them write the analysis plan that would have prevented the failure. Then — the twist — have them write the plan's drift log: the three moments the real project departed from it. Students learn that plans fail by drift, not by being wrong, and that the gate's job is to make drift loud.

Key Takeaways

- Four phases, real gates: scope, plan, analyse, deliver — with the sponsor's signature between each.
- The one-page analysis plan is drift detection: when work and plan diverge, one of them is wrong, loudly.
- The stranger test gates every artefact: a stranger states the decision in a minute or it is not done.
- Reviews at one-third and two-thirds, work-in-progress included; critique without defence is the professional baseline.
- End every project with its measurement plan — the recommendation is a hypothesis until it is watched.

Practice Lab

1. Write the analysis plan for Case 1 (the promotion post-mortem): question, metric, baseline, methods in order, artefacts, timeline, review dates — one page.
2. Run the stranger test on your last deliverable; log every question the stranger needed to ask — each one is context that was in your head instead of on the page.
3. Run a mid-project review on a live piece of work (yours or a classmate's): two strengths, two risks, one recommendation, and the drift log against the plan.
4. The deliverable ladder decision: take three recent requests in your organisation and place each on the ladder rung it deserved — then note what was actually delivered and what the gap cost.

5. The scope-creep rehearsal: script the conversation when the sponsor adds "just one more cut" at week two — practice saying the re-scope sentence out loud until it is comfortable.
6. Write the measurement plan for your capstone: what gets watched, by whom, for how long, and what result would prove the recommendation wrong.

Further Reading

- *The Consulting Bible* — Weiss (the engagement mechanics, secularised)
- Appendix J (five projects, each with a plan to audit)
- Chapter 49 (the ladder from the product side), Chapter 74 (the memo's structures), Chapter 75 (the log that survives the project)

Chapter 72: Scoping and the Options Paper

Part XII — The Practitioner's Path: Projects, Clients and the Craft of Delivery

"The most valuable page you will write is the one that talks the client out of the project."

In this chapter you will learn

- Scoping as diagnosis: what is actually being asked, and what would answer it.
- The options paper: the do-nothing option, and pricing them all honestly.
- Pricing models: fixed, day-rate, retainer, value — and when each is right.
- The refusal clause: the engagements you decline, on purpose.
- Credit, correct, extend: taking over existing work without burning it.
- Failure modes: the yes-scoped project and the free pilot that never ends.

72.1 Scoping Is Diagnosis

Before the project is a project, it is a request — and requests arrive as solutions. "We need a churn model" is a diagnosis the client has already made; the scoping conversation's job is to walk it back to the symptom (win-back spend is rising and conversion is falling), then to the decision (who to call, this week, with what offer), and only then ask whether a model answers it. The tools of the conversation are Chapter 73's question ladder, and its output is the brief Chapter 71 gates. The scoping rule: **you are not selling the analysis; you are diagnosing the decision** — and the analysis is the treatment plan that survives the diagnosis.

72.2 The Options Paper

The signature artefact of honest scoping: one page, four options, priced — including the one nobody prices.

Option	What it is	Co st	Answers the question?	When it is right
Do nothing	Keep deciding as today	\$0	The status quo, honestly stated	When the decision is rare or low-stakes
The quick answer	A day of analysis on existing data	\$4 00	Often, surprisingly	When the data already holds the answer
The full project	The engagement as requested	\$4, 00 0	Thoroughly, with intervals	When the decision recurs or the stakes justify
The product	Build the repeating artefact	\$1 5,0 00	Forever, if it is maintained	When the question is weekly and the answer is stable

The do-nothing option is the paper's spine: it prices the status quo's risks and forces the comparison that matters — not "project versus blank", but **project versus the world as it is**. Clients respect the paper that can talk them out of it; and the practitioner who writes it is the practitioner who gets asked back, because the paper proves the advice is not self-serving. Case 3's lesson lives here: the intern's optimisation was *incomplete*, not wrong — the options-paper discipline is credit, correct, extend, with the pricing of each step stated up front.

72.3 Pricing the Work

Four models, and the conditions under which each is honest:

- **Fixed price** — when the scope is genuinely known (rare) and the analysis plan is firm. Clear for the client; all the risk sits with you; every scope wobble must re-open the contract, which keeps scope honest.
- **Day rate** — when the work is exploratory and the scope will move. Honest about uncertainty; requires the discipline of the weekly note so the client sees value arriving.
- **Retainer** — when the questions are recurring and the relationship is the product (Chapter 79's service catalogue). Predictable for both; needs an explicit definition of what is *not* included.
- **Value-based** — a share of the measured outcome, when the outcome is measurable and attributable (Chapter 80's value ledger makes this possible). The boldest and the most dangerous: it demands the measurement plan be written before the pricing.

The floor under all four: **your rate is a function of the decision's value, not your effort** — and the options paper is where the client sees that arithmetic, which is why it comes before the pricing conversation, not after.

72.4 The Refusal Clause

The engagements you decline, written down so the deciding is policy rather than courage:

- **No decision attached** — "we just want to understand the data better" with no decision, owner, or date: it will eat a quarter and end in a folder.
- **The answer is predetermined** — the analysis is decoration for a choice already made; you are renting your credibility.
- **Unfixable data, unspoken** — when the scoping shows the data cannot answer the question, and the client wants to proceed anyway, the honest refusal is the service.
- **Harm** — the use that fails Chapter 29's harm test (targeting the vulnerable, surveillance, discrimination). Refusing this work is the credential that makes every other engagement trustworthy.

The refusal is delivered with a path: what *would* answer the question, who else could do it, what to fix first. The clause's quiet economics: the engagements you decline are the margin that lets you do the good ones properly.

72.5 Taking Over Existing Work

The practitioner inherits: a half-built dashboard, a predecessor's model, the intern's optimisation (Case 3). The discipline that keeps both the work and the relationships: **credit, correct, extend**.

1. **Credit** — name what the prior work got right, in writing, to the people who own it.
2. **Correct** — fix what is wrong as a scoped, visible change, not a silent rewrite.
3. **Extend** — build the new thing on the corrected base, so the lineage (and the credit) survives into the final artefact.

The alternative — the incoming expert's silent rewrite — is how organisations lose their memory every eighteen months, and how consultants earn their reputations for arrogance. The graveyard of prior work is also a graveyard of trust.

72.6 Failure Modes

- **The yes-scoped project** — agreement on everything except the decision, metric, and date; six weeks of harmony ending in a disappointed read-out. The gate is the brief's signature, every time.
- **The free pilot that never ends** — the "just show us something" week that becomes an unpaid engagement; the options paper is the vaccine, and the pilot is Option 2, priced.
- **The undersold quick answer** — the day of work that was worth the full price; value-based thinking protects the practitioner's own side of the ledger too.
- **Scope as favours** — each small addition free, the sum of them a different project; re-scope is a conversation, not a confrontation.

Teaching Tip — Price the do-nothing: give students a request (a real one, anonymised) and fifteen minutes to write the options paper — then score the papers on one criterion only: could the client choose Option 1 and feel well served? The papers where do-nothing is a real option, honestly priced, are the papers that would win the work — and students discover that the credibility sale and the honest sale are the same sale.

Key Takeaways

- Scoping is diagnosis: walk the request back from solution to symptom to decision.
- The options paper prices four worlds, including do-nothing — and its credibility is that it can talk the client out of the project.
- Pricing models are honesty structures: fixed for firm scope, day-rate for moving scope, retainer for recurrence, value for measurability.
- The refusal clause is policy, not courage — and always delivered with a path.
- Inherit by credit, correct, extend: the lineage of work is the lineage of trust.

Practice Lab

1. Write the options paper for Case 4's tariff brief (the two-week evaluation): all four options, priced, with the do-nothing option's risks stated in the client's own numbers.
2. Take a real request from your organisation; write the scoping conversation's five questions (Chapter 73's ladder) and the brief you expect them to produce.
3. Price the same engagement three ways — fixed, day-rate, value-based — and write the paragraph explaining which you would choose and what each model assumes.
4. Draft your refusal clause: the four declinations, each with its delivered path; rehearse one refusal out loud until it sounds like service rather than rejection.
5. The inheritance drill: take Case 3's intern's optimisation and write the credit-correct-extend memo — three paragraphs, in order, that the board could read aloud.
6. The scope-creep ledger: track one engagement's additions for a month (requested, granted, re-scoped); compute what the "favours" cost at your day rate.

Further Reading

- *The Win Without Pitching Manifesto* — Enns (the scoping conversation's rhetoric)
- *Managing the Professional Services Firm* — Maister (the pricing economics)
- Appendix J Case 3 (the inheritance) and Case 4 (the narrow brief, honoured)
- Chapter 71 (the gate the brief passes), Chapter 73 (the conversation), Chapter 80 (the value ledger that value-pricing requires)

Chapter 73: Working with Stakeholders — The Question Ladder

Part XII — The Practitioner's Path: Projects, Clients and the Craft of Delivery

"The person closest to the work knows the variable your model is missing."

In this chapter you will learn

- The stakeholder map: who decides, who uses, who knows, who blocks.
- The question ladder: five questions that turn a request into a brief.
- The listening tour: structured interviews before the data.
- The person closest to the work: front-line knowledge as a data source.
- Managing the room: expectations, weekly notes, and the no-surprises rule.
- Failure modes: the sponsor who vanished and the stakeholder you never met.

73.1 Four Kinds of People

Every engagement has four roles, and confusing them is how good analysis dies in bad meetings:

- **The decision-maker** — the one whose behaviour the analysis should change; often not the one who requested it.
- **The user** — who will live with the artefact (the dashboard's daily driver, the model's operator); their workflow is a requirement.
- **The knower** — the front-line staff, the data steward, the twenty-year veteran; your best data source, and the most commonly skipped interview.
- **The blocker** — finance, legal, the union rep (Case 3), whoever holds a veto; found early, a constraint; found late, a casualty.

The map is drawn in week one, on one page, and it names names. **From Your Toolkit — Power BI:** the stakeholder map outlives the meeting, and the dashboard's usage telemetry (who actually opens it, how often) is the map redrawn by reality — Chapter 19's audience discipline, checked against Chapter 43's logs.

73.2 The Question Ladder

Five questions, in order, that convert a request into a brief. Each rung produces a sentence of the brief (Chapter 71's one-pager):

1. **"What will you do differently once you know this?"** — the decision. If there is no answer, Chapter 72's refusal clause is on the table; if the answer is "present it to the board", keep climbing.
2. **"What would make you change your mind?"** — the falsifier: the finding that would reverse the decision. It defines the analysis's honest target.

3. **"What did you decide last time, and what did you use?"** — the organisation's real evidence standard, as practised rather than professed.
4. **"Who will disagree, and what will they point at?"** — the objection and its data; the analysis addresses it *in the document*, not in the meeting.
5. **"If I could only give you one number on Friday, what number?"** — the metric. Everything else is scaffolding for it.

The ladder's discipline is the order: decision before data, every time — and the sponsor who cannot climb rung one is a scoping conversation, not a project.

73.3 The Listening Tour

Before the data, the people: six to ten structured interviews, thirty minutes, the ladder's questions adapted per role. The tour's outputs, written into the brief:

- The decision, in the decision-maker's own words (Chapter 74 will quote it back to them)
- The known data problems, from the people who live with them (the register that is "mostly right", the outlet that double-enters on Saturdays)
- The history — the last analysis on this question, and why it did or did not stick
- The vocabulary — the organisation's own terms for its metrics, which your artefacts must use or deliberately translate

From Your Toolkit — SPSS: the interview notes are survey data you gathered by hand — code them like it (thematic buckets, counts, the dissenting view preserved), and the brief's "what we heard" section writes itself with evidence instead of impressions.

73.4 The Person Closest to the Work

Case 3's union rep is the part's patron saint: one sentence about loading order — the constraint the optimiser missed, the variable that reversed two routes. The discipline: **front-line knowledge is data**, gathered and cited like data. The till operator knows the 16:00 rush's true shape; the nurse knows which register is fiction; the driver knows which road the map does not believe. Interview them early, quote them in the analysis (with permission), and check the model's behaviour against their account — when the model and the front line disagree, one of them is teaching you something, and it is usually the model being taught.

73.5 Managing the Room

- **The weekly note** — five lines every Friday: what we did, what we found, what is next, what we need, decisions pending. The no-surprises rule's mechanism: the sponsor never learns a headline in the read-out they have not seen in a note.
- **Expectations as commitments** — "the draft is Thursday 10:00" is a promise with a name on it; slipped dates are announced before they pass, never after.
- **Disagreement engineered early** — the ladder's rung four finds the objector; bring them the interim finding before the final, so the objection arrives while it can improve the work instead of ambushing it.

- **The meeting is not the medium** — the artefact carries the analysis; the meeting carries the decision. Never present findings for the first time in the meeting where they must be decided (the no-surprises rule again, from the other side).

73.6 Failure Modes

- **The sponsor who vanished** — no gate signatures, no weekly reads; the project ends with a full report and an empty room. The gates exist to make the absence visible in week two, not week twelve.
- **The stakeholder you never met** — the knower or blocker skipped for speed; Case 3's route plan, signed off by everyone except the person who knew the loading rule.
- **The echo chamber** — interviews with the leadership team only; the front line's reality never enters the brief, and the analysis beautifully answers the question as misunderstood.
- **Rung-skipping** — data before decision: the ladder climbed from the bottom, producing a technically superb answer to the wrong question.

Teaching Tip — The silent stakeholder: run a stakeholder role-play with one rule — one participant receives a private brief with a critical fact (the loading order, the broken register) and may not volunteer it; they only answer questions asked. Groups that interview poorly ship a flawed plan and feel the cost; groups that ask "what does your day actually look like?" get the fact and win. The exercise teaches the ladder better than the ladder does.

Key Takeaways

- Map four roles — decision-maker, user, knower, blocker — with names, in week one.
- The question ladder turns requests into briefs: decision, falsifier, precedent, objection, the one number.
- The listening tour is data collection: code it, cite it, and use the organisation's own vocabulary.
- The person closest to the work knows the missing variable; treat front-line knowledge as data.
- The weekly note makes no-surprises mechanical, and disagreements are engineered to arrive early.

Practice Lab

1. Draw the stakeholder map for Case 5 (the co-operative evaluation): decision-maker, users, knowers, blockers — and note which of them the case's analyst actually needed.
2. Run the question ladder on a real request in your organisation; write the five brief-sentences it produces, and note where the climbing stalled.

3. Conduct three listening-tour interviews (colleagues playing roles, or real ones); code the notes into themes and write the "what we heard" section with counts.
4. The front-line check: take a live model or dashboard and validate it against a front-line account; document every disagreement and which party was right.
5. Draft the weekly note for a project you know (or Case 3, week two): five lines, no surprises, one pending decision.
6. Run the silent-stakeholder role-play; write the two interview questions that would have extracted the private fact, and keep them for the rest of your career.

Further Reading

- *Data Science for Business* — Provost and Fawcett (the stakeholder framing chapters)
- *Crucial Conversations* — Patterson et al. (when the ladder gets hard)
- Appendix J Case 3 (the union rep) and Case 5 (two audiences, one memo)
- Chapter 72 (the brief this chapter produces), Chapter 74 (the vocabulary in the room), Chapter 78 (the trust ledger it all compounds into)

Chapter 74: Storytelling for Decision-Makers

Part XII — The Practitioner's Path: Projects, Clients and the Craft of Delivery

"The analysis is done when the decision is made, not when the deck is polished."

In this chapter you will learn

- Three structures that carry any finding: the pyramid, the arc, and the memo.
- The executive summary: the one paragraph that survives if nothing else does.
- Charts in the room: one message per slide, stated as its title.
- The hostile-question protocol: preparing for the three hardest questions.
- The no-surprises rule: pre-briefing as a delivery discipline.
- Failure modes: the deck as artefact, the buried lede, and the defensive answer.

74.1 Three Structures

Every finding this book has produced — a model, a forecast, a case — ships in one of three shapes:

- **The pyramid (Minto)** — the answer first, then the three reasons, then the evidence for each. The structure of the executive read-out: the decision-maker can stop at any level and leave informed. *Recommendation* → *reasons* → *evidence*, never the reverse.
- **The arc** — context, complication, resolution. The structure of the narrative memo and the presentation: the world as it was, what changed or broke, what we now do. The arc earns attention; the pyramid spends it.
- **The memo** — the one-pager: BLUF (bottom line up front), the three supporting facts with their numbers, the risks and what would reverse the recommendation, the ask. The structure of organisations; the rarest and most valuable habit.

The practitioner's rule: **the same analysis, told three ways, or it was never done** — the memo for the record, the pyramid for the room, the arc for the room's attention. Chapter 60's coach-chairman-journalist exercise, professional edition.

74.2 The Executive Summary

One paragraph, written last, that contains: the decision it recommends, the single strongest number behind it, the honest uncertainty, and the ask. Test it with Chapter 71's stranger standard: a reader with the relevant job but zero context states the recommendation and its reason after one reading. The summary's discipline is what it *omits* — every method, every caveat that does not change the decision, every number that is not load-bearing. The buried lede (findings on page 14) is the summary written by an analyst protecting themselves; the summary exists to protect the reader.

74.3 Charts in the Room

Chapter 17's craft rules, presentation edition:

- **The slide title is the takeaway** — "Churn paused, then resumed: the offer delayed, not prevented", not "Survival curves by cohort". The deck skims as a story if someone reads only the titles — and that is how it will be read.
- **One chart per slide, one message per chart** — the dual-axis monstrosity is two slides hiding in one.
- **The number, named** — every number carries its comparison: "12 minutes" is nothing; "12 minutes, versus 4 last quarter" is a finding.
- **Appendix everything** — the method, the sensitivity tables, the data card: in the deck's back pages, one hyperlink away, shown only when asked (and the asking is a good sign).

From Your Toolkit — Power BI: the dashboard is not the deck. The dashboard answers the repeating question; the deck argues the one-time decision — and the commonest craft error is presenting a dashboard walkthrough when the room needs an argument. Pull the one exhibit the argument needs; leave the filters behind.

74.4 The Hostile-Question Protocol

The read-out's real work is the Q&A, and it is prepared, not improvised:

1. **Write the three hardest questions** — the ones you hope nobody asks, asked by the smartest sceptic in the room (Chapter 50's students meet them in the capstone; Chapter 61's "anyway" sceptic is their patron).
2. **Answer each in one breath, then stop** — "Good question — the honest answer is we cannot rule that out; here is what we did to bound it" beats a two-minute deflection that concedes the room.
3. **Bridge, don't dodge** — "That's the data question; the decision question it feeds is..." returns the room to the decision without dismissing the asker.
4. **Know your one "I don't know"** — the answer you will give honestly, with a commitment ("by Friday"), because credibility is the asset the Q&A defends — and one honest "I don't know" buys more than three confident guesses.

The protocol's quiet secret: **preparing the hard questions improves the analysis.** The question you cannot answer in one breath is usually the analysis's weak spot —

better found in preparation than in the room.

74.5 The No-Surprises Rule

Nobody in the decision meeting learns a headline for the first time. The stakeholders who will speak (the sponsor, the objector of Chapter 73's rung four, the budget-holder) have seen the finding in the weekly note or a pre-brief; the meeting decides on a shared understanding instead of staging a reveal. The rule's logic is risk, not manners: surprises trigger status responses (defend, defer, derail) that consume the meeting; shared understanding triggers the decision. The reveal-culture deck is the analyst seeking applause; the no-surprises read-out is the practitioner seeking the signature.

74.6 Failure Modes

- **The deck as artefact** — the deck polished past the analysis, the medium become the work; the pyramid test (read only the titles) finds it in minutes.
- **The buried lede** — the finding on page 14, wrapped in method; the executive summary exists to prevent exactly this.
- **The defensive answer** — the two-minute rebuttal that concedes the room; one breath, bridge, or the honest "I don't know".
- **The presentation of the dashboard** — filters in a room that needed an argument; rung discipline again (Chapter 49, 71).

Teaching Tip — The three audiences, one analysis: take one finding (Appendix F plants several) and have students deliver it as the memo, the three-slide pyramid, and the ninety-second arc — then face the hostile-question round (you ask the three hardest). Grading rubric: does each version survive its stranger test, and did any answer run past one breath? Students discover that the constraint is not the analysis but the respect for the audience's minute — which is the entire chapter.

Key Takeaways

- Three structures, one analysis: memo for the record, pyramid for the room, arc for attention.
- The executive summary: decision, number, uncertainty, ask — and the discipline of omission.
- Slide titles are the story; one chart, one message; the method lives in the appendix.
- Hostile questions are prepared: the three hardest, one breath each, and the honest "I don't know".
- No surprises in the decision room: pre-brief the speakers; the meeting decides, it does not reveal.

Practice Lab

1. Take Case 1's promotion finding and write it three ways: the one-paragraph memo, the three-slide pyramid, the ninety-second arc. Run the stranger test on each.
2. The title audit: take your last deck and rewrite every title as the takeaway; read only the titles aloud — does the deck tell the story? If not, the slides were labels, not messages.
3. Write the three hardest questions for your capstone (or Case 4's tariff memo); answer each in one breath, on paper, and identify which question exposed a real weakness in the analysis — then fix the analysis.
4. Run the hostile-question round with a partner playing the smartest sceptic; count your breaths; log every defensive answer and rewrite it as bridge-plus-commitment.
5. The pre-brief plan for a fictional read-out: who sees the finding before the meeting, in what form, and what each is asked not to do (reveal, relitigate, reframe).
6. The number audit: list every number in one deliverable; for each, its comparison and what decision it carries — cut the numbers that carry none.

Further Reading

- *The Pyramid Principle* — Minto (the structure, from its source)
- *Resonate* — Nancy Duarte (the arc, for presentations)
- *The Craft of Research* — Booth, Colomb and Williams (the memo's discipline)
- Chapter 17 (chart craft), Chapter 20 (storytelling foundations), Chapter 73 (the room you are speaking into), Appendix J (five memos to autopsy)

Chapter 75: Documentation That Outlives You

Part XII — The Practitioner's Path: Projects, Clients and the Craft of Delivery

"The analysis you cannot reconstruct is a rumour you happened to compute."

In this chapter you will learn

- The four artefacts: the README, the analysis log, the methodology note, the handover memo.
- The analysis log: a dated record of decisions and dead ends, kept as you work.
- Reproducibility as habit: seeds, environments, and data snapshots.
- Where documentation lives, and why "in the same repo" is the whole answer.
- The doc-debt audit, run quarterly.
- Failure modes: the tribal-knowledge project and the retrospective novel.

75.1 Four Artefacts

Every engagement produces four documents, and three of them are written *during* the work, not after:

- **The README** — the project's front door: the question, the decision it served, the data sources, how to run the code, and where everything else lives. Two minutes to read; the test is that a stranger can start work from it.
- **The analysis log** — the dated, running record: what was tried, what was decided, what died and why. The part's signature artefact; Section 75.2.
- **The methodology note** — the how-it-was-done: definitions, assumptions, the sensitivity table (Chapter 61), the limitations stated without shame. The document Chapter 76's audit reads first.
- **The handover memo** — the end-of-engagement document: what was delivered, what was not, what to watch (the measurement plan), who owns it now, and the three things the next practitioner should know.

75.2 The Analysis Log

The discipline that separates the practitioners from the tourists. The log is a plain file (markdown, one per project), appended as work happens, in four kinds of entry:

```
2026-03-04 Chose 30-day churn window (not 60). Reason: retention team
           calls at day 21; the window must match the intervention.
2026-03-06 DEAD END: outlet-level features in the member model. Leakage
           suspicion correct? No - dropped for grain mismatch, not leakage.
2026-03-09 Review 1: sponsor flagged price-sensitivity question. Added
           elasticity bound (ch61 method) to plan; memo updated.
```

2026-03-11 Data issue: 2022 gap is 78% missing, MAR on plot size.
Strategy: MI + IPW bound (case 5 pattern). Logged for note.

The log's value is exactly the entries that feel wasteful: **the dead ends**. Six months later, the dead end re-suggested by a colleague costs a week to re-die — unless the log says it already died, with its reason. The log is also Chapter 71's drift detector made continuous, and the raw material the methodology note is distilled from. Write it at the end of the day, every day; five minutes buys the project's memory.

75.3 Reproducibility as Habit

- **Seed everything** — every script that samples, splits, or initialises takes an explicit seed, recorded in the log; "the numbers moved slightly" is a bug report, not a mystery.
- **Environment as file** — `requirements.txt`, pinned; the environment is part of the artefact, versioned beside the code.
- **Data snapshot as data card** — the extract date, source, row counts, and checksums; Chapter 42's lineage, made physical. The analysis reproduces against *the data it analysed*, with today's data as a documented refresh.
- **One command to rerun** — the README's promise: `make all` (or its equivalent) rebuilds every table, figure, and number in the memo. If it cannot, the memo is a claim, not an artefact.

From Your Toolkit — SQL: commented queries *are* documentation when they live in the repo — the craft habit is writing the query as its own methodology note (grain stated at the top, assumptions inline, the why beside the what). The uncommented 200-line query is the tribal-knowledge project in miniature.

75.4 Where It Lives

One answer: **in the repository, beside the code, versioned with the work**. Not in a documents drive, not in email, not in the wiki that the last person who updated it left in 2023. The README points to everything; the log and the note travel with the analysis they describe; the handover memo is the last commit's companion. The "where does documentation live?" question is answered by the same discipline that answers "where does code live?" — and the answer's power is that search, diff, and blame then work on the documentation too.

75.5 The Doc-Debt Audit

Quarterly, one hour, per project: the four artefacts exist and pass their tests?

Artefact	Test	Fail = debt item
README	Stranger starts work in ten minutes	Rewrite front door
Analysis log	Last week's decisions findable	Backfill from memory (costly, do now)
Methodology note	A peer could defend it to an auditor	Distill from log
Handover	Owner named, watch-items dated	Write before the exit, not after

The audit's rule: debt items get owners and dates — undocumented is a state with a repair plan, not a permanent condition.

75.6 Failure Modes

- **The tribal-knowledge project** — everything worked and only the departed analyst knows why; the four artefacts are the vaccine, and the audit is the booster.
- **The retrospective novel** — documentation written in a heroic weekend *after* the work: no dead ends (forgotten), no real reasons (reconstructed), no trust. Three of the four artefacts exist because they are written *during*.
- **Documentation as monument** — the 60-page methodology note nobody reads; the note is distilled, the log is complete, and confusing the two wastes both.
- **The moving target** — numbers quoted in the memo that the pipeline no longer produces; the snapshot card and the one-command rerun keep memo and pipeline married.

Teaching Tip — The two-years-later test: hand students a project bundle (code, data card, and either a real log or a fabricated gap) and give them one question to answer from it ("why did we drop the outlet features?"). The bundle with the log answers in thirty seconds; the bundle without burns the hour — and the hour *is* the lesson: documentation is time travel, and the person you are doing the favour for is you.

Key Takeaways

- Four artefacts, three written during: README (front door), log (memory), methodology note (defence), handover (continuity).
- The analysis log's value is its dead ends — five minutes a day buys the project's memory and the note's raw material.
- Reproducibility is habit: seeds, pinned environments, snapshot cards, and one command to rerun everything.
- Documentation lives in the repo, versioned with the work — search, diff, and blame for prose too.
- Audit the debt quarterly: each artefact with its test, each failure with an owner and a date.

Practice Lab

1. Start an analysis log for your current project; write today's entry including one dead end you have already forgotten the reason for (reconstruct it, and note the cost of the reconstruction).
2. Write the README for a project you have finished: front door test — hand it to a classmate and watch where they stall.
3. The reproducibility pass: pin the environment, seed every random step, snapshot the data card, and get to one command; document the gap you cannot close and

why.

4. Distill a methodology note from an analysis log (yours or Appendix F's planted log): definitions, assumptions, the sensitivity table, limitations without shame.
5. Run the doc-debt audit on two projects; produce the debt list with owners and dates.
6. The handover memo for Case 5's evaluation: what was delivered, what to watch, who owns it, and the three things the next evaluator must know.

Further Reading

- *The Art of Doing Science and Engineering* — Hamming (the log-keeping habit's pedigree)
- Reproducible research literature (the "one command" tradition)
- Chapter 71 (the plan the log audits), Chapter 42 (lineage), Chapter 76 (the audit that reads your note)

Chapter 76: Audits and Due Diligence — Ten Questions

Part XII — The Practitioner's Path: Projects, Clients and the Craft of Delivery

"Every model is innocent until probed; the probe is ten questions long."

In this chapter you will learn

- The audit mindset: health check, not inquisition — and the report that helps.
- The ten questions, with what each catches and what a good answer sounds like.
- Due diligence on someone else's model: the vendor version of the same probe.
- The findings report: ranked, evidenced, with effort and fix.
- Red flags that end audits early.
- Failure modes: the checkbox audit and the audit as warfare.

76.1 The Audit Mindset

You will be audited (your models, your dashboards, your numbers) and you will audit (inherited work, a vendor's claims, a predecessor's pipeline). Both go better with the same framing: **a health check that helps** — ranked findings, each with its evidence and its fix, delivered to make the work better rather than the auditor look smart. The audit that helps gets invited back; the audit that performs gets performed at.

The ten questions are the probe. They are ordered by the failure frequency of real practice — the top three catch most of what matters.

76.2 The Ten Questions

1. **What is the grain?** — one row of this table means what, exactly? The unanswerable version is the fan-out bug (Chapter 6) waiting to be rediscovered. Good answer: a sentence, instantly.
2. **Where did this data come from, and what was done to it?** — lineage from source to table (Chapter 42). Good answer: the data card plus the transformation trail, produced without a séance.
3. **How was the evaluation split, and is there leakage?** — time-based or grouped splits, the leakage checklist (Chapter 22), the definitional leaks Case 2 exposed. Good answer: "we split by member and by time; here are the three features we removed and why."
4. **What is the baseline?** — the seasonal-naïve, majority-class, last-year comparison (Chapter 23). Good answer: the league table, model and baseline side by side. No answer: the finding that the model never beat the cheap alternative.
5. **Is it calibrated where it must be?** — thresholds and rationing decisions need calibrated risk (Chapter 25, 54). Good answer: the calibration curve by decile.

- 6. **How sensitive is the conclusion?** — the robustness table of Chapter 61: specifications down the rows, the estimate's range highlighted. Good answer: a range with its reasons. Bad answer: one number, defended.
- 7. **What is the privacy posture?** — minimisation, tiers, suppression, the harm test (Chapters 54, 68); consent and provenance for people-data. Good answer: the stated method, named limits.
- 8. **How is it monitored, and what has drifted?** — the PSI discipline, the drift dataset, the champion-challenger rhythm (Chapters 43, 53). Good answer: recent monitoring output, including something it caught.
- 9. **Who owns it now, and what breaks first?** — the named owner, the failure modes, the runbook (Chapters 43, 69). Good answer: an owner, an alert, a rehearsal.
- 10. **What would make you withdraw it?** — the pre-registered reversal condition (Chapter 71's measurement plan, sharpened). Good answer: specific, already written. No answer: work that has never considered its own death.

76.3 Due Diligence, the Vendor Edition

The same probe, pointed outward, with three additions:

- **The claims audit** — map every claim in the pitch ("30% better", "real-time") to its measurement: measured how, on what data, against what baseline, by whom? The unmeasurable claim is a wish.
- **The eval-set question** — Chapter 65's discipline, outward: whose evaluation, frozen when, annotated by whom? Vendors who cannot show their eval set cannot show their number.
- **The exit test** — if the vendor vanishes, what survives? Data export formats, model artefact portability, the documented schema. The answer prices the lock-in.

From Your Toolkit — Stata: the audit trail is the econometric tradition's gift — the do-file that runs top to bottom, producing every table in the output, is the audit question 3, 4, and 6 answered in artefact form. Demand the same of everything: the do-file, the notebook that runs clean, the pipeline with one command (Chapter 75).

76.4 The Findings Report

One page, ranked:

R an k	Finding	Evidence	Fix	Ef fo rt
1	Churn target leaks via nightly status field	Feature availability timeline	Remove 3 features, refit; AUC 0.97 → expected ~0.80	2 da ys
2	No baseline in evaluation	Report lacks league table	Add seasonal-naive comparison	1 da y

R an k	Finding	Evidence	Fix	Ef fo rt
3	Calibration untested for threshold use	No calibration output	Calibrate by decile; recalibrate quarterly	2 da ys

The report's rules: every finding carries its **evidence** (the artefact that proves it), its **fix**, and its **effort** — because the reader's next act is prioritisation, and findings without effort are anxiety, not aid. Severity is ranked by decision-risk, not technical elegance: the leak that flatters the model outranks the ugly code that works.

76.5 Red Flags

Findings that end audits early, because everything downstream is suspect:

- **The number that cannot be reproduced** — no seeds, no snapshot, "it was on the old laptop" (Chapter 75's moving target).
- **The evaluation that improved when it should not have** — accuracy up after a data change nobody can explain: leakage until proven otherwise (Case 2's law).
- **The refusal to show the data card** — the lineage question answered with adjectives ("clean", "comprehensive") instead of artefacts.
- **No owner** — question 9 answered with a team email address; the system is already an orphan.

76.6 Failure Modes

- **The checkbox audit** — ten questions, ten "yes"s, zero artefacts requested; the audit as ritual. Every yes should be *shown*, not said.
- **Audit as warfare** — findings deployed for politics; the audit that helps is the audit that gets told the truth next time.
- **The novelty bias** — findings about code style while the calibration failure (question 5) goes unasked; the probe's order is the shield.
- **The audited who performs** — preparing theatre for the audit instead of fixing the work; the artefact-yes rule makes theatre expensive.

Teaching Tip — Audit swap: pair students; each audits the other's capstone with the ten questions, artefacts required for every yes. The debrief's question is not "what did you find?" but "what did you *fix* before handing it over?" — because the audit's approach changes how work is *built*: students start keeping logs, pinning environments, and writing reversal conditions when they know the probe is coming. The audit's greatest value is the work it prevents you from doing badly.

Key Takeaways

- Audit as health check: ten questions, ranked findings, each with evidence, fix, and effort.

- The top of the probe catches the most: grain, lineage, and leakage — asked first, artefacts required.
- Questions 4–6 are the honesty core: baseline, calibration, sensitivity — the league table, the curve, the range.
- DD on vendors adds the claims audit, the eval-set question, and the exit test.
- Red flags end audits early: unreproducible numbers, unexplained improvements, adjective lineages, orphan systems.

Practice Lab

1. Audit a model you built (Part V's churn, or any Kaggle submission) with the ten questions; write the findings report with evidence, fix, and effort for every failure you find in your own work.
2. Audit swap: exchange audits with a classmate; rank the severity of each other's findings by decision-risk and compare the rankings — where they differ is a lesson about what matters.
3. The vendor probe: take a published AI product's claims page; map each claim to its measurement and write the questions the sales call needs.
4. Question 10's test: take three live artefacts in your organisation and find whether any states its reversal condition; write the conditions they lack.
5. The red-flag drill: fabricate one red flag (a planted leak or an unreproducible number) in a classmate's audit bundle and see whether the probe catches it; debrief on which question did the catching.
6. The DD memo: Case 2's v2 model is a vendor pitch ("AUC 0.97, deploy Monday") — write the two-page due-diligence memo to Azu.

Further Reading

- *The Flaw of Averages* — Savage (why unexamined numbers mislead)
- Model cards literature (Mitchell et al.) — the artefacts question 10 looks for
- Chapter 22 (leakage — the probe's favourite catch), Chapter 43 (monitoring), Chapter 75 (the artefacts every yes must show)

Chapter 77: The Constrained Practitioner — The \$200 Stack and the Offline-First World

Part XII — The Practitioner's Path: Projects, Clients and the Craft of Delivery

"Constraint is not the absence of tooling; it is a design brief."

In this chapter you will learn

- The \$200 stack: what a serious analytics practice runs on when budget is not the point.
- Offline-first: designing for connectivity as a bonus, not a requirement.
- The data realities of constrained environments, and the methods that respect them.
- Lightweight pipelines: from USSD menus to paper registers with dignity.
- When constraint produces better design, and when it just hurts.
- Failure modes: tool-shaming and the unacknowledged cloud bill.

77.1 The Stack

The premise: a laptop, intermittent power and connectivity, and a \$200 budget that covers everything else. The stack that runs a real practice under it:

Layer	Tool	Cost
Compute	Any laptop from the last six years, 8 GB RAM	(owned)
Storage/analysis	Python + pandas + DuckDB — query files at a million rows a second	\$0
Databases	SQLite for operational, DuckDB for analytical, Postgres when a server exists	\$0
Statistics	statsmodels, scipy, lifelines, or Stata/SPSS you already own	\$0
Dashboards	Power BI Desktop, or Metabase on a local server, or Plotly HTML exports	\$0
The spreadsheet	Excel/LibreOffice — the organisation's lingua franca, respected (Chapter 5)	\$0
Versioning	Git, local; pushed when connectivity allows	\$0
The \$200	A UPS (uninterruptible power supply) for charge-through outages, a 4G dongle for the good hours, and a backup drive	~\$200

The point is not austerity theatre. The point: **every method in this book runs on this stack** — the star schemas, the churn models, the causal evaluations, the queueing arithmetic, the Monte Carlo kitchens. DuckDB alone retires most "we need a big data platform" conversations (Chapter 37's decision test, applied to a budget).

77.2 Offline-First

Connectivity as bonus, not requirement — the design discipline of environments where the network is intermittent, metered, or absent:

- **Batch, don't stream** — the pipeline that syncs when a connection appears (and resumes cleanly when it vanishes mid-sync) beats the real-time design that dies at the first outage.
- **Local-first artefacts** — the dashboard that is an HTML file on a memory stick still works when the cloud does not; the Power BI file with a scheduled refresh beats the live connection that is only live sometimes.
- **The connectivity budget** — treat bandwidth like spend: the sync queue is prioritised (aggregates first, raw second), the big pull waits for the good hours, and every tool's "phoning home" is audited.
- **Paper as interface, not shame** — the register that is photographed and keyed (Chapter 54) with validation gates is an offline-first *input device* that happens to be made of paper; design the transcription pipeline, respect the operator, and reconcile the gap (Chapter 58's whiteboard discipline).

77.3 Methods That Respect Constraint

The constrained environment is not a smaller version of the cloud world; it has its own best practices:

- **Sample where you cannot census** — Chapter 15's discipline becomes operational: a well-drawn sample analysed honestly beats an ambition of "all the data" that never lands.
- **Rules before models** — Chapter 46's stack, reordered by budget: the rules catch the blunt patterns explainably, and the model earns its complexity only where rules leave money on the table.
- **Resampling over infrastructure** — the bootstrap (Chapters 15, 64) delivers honest intervals on a laptop; the cluster does it faster, later.
- **Small models, kept current** — a refit logistic regression on last quarter's data beats a giant model frozen two years ago; the constrained practitioner's monitoring cadence is the envy of the over-tooled (Chapter 43's drift discipline, budget edition).

From Your Toolkit — all six: the constrained stack is where the six-tool grammar proves it was never about the tools — SQL in DuckDB, Excel as the interface everyone shares, Python for the models, Power BI for the room, Stata's caution and SPSS's rigour for the inference. The practitioner fluent in the grammar is employable at every budget level; the practitioner fluent in one platform is employable at one.

77.4 When Constraint Helps

Constraint earns its keep when it forces the disciplines the abundant environment postpones: the data card (because every extract costs an hour), the baseline (because complexity is expensive), the sample (because census is impossible), the honest interval (because the over-fit model cannot be retrained weekly). The constrained practitioner writes better methodology notes (Chapter 75) because the constraints are visible in them. But honesty cuts both ways: **constraint that costs the analysis its honesty is not character-building, it is harm** — the survey that cuts corners on sampling is wrong at any budget, and Chapter 72's refusal clause applies to engagements whose constraints guarantee bad answers.

77.5 The Unacknowledged Cloud Bill

The mirror failure: the organisation that believes it is constrained while its "free" tools bill by the query. The egress charge (Chapter 41), the per-seat BI licence, the per-API-call model endpoint — the constrained practitioner audits the *total* cost of ownership, including the free tier's cliff, and often discovers the \$200 stack was not the cheap alternative but the honest one. The audit is annual: what does each tool cost at this organisation's actual usage, in the currency the finance team uses?

77.6 Failure Modes

- **Tool-shaming** — the practitioner who dismisses the spreadsheet-based analysis because it is not "real data science"; Chapter 50's rule (respect for prior knowledge) applies to tools: the analysis that answers the question at the budget available is the real one.
- **Cloud cosplay** — the Kubernetes cluster for a 40 MB dataset; Chapter 37's decision test, violated at altitude.
- **The offline afterthought** — designs that assume connectivity and "degrade" (die) without it; offline-first is a design input, not an error message.
- **Austerity as identity** — refusing the tool that would genuinely pay (Chapter 70's triage) because the stack is a matter of pride; the constraint is a brief, not a religion.

Teaching Tip — The offline week: run one lab week with the wifi off (or, more gently, a rule that all cloud calls are banned). Students discover the local stack's speed, the sync-when-possible pipeline they must design, and — the real lesson — which parts of their practice were quietly renting infrastructure they never needed. The debrief's question: "what did you *not* miss?"

Key Takeaways

- The \$200 stack runs every method in this book: DuckDB + Python + the spreadsheet + your rigour.
- Offline-first is a design discipline: batch, local-first artefacts, a connectivity budget, and paper as a respected interface.

- Constraint has its own best practices: sample, rules before models, resampling, small models kept current.
- Constraint helps when it forces honesty (data cards, baselines, intervals) and harms when it guarantees bad answers — the refusal clause applies.
- Audit the total cost of ownership: the free tier's cliff is a bill; the \$200 stack is often the honest accounting.

Practice Lab

1. Build the \$200 stack on your machine: DuckDB querying a year of Soulfya's generated data (Appendix F) in files, no server; time the queries and write the memo retiring the "we need a cluster" conversation.
2. Design the offline-first sync: a pipeline that pulls when connected, resumes after interruption, and prioritises aggregates; test it by pulling the plug mid-sync (literally).
3. The paper pipeline: photograph a page of handwritten register, key it, and run Chapter 11's validation gates; write the reconciliation report the register's owner deserves.
4. The sampling exercise: estimate a population parameter from a 5% sample with honest intervals; compare against the census in the generator's key; write the two sentences that defend the sample to a sceptic.
5. The bill audit: cost out your current toolset at your organisation's real usage — including the free tier's cliff — and present the comparison against the \$200 stack in the finance team's currency.
6. The rules-first build: replace a model in Chapter 46's fraud stack with five explainable rules; measure what the rules catch, what they miss, and the model's incremental value at the budget.

Further Reading

- DuckDB's documentation (the constrained stack's engine room)
- *Small Data* — the argument this chapter makes, from the research side
- Chapter 37 (the decision test), Chapter 41 (cloud economics), Chapter 5 (the spreadsheet, respected)

Chapter 78: Your First Ninety Days — Quick Wins and the Trust Ledger

Part XII — The Practitioner's Path: Projects, Clients and the Craft of Delivery

"You were hired for your skills; you will be re-hired by your trust balance."

In this chapter you will learn

- The ninety-day arc: listen, deliver, establish — in that order.
- The listening tour, professional edition: the question ladder as an onboarding instrument.
- The quick win: small, visible, decision-linked, finished — and what it is not.
- The trust ledger: deposits, withdrawals, and the balance you actually manage.
- The first dashboard you do not build, and the metrics you do not volunteer.
- Failure modes: the rookie consultant and the permanent guest.

78.1 The Arc

Whether the ninety days are your first job, a new client, or an internal transfer, the arc is the same:

Days	Phase	The work	The output
1-30	Listen	The tour, the map, the reading	The brief for your own first project
31-60	Deliver	The quick win, done properly	One decision measurably improved
61-90	Establish	The second project, the systems, the rhythm	The practice's foundations (Chapter 79)

The arc's discipline is its order: **listen before delivering, deliver before establishing** — and the rookie error is delivering in week one (proving competence) at the cost of listening (learning what competence means here).

78.2 The Listening Tour, Onboarding Edition

Chapter 73's ladder, repurposed: six to ten interviews in the first month, each ending in the five questions — plus three that belong to onboarding:

- **"What did the last analyst do that you want kept?"** — the inheritance question (Chapter 72's credit, before any correction).
- **"What analysis exists that nobody trusts?"** — the map of graves; each one is either a fixable lineage problem or a lesson in why it died.
- **"If I fixed one number's credibility, which number?"** — the organisation's pain, in its own words; often the quick win's origin.

The tour's discipline is the write-up: the notes are coded (Chapter 73's SPSS habit), the map is drawn (stakeholders, systems, graves), and the brief for days 31–60 emerges from evidence rather than from whatever the loudest meeting requested.

78.3 The Quick Win

Days 31–60 deliver one thing, chosen by four criteria:

1. **Small** — a fortnight's work, not a quarter's; scope that survives contact with the calendar.
2. **Visible** — attached to a decision the organisation knows it makes (the number nobody trusts, fixed; the report that took days, automated honestly).
3. **Decision-linked** — it changes a decision, not a dashboard count (Chapter 1's rule, now yours).
4. **Finished** — delivered with the handover artefacts (Chapter 75), the measurement plan (Chapter 71), and its place in the trust ledger.

The quick win is *not*: the data warehouse rebuild, the ML platform evaluation, the "let's first get all the data clean" project. Those are the second-ninety-days conversations — and the quick win's job is to buy the standing to have them.

78.4 The Trust Ledger

The mental model that governs all ninety days: an account per relationship (sponsor, colleague, operator, auditor), with **deposits** (delivered promises, honest "I don't know"s, credit shared publicly, the no-surprises rule kept) and **withdrawals** (missed dates, surprised stakeholders, credit quietly absorbed, defensiveness in reviews).

The ledger's rules:

- **The balance is what you spend** — the second project's scope, the difficult finding's reception, and the refusal (Chapter 72) are all priced from the balance.
- **Deposits compound small** — the Friday five-line note, the fixed number, the public credit: none is dramatic; together they are the practice.
- **Withdrawals are lumpy** — one missed date after three quiet weeks costs more than it seems; the account remembers.
- **Balances are asymmetric by role** — the operator's trust is earned in the trenches (their register respected, their knowledge cited); the sponsor's in the gates (signed briefs, no surprises); audit them separately.

From Your Toolkit — Excel: keep the actual ledger — a spreadsheet of relationships, dates, deposits, withdrawals, and the balance read monthly — for the first year. It feels clinical for a month, then becomes the most useful "meeting notes" you have ever kept: the practice's accounts receivable, in the only currency organisations actually pay in.

78.5 The Restraint List

What you deliberately do not do in ninety days:

- **The dashboard you were asked for in week one** — "build us a dashboard like the last place's" is a request wearing a solution costume (Chapter 72); the ladder goes up first, and the dashboard that survives the questions is worth ten that precede them.
- **The metrics you volunteer** — the vanity metric offered to seem busy (Chapter 19's discipline, self-applied); every metric you own is a promise to maintain it.
- **The rewrite of the predecessor's work** — Chapter 72's credit-correct-extend; the graveyard analysis often contains the organisation's memory, and the newcomer who burns it spends the next year as the suspect.
- **The opinion on everything** — the first weeks' hot takes are withdrawals with no deposits behind them; the tour's questions are free, its judgements are not.

78.6 Failure Modes

- **The rookie consultant** — delivers week one, listens never: competent artefacts answering the wrong questions, a trust ledger overdrawn by the first surprised stakeholder.
- **The permanent guest** — listens forever, delivers nothing: the tour that becomes tourism; day 60 is the quick win's due date, and the calendar is the cure.
- **The win that wasn't visible** — brilliant work on a decision nobody knew they made; the quick win's second criterion (visible) is not optional.
- **The borrowed credit** — the analysis that quietly absorbs the operator's twenty years of knowledge (Chapter 73's knower) and presents it as the analyst's insight; the ledger's fastest withdrawal.

Teaching Tip — The trust ledger, graded: in the final weeks of the course, have students role-play the first ninety days: three stakeholder meetings (one sponsor, one operator, one sceptic) with decisions to make about promises, credit, and a discovered error in the predecessor's work. Grade the ledger, not the script — the write-up of deposits and withdrawals, and the balance's trajectory, is the deliverable. Students report it is the most uncomfortable and most memorable exercise of Part XII: the maths of trust, made visible.

Key Takeaways

- The arc is ordered: listen (1–30), deliver (31–60), establish (61–90) — and reversing it is the rookie error.
- The onboarding tour adds three questions: what to keep, what is not trusted, which number to fix.
- The quick win is small, visible, decision-linked, and finished — never the platform rebuild.
- The trust ledger is the real account: small compounding deposits, lumpy withdrawals, a balance per relationship, audited monthly.

- The restraint list is as strategic as the to-do list: no premature dashboards, no volunteered metrics, no graveyard-burning, no opinions on everything.

Practice Lab

1. Write your own ninety-day plan for a real or plausible role: the tour's interview list, the three onboarding questions per interview, the quick-win criteria applied to three candidates, and the chosen one with its fortnight scope.
2. Start the actual trust ledger (the spreadsheet): every stakeholder relationship in your current world, the balance you estimate today, and one deposit scheduled for this week.
3. The quick-win selection drill: take three candidate wins and score each against the four criteria; write the rejection notes for the two you did not choose (they are next quarter's pipeline).
4. The restraint rehearsal: script the week-one dashboard request and your ladder response; practise the sentence that says "yes, in three weeks, after five questions" until it is comfortable.
5. The inheritance memo: for a real predecessor artefact (or Case 3's optimisation), write the credit-correct-extend plan and the paragraph that keeps the operator's knowledge credited.
6. The retrospective: audit a past "first ninety days" (a job, a course, a project) against the arc; write the two withdrawals you would intercept if you could.

Further Reading

- *The First 90 Days* — Watkins (the classic, secularised for analytics)
- *The Trusted Advisor* — Maister, Green and Galford (the ledger's theory)
- Chapter 73 (the ladder and the tour), Chapter 72 (the refusal your balance buys), Chapter 79 (what the ninety days establish)

Chapter 79: Building a Practice — The Service Catalogue

Part XII — The Practitioner's Path: Projects, Clients and the Craft of Delivery

"A job is work you repeat; a practice is work that repeats itself, better each time."

In this chapter you will learn

- The difference between doing projects and running a practice.
- The service catalogue: defined offers, each with scope, artefacts, and price.
- Repeatability as strategy: templates, checklists, and the asset library.
- The practice dashboard: pipeline, delivery, reuse — measured like anything else.
- Growing the team: juniors, apprenticeship, and the work only seniors should do.
- Failure modes: the bespoke trap and the practice that never systematises.

79.1 From Projects to Practice

The project-by-project practitioner reinvents everything quarterly: the scoping conversation from scratch, the pipeline rebuilt per client, the memo format re-decided, the pricing re-guessed. The practice does the opposite — it **systematises what repeats and reserves creativity for what does not**. The instrument of systematisation is the service catalogue: the menu of things the practice does, defined well enough to price, staff, and improve.

The catalogue serves three audiences at once: **clients** (who see a menu of understood offers rather than a mystique of bespoke genius), **the practitioner** (who can scope, staff, and quote from standing definitions), and **the future** (the practice that can be sold, handed over, or scaled — Chapter 80's exit options — because its value is not solely in one head).

79.2 The Catalogue

A worked example — the eight services of a small analytics practice (the internal version, for a team inside an organisation, is the same list with internal prices and internal sponsors):

Service	Scope	Core artefacts	Price shape
Data health audit	2 weeks	Ten-question findings report (Ch. 76)	Fixed
Metric definition & warehouse layer	3–6 weeks	Contract, models, tests (Ch. 69)	Fixed + retainer
Dashboard build	2–4 weeks	One-audience dashboard + handover (Ch. 18–19)	Fixed

Service	Scope	Core artefacts	Price shape
Churn / risk model	4–8 weeks	Model, card, monitoring plan (Ch. 22–25, 43)	Fixed + monitoring retainer
Experiment design & analysis	Per test	Design doc, power calc, read-out memo (Ch. 67)	Fixed per test
Programme evaluation	4–12 weeks	Evaluation design, bounded estimate, two-audience memo (Ch. 61, App. J)	Fixed
Training & upskilling	Per cohort	Course from this book, assessed (Ch. 50)	Per head
Analytics on retainer	Monthly	The question ladder, standing (Ch. 73)	Monthly retainer

Each service carries a one-page definition: **what it is, what it is not, what the client provides, what the artefacts are, what it costs, what it assumes**. The "what it is not" column is the catalogue's spine — it is Chapter 72's refusal clause, embedded in the menu, so the scope conversation starts from boundaries rather than discovering them in week three.

79.3 Repeatability as Strategy

Behind every catalogue line, the asset library — the practice's compounding capital:

- **Templates** — the analysis plan, the options paper, the executive memo, the findings report, the handover memo (Chapters 71–75): each a standing document with the craft already in it.
- **Checklists** — the leakage checklist, the audit's ten questions, the ship-week rubric: the practice's judgement, externalised into artefacts that survive the practitioner's bad week.
- **Pipelines** — the ingestion/validation/transform code (Chapter 69's layer cake) that runs, tested, on the next client's data with configuration rather than rewriting.
- **The case library** — anonymised past engagements, each one a scoping shortcut ("like the co-operative evaluation, but with better records").

The strategic arithmetic: a practice that reuses 60% of its assets on each engagement can quote lower, deliver faster, and price its *novelty* honestly — and every engagement adds assets back. The bespoke practitioner charges for reinventing the wheel; the practice charges for the car.

79.4 The Practice Dashboard

The practice measures itself with its own methods (the dashboard eats itself, happily):

- **Pipeline** — enquiries, scoping conversations, options papers out, signed briefs; conversion by service (Chapter 52's funnel, applied to yourself).
- **Delivery** — on-time rate by service, gate slippage (Chapter 71's gates), the review findings that recur (the template's teaching moments).

- **Reuse** — asset-library hit rate per engagement: what percentage of the delivery was standing assets versus new build; the practice's compounding gauge.
- **The value ledger feed** — every engagement's measured outcome (Chapter 80): the dashboard's most important column and the practice's only defensible price list.

From Your Toolkit — Power BI: the practice dashboard is the eat-your-own-cooking artefact — cohort curves on client retention, a decomposition tree on revenue by service, the p90 delivery week rather than the mean. Clients who see their consultant's own dashboard trust the craft faster; it is the portfolio that runs.

79.5 Growing the Team

The practice's scaling question is who does what, and the answer is apprenticeship-shaped:

- **Juniors do the auditable** — pipelines, validation gates, dashboard builds: work with templates, checklists, and reviews; the apprentice learns the craft where errors are caught (Chapter 50's feedback slot, professional edition).
- **Mid-level does the repeatable complex** — the catalogue's core services, delivered solo, with the senior reviewing the gates.
- **Seniors do the two irreplaceables** — the scoping conversation (the ladder, the refusal, the pricing judgement) and the hardest analysis calls (the sensitivity judgement, the two-audience memo): the work where the experience is the asset, and the only work that cannot be templated yet.

The failure to avoid: the practice where the senior does everything (no scale, no succession) and its mirror, the practice where juniors do everything (the brand hollowed out quietly). The catalogue is the answer to both — each service names its staffing shape, and the shape is policy.

79.6 Failure Modes

- **The bespoke trap** — every engagement heroic, nothing reused, the practice's revenue capped at the practitioner's calendar; the catalogue is the exit.
- **The never-systematised practice** — busy for years, no assets, unsellable, unscalable; the reuse gauge reads zero and the retirement plan reads "keep working".
- **Catalogue rigidity** — the menu ossified in 2023's methods; Chapter 70's reading discipline is the catalogue's maintenance contract (each service re-priced and re-scoped annually against the grammar's changes).
- **The services nobody buys** — lines kept out of pride; the pipeline data retires them (and the post-mortem — too expensive? badly sold? genuinely unwanted? — feeds Chapter 80's pricing).

Teaching Tip — Write the catalogue for your job: the assignment is one page: your role's eight services, each with its "is not" column and its internal price (hours, or political capital). Students discover their role was always a practice — they were just running it bespoke — and the two services they should retire (nobody values them) and the one they should productise (everybody asks for it) are visible for the first time.

Key Takeaways

- A practice systematises what repeats: the service catalogue is the menu, the "is not" column is its spine.
- Behind the catalogue, the asset library compounds: templates, checklists, pipelines, cases — reuse is the margin.
- The practice measures itself: pipeline, on-time, reuse, and the value-ledger feed.
- Staffing is policy: juniors on the auditable, mids on the repeatable, seniors on the scoping and the judgement calls.
- Re-price the catalogue annually against the grammar's changes — rigidity is the catalogue's death.

Practice Lab

1. Write your role's service catalogue: eight lines, each with scope, "is not", artefacts, price shape, and staffing — then mark the two to retire and the one to productise.
2. Start the asset library: list every template, checklist, and pipeline you have built in the last year; score each for reuse potential and pick the three to make standing.
3. Build the practice dashboard (even with one person and three engagements): pipeline, delivery on-time, reuse rate, and the value-ledger column; review it monthly.
4. The staffing plan: for each catalogue service, the junior/mid/senior shape and the review gate; write the apprenticeship path for the junior you will hire or mentor.
5. The pricing review: take one service and re-price it against the last three deliveries' actual effort and the value ledger's outcomes; write the paragraph that justifies the new price to a client.
6. The catalogue audit: score each line against Chapter 70's triage — which service implements changed grammar? Draft next year's version.

Further Reading

- *Managing the Professional Services Firm* — Maister (the practice economics)
- *The Firm of the Future* — the productisation argument
- Chapter 50 (teaching as the apprenticeship's engine), Chapter 72 (pricing the catalogue's lines), Chapter 80 (the value ledger that prices them)

Chapter 80: The Business of the Craft — The Value Ledger, Productisation, and Compounding Practice

Part XII — The Practitioner's Path: Projects, Clients and the Craft of Delivery

"Analytics is a craft that behaves like capital: it compounds if you keep the books."

In this chapter you will learn

- The value ledger: recording every engagement's measured outcome, and pricing from it.
- Value-based pricing in practice: the arithmetic that makes it honest.
- Productisation: turning the repeated engagement into an asset that earns while you sleep.
- The refusal clause, revisited as strategy: the work you decline defines the work you get.
- The compounding practice: datasets, templates, reputation, and teaching as capital.
- The book's closing argument: the grammar, the domains, the frontier, the delivery — and what compounds.

80.1 The Value Ledger

Part XII's instrument, and the practice's accounting system: **one row per engagement, recording what changed because the work happened** — the decision, the measured outcome, the confidence interval, the date it was measured, and the attribution honesty (what else moved that you must not claim).

D a t e	Engagement	Decision changed	Measured outcome	Attribution	Value (ann ualised)
M a r	Churn retention build	Who gets called weekly	Churn among called: -1.8pts, CI [-3.1,-0.5]	Holdout, 10%	\$46,000
M a y	Markdown ledger	Dead-stock clearance policy	Working capital released	Stock ledger delta	\$31,000
Ju l	Queueing memo	Second till at peak	Walk-outs, p90 Saturday	Before/after, matched	\$5,900/yr
Se p	Tariff evaluation	Lifeline redesign deferred	9-11% deficit avoided	DiD bounds	political

The ledger's disciplines: **measured, not claimed** (the measurement plan from Chapter 71 is the row's birth certificate); **attributed honestly** (Chapter 61's counterfactual discipline applied to your own invoice — the same quarter's economy

does not get booked as your win); and **interval-ised** (a value carried as a range survives the audit that a point estimate dies in, Chapter 76's question six).

The ledger prices the future: it is the evidence base for value-based pricing (Section 80.2), the portfolio that wins the next engagement ("here is what changed the last four times"), and the only honest answer to the question every practitioner must face — *what, exactly, have you changed?*

80.2 Value-Based Pricing, Honestly

Chapter 72's boldest pricing model, now with its arithmetic: price the engagement as a share of the *expected, measured* value from the ledger's comparable rows, with the measurement plan written before the work.

- **The comparable** — the ledger's rows for this kind of decision: retention builds of this type returned \$30k-60k annualised.
- **The share** — 10-20% of the expected value's lower bound, priced against the delivery cost as the floor.
- **The honesty clauses** — the client sees the ledger's comparable rows (transparent basis), the measurement plan is in the contract (both parties price the same truth), and the attribution rule is pre-agreed (what gets claimed, what does not).

Value pricing fails without the ledger — it becomes the consultant's confidence wearing a formula. With the ledger, it is the rare pricing conversation both parties leave feeling well served: the client buys a measured expected change, not hours; the practitioner is paid from outcomes they can evidence, not effort they can inflate.

80.3 Productisation

The repeated engagement becomes an asset (Chapter 79's catalogue, matured): the data-health audit that runs on a questionnaire and a pipeline; the dashboard that configures in a day; the training that is this book, delivered. Productisation's tests: does the service repeat with *configuration, not rewriting*? Does it survive delivery by someone other than its author? Does its quality *rise* with each delivery (the template learns), rather than erode?

The product ladder for the practitioner: **service → templated service → product** — each step trades some of the engagement's margin for scale, reach, and sleep. The honest stopping point varies: some practices productise everything but the scoping conversation; some keep the two irreplaceables (Chapter 79) bespoke forever. The strategic point is that the choice is now deliberate — productisation is a design decision, not a drift.

80.4 The Refusal Clause as Strategy

Chapter 72's refusals, seen from the far end of the practice: the work you decline is not a cost, it is the **positioning**. The practice known to refuse the decision-less exploration, the predetermined answer, and the harmful deployment becomes the practice trusted with the decisions that matter — because everyone can see what its credibility was priced at. The refusal's compounding: each declined engagement polishes the credential that wins the next good one, and the ledger's rows (all measured, all attributed) are the visible proof that the practice's interests are the client's outcomes.

80.5 The Compounding Practice

What actually compounds in this craft — the practitioner's balance sheet, four asset classes:

- **Datasets and eval sets** — the corpora you built (Chapter 65's moat), the benchmark collections, the planted generators (Appendix F): assets that appreciate as they are used and cannot be bought, only built.
- **Templates and pipelines** — Chapter 79's library: the practice's plant and equipment, self-built, appreciating with every engagement.
- **Reputation** — the trust ledger (Chapter 78) at organisational scale: the refusal clause, the no-surprises rule, the public credit — each a deposit in the only account that referrals clear through.
- **Teaching** — Chapter 50's compounding: every analyst trained multiplies the craft's honesty and sends the practice both work and successors. The course you teach is the practice's R&D, recruitment, and marketing in one artefact.

The compounding's arithmetic is the book's quiet thesis: a practitioner who keeps the books — the value ledger, the trust ledger, the asset library, the teaching log — ends each year with more than a salary; they end it with a practice, whose returns arrive while they sleep.

80.6 The Closing Argument

Twelve parts, eighty chapters. The grammar (Parts I-VIII): questions before data, grain before joins, baselines before models, intervals before points, the counterfactual before the claim, the artefact before the memory. The domains (Part X): one grammar, ten industries, every one with its canvas and its failure modes. The frontier (Part XI): causality, space, time, language, simulation, privacy, and the discipline of entering rooms that are still being built. The delivery (Part XII): the gates, the ladders, the ledgers — the craft as a service that changes decisions and can prove it.

And underneath it all, the six tools you started with — SQL, Excel, Python, Power BI, Stata, SPSS — which were never six subjects, but one grammar that every chapter of this book has been speaking. The tools will churn (Chapter 70 has the predictions); the grammar will not; and the practitioner who owns the grammar, keeps the books, and teaches the craft has nothing to fear from the churn and everything to gain from

it.

The data is already pouring in somewhere near you. Go and meet it with everything you now carry.

— *Tatenda Makuvaza, "The Big Data Analyst", Harare, 2026*

Key Takeaways

- The value ledger is the practice's accounting: measured, honestly attributed, interval-ised — one row per changed decision.
- Value-based pricing is only honest on ledger evidence: comparables, share of the lower bound, measurement plan in the contract.
- Productise deliberately: service → templated service → product, each step a design decision about scale and margin.
- The refusal clause is positioning: the work you decline prices the credibility you sell.
- Four assets compound — datasets, templates, reputation, teaching — and their books are the ledgers of this part.

Practice Lab

1. Start the value ledger: backfill every engagement you can (job, course, consulting) with its decision, its measured outcome, its attribution honesty — and mark the rows that are claims rather than measurements (they are next quarter's measurement plans).
2. Price one real engagement value-based: the comparable rows, the expected value's lower bound, your share, and the two honesty clauses as they would appear in the contract.
3. The productisation audit: pick your most-repeated service and apply the three tests (configuration, survivable authorship, learning template); write the ladder's next step or the deliberate stop.
4. Write your refusal clause's annual review: the engagements declined this year, the credential each polished, and the work each refusal made room for.
5. The balance sheet: list your four asset classes' current holdings (datasets, templates, reputation events, teaching hours) and write next year's one-line investment plan for each.
6. The five-year letter: write to yourself, five years ahead, describing the practice you are compounding toward — its catalogue, its ledger's best rows, and the refusal it is known for. Date it, keep it with the ledgers.

Further Reading

- *The Strategy and Tactics of Pricing* — Nagle and Müller (the value arithmetic)
- *Company of One* — Jarvis (the deliberate-scale argument)

- The whole of Part XII, which this chapter keeps the books for — and Chapter 50, where the compounding starts with teaching.

APPENDICES

Reference Section

Quick answers, cheat sheets, solutions and practice data

Appendix E: Datasets and Resources

Practice data, public datasets, and the reading list behind the course

"Data is the course's tuition: every dataset below is chosen because it teaches something specific."

How to Use This Appendix

Every chapter's Practice Lab assumes data. Three sources cover the whole course: **Appendix F's Soulfya's generator** (fabricates the running case's entire universe with planted patterns), the public datasets listed here, and your own organisation's extracts (used under Chapter 42's governance and Chapter 68's privacy floor). Work the labs in that order: generated data first (you know what should be found), public data second (you practise finding it), real data last (nothing teaches governance like real data).

When you download a public dataset, practise the Part II discipline from the first minute: write the data card (source, licence, grain, known gaps), load it through your validation gates, and only then begin analysis. The habit is the point — a dataset you have not profiled is a stranger you are about to marry.

Core Public Datasets, by Part

Dataset	Where	Use it for	Part
Soulfya's generated suite	Appendix F script	Every running case lab	I–XII
Titanic passenger list	Public (Kaggle, many mirrors)	First classification, missing data, leakage-safe practice	V
Ames / Boston housing	Public (scikit-learn, Kaggle)	Regression, feature engineering, honest evaluation	III, V
NYC taxi trips	NYC Open Data	Big-ish data, DuckDB/Spark practice, fares and geography	VII
Bikeshare trips (Capital Bikeshare or local)	Open data portals	Time series, seasonality, forecasting leagues	III, V
Online Retail II (UCI)	UCI ML Repository	Market basket, RFM, CLV — the Part X retail labs	IV, X
Telco customer churn	Public (IBM sample, Kaggle mirrors)	Churn baselines, calibration, uplift practice	V, X
Bank marketing (UCI)	UCI ML Repository	Uplift and campaign response, imbalanced targets	V, X
Olist Brazilian e-commerce	Kaggle	Multi-table joins, star schema, delivery ETAs	II, VII, X
Superstore sample	Tableau Public	Dashboard drills, metric trees	IV
Gapminder / World Bank indicators	gapminder.org, data.worldbank.org	Trend craft, small multiples, development analytics	III, IV, X

Dataset	Where	Use it for	Part
DHS / MICS survey data (aggregated)	dhsprogram.com, unicef.org/mics	Survey weights, stratified estimation	III, X
CRAN / UCI time series	UCI, Monash TS archive	Forecasting league tables	V
Medical cost / diabetes readmission (UCI)	UCI ML Repository	Clinical risk, calibration	X
Palmer penguins	Public (allisonhorst)	First EDA, tidy graphics	I, IV
Open FIFA / football events	Public (Kaggle)	Expected goals, sport analytics	X

Licence note: check each source's licence before publishing derived work; academic-teaching use is broadly permitted for most of the above, but "public" is not "yours to redistribute" — the data card records the licence with everything else.

Tools (Free), by Purpose

- **Python stack:** pandas, scikit-learn, statsmodels, matplotlib, seaborn, statsforecast or sktime for leagues; DuckDB for analytical SQL on files.
- **Big data practice:** DuckDB first (single-node is the honest default, Chapter 37); Spark locally via pyspark when the decision test says so; Dask rarely and deliberately.
- **Dashboards:** Power BI Desktop (free), or Plotly Dash / Streamlit (Chapter 49) for products.
- **Causal inference:** statsmodels, causalimpact, dowhy, econml (Part XI's frontier reading).
- **Survey analysis:** Stata or R with survey; Python's statsmodels with weights for the same discipline.
- **Geospatial:** geopandas, libpysal (Moran's I and friends, Chapter 62).
- **Survival:** lifelines (Chapter 63).
- **Experimentation:** scipy.stats, statsmodels sequential tests, and Chapter 67's simulation notebooks.
- **Text:** sentence-transformers, sentencepiece (Chapter 65's low-resource work).

Domain Playbook Addendum (Part X)

Each playbook chapter pairs with the datasets above and one domain-specific practice source. The pattern: run the chapter's Practice Lab twice — once on the Soulfya's-generated analogue (where Appendix F plants the answers), once on the public source (where the world decides).

- **Retail (Ch. 51)** — Online Retail II for basket and RFM; Olist for the multi-table version; the dead-stock ledger lab runs on Appendix F's inventory snapshot.
- **Marketing (Ch. 52)** — Telco churn and Bank marketing for the uplift and cohort labs; any app-store or SaaS sample for payback-week practice.
- **Financial services (Ch. 53)** — The UCI credit-approval style samples and Give Me Some Credit (Kaggle) for scorecard practice; generate fraud with Appendix F's

planted-adversary mode.

- **Healthcare (Ch. 54)** — Diabetes 130-US readmission (UCI) is the canonical readmission lab; any DHS/MICS extract for the survey-weight discipline.
- **Agriculture (Ch. 55)** — FAOSTAT for production series; CHIRPS rainfall (public) and MODIS NDVI for the layered forecast; Sable Fields comes from Appendix F.
- **Energy (Ch. 56)** — UMass/ISO-NE or ENTSO-E public load data for the day-ahead forecast league; smart-meter samples (UK-DALE, Pecan Street) for interval analytics.
- **Government (Ch. 57)** — Your own city's open-data portal (Harare's and most capitals publish something); World Bank indicators for the evaluation labs; Case 4's tariff data is generated.
- **Manufacturing (Ch. 58)** — The SECOM dataset (UCI) for SPC-style quality; Appendix F's line counters for OEE and the maintenance hazard.
- **Transport (Ch. 59)** — NYC taxi for routing-adjacent ETA practice; any GTFS feed for schedule reality; Kunaka's telematics is generated (Case 3's numbers).
- **Media and sport (Ch. 60)** — Public football event data (Kaggle's European matches) for xG; any podcast/streaming sample for decay curves; Kumba's corpora are built, not downloaded (Chapter 65).

Frontier and Practitioner Resources (Parts XI-XII)

- **Causal inference (Ch. 61)** — Scott Cunningham's *Causal Inference: The Mixtape* (free online); the World Bank's DiD resources; *causalimpact* documentation.
- **Geospatial (Ch. 62)** — *geopandas* gallery; the Geographic Data Science textbook (free); *libpysal*'s spatial-statistics tutorials.
- **Survival and forecasting depth (Ch. 63-64)** — *lifelines* documentation (an excellent tutorial in itself); FPP3 (*Forecasting: Principles and Practice*, free online) for hierarchical methods; *sktime* for the leagues.
- **Low-resource NLP (Ch. 65)** — Masakhane's published work (the reference for African-language NLP); *sentencepiece* papers; Hugging Face's course.
- **Simulation (Ch. 66-67)** — Julia/Python queueing tutorials; the Monte Carlo kitchen notebooks from Chapter 67; any discrete-event library's manual (*SimPy*) is a course in itself.
- **Experimentation (Ch. 67)** — Kohavi, Tang and Xu, *Trustworthy Online Controlled Experiments* — the practitioner's bible; CUPED references from Microsoft's published papers.
- **Privacy (Ch. 68)** — The Differential Privacy textbook (Dwork and Roth, free online); OpenDP's documentation.
- **Analytics engineering (Ch. 69)** — *dbt*'s documentation and blog; Kimball's dimensional toolkit.
- **Staying current (Ch. 70)** — The newsletters and reading discipline of Chapter 70; Papers With Code for method tracking.
- **Delivery craft (Ch. 71-80)** — Appendix J's five cases as the role-play corpus; *The Flaw of Averages* (Savage) for the constraint of the week; the options-paper and

value-ledger templates from Part XII, used on your own engagements.

Building Your Own Practice Data

When no public dataset fits (often, for Parts VII-XII), generate it: the discipline is Appendix F's — write down every planted pattern before generating, so the analysis can be *scored against truth*, which no real dataset offers. A generated dataset with documented plants is a better teacher than a real dataset with unknown structure; the sequence is always generated → public → real, in that order, for every new skill.

Appendix F: The Soulfya's Practice-Dataset Generator

The running case's entire universe, fabricated with documented, planted patterns

"A generated dataset with documented plants is a better teacher than a real dataset with unknown structure: you can be scored against the truth."

F.1 Why Generate

Every practice lab in this edition assumes data from Soulfya's Restaurant Group and its neighbouring companies. Rather than ship opaque CSVs, this appendix ships the **generator**: one script that fabricates the data with every pattern **written down before it is planted**. Working the labs on generated data means you always have an answer key — the analysis is scored against the truth, which no real dataset offers. The sequence from Appendix E holds: generated first, public second, real last.

F.2 The Planted Patterns

Every pattern the book teaches is planted, dated, and documented here. If your analysis finds them, it is working; if it finds something *not* on this list, you have either discovered noise or a bug — both findings.

Pattern	Where it is planted	What should find it
Weekly seasonality	Visits peak Fri-Sun, ~1.4x weekday	Ch. 64's day-of-week features; Ch. 28's seasonal baselines
Payday spikes	0-2 days after month-end, +18% baskets	Ch. 56's payday feature; Ch. 66's arrival clumps
Outlet differences	Avondale +6% basket vs Bulawayo; Borrowdale delivery-heavy	Ch. 51's outlet layer; Ch. 62's maps
The promo lift (Case 1)	Double-points window: +9% true uplift among persuadables; +18 points selection bias; +4 regression to the mean	Ch. 61's DiD; Ch. 32's four cells
The churn signal	Members with rising <code>days_since_visit</code> and falling basket churn at higher hazard; early cliff, plateau, late sag	Ch. 63's survival curves: $S(15) \approx 0.389$
The planted leak (Case 2)	<code>loyalty_status_current</code> becomes "lapsed" 30 days after last visit — the target's definition	Ch. 76's question 3; Ch. 22's audit
Dead stock	~22% of SKUs: slow sell-through after week 12	Ch. 51's ledger
Saturday variance	Covers ~ $N(260, 60)$; walk-out risk past 240 covers	Ch. 67's Monte Carlo kitchen (mean $\approx \$302$, $p_{10} \approx \$170$)
Queue pressure	Avondale Saturday $\rho \approx 0.9$ on one till	Ch. 66's W_q arithmetic
Seasonal demand	Delivery orders rise in the May-July cool season	Ch. 62's hot spots; Ch. 57's service baseline

F.3 The Generator

One file, standard library plus NumPy and pandas. Every random choice is seeded; rerunning reproduces the universe exactly (Ch. 75's discipline, applied to fiction).

```

"""soulfyas_generator.py – fabricates the running case's core data."""
import numpy as np, pandas as pd
RNG = np.random.default_rng(20260101)
OUTLETS = ["AVD", "BOR", "BYO"]
OUTLET_EFFECT = {"AVD": 1.06, "BOR": 1.00, "BYO": 0.94}
def members(n=48000):
    ids = [f"M{ i:06d}" for i in range(1, n + 1)]
    joined = pd.to_datetime("2023-01-01") + pd.to_timedelta(
        RNG.integers(0, 700, n), unit="D")
    email = RNG.random(n) < 0.35 # Case 1's selection: email skews urban
    base_risk = RNG.beta(2, 6, n) # heterogeneous churn risk
    return pd.DataFrame(dict(member_id=ids, joined=joined, email=email,
                             base_risk=base_risk))
def visits(mem, start="2025-01-01", end="2026-03-31"):
    days = pd.date_range(start, end, freq="D")
    rows = []
    promo = (pd.Timestamp("2025-07-01"), pd.Timestamp("2025-08-15")) # Case 1
    for _, m in mem.iterrows():
        # a member's rhythm: weekly habit, decaying with risk
        rate = 1.6 * (1 - 0.6 * m.base_risk)
        lam = np.where(np.isin(days.dayofweek, [4, 5, 6]), rate * 1.4, rate)
        payday = np.clip(31 - days.day + 2, 0, 2)
        lam = lam * (1 + 0.18 * (payday > 0))
        if promo[0] <= pd.Timestamp(start) or True:
            in_promo = (days >= promo[0]) & (days <= promo[1])
            lift = { # the four cells, planted (Case 1's numbers)
                "persuadable": 1.09, "sure_thing": 1.00,
                "lost_cause": 1.00, "sleeping_dog": 0.93}
            cell = RNG.choice(list(lift), p=[.58, .22, .12, .08])
            lam = lam * np.where(in_promo, lift[cell], 1.0)
        n_visits = RNG.poisson(lam * 0.14).sum()
        if n_visits:
            pick = RNG.choice(len(days), n_visits)
            for d in pick:
                rows.append((m.member_id, days[d],
                             RNG.choice(OUTLETS, p=[.4, .35, .25])))
    v = pd.DataFrame(rows, columns=["member_id", "visit_date", "outlet_id"])
    v["basket"] = (19.0 * OUTLET_EFFECT_FULL(v.outlet_id)
                  * (1 + 0.18 * payday_flag(v.visit_date))
                  * RNG.lognormal(0, 0.16, len(v))).round(2)
    return v
def inventory(n_sku=800, weeks=26):
    sku = [f"SKU{ i:04d}" for i in range(1, n_sku + 1)]
    healthy = RNG.random(n_sku) < 0.78 # ~22% go dead (the plant)
    received = RNG.integers(40, 300, n_sku)
    sold = np.where(healthy, received * RNG.uniform(0.6, 1.0, n_sku),
                    received * RNG.uniform(0.05, 0.35, n_sku))
    return pd.DataFrame(dict(sku=sku, weeks_on_shelf=weeks,

```

```

        units_received=received,
        units_sold=sold.astype(int),
        units_on_hand=(received - sold).astype(int),
        retail_price=RNG.uniform(3, 42, n_sku).round(2)))

def saturday_margins(n_saturdays=52):
    covers = np.maximum(80, RNG.normal(260, 60, n_saturdays))
    basket = RNG.normal(19.0, 3.2, n_saturdays)
    walkout = RNG.binomial(np.maximum(covers - 240, 0).astype(int), 0.18)
    margin = (covers - walkout) * basket * 0.62 - 620
    return pd.DataFrame(dict(covers=covers, basket=basket,
                            walkouts=walkout, margin=margin.round(2)))

if __name__ == "__main__":
    mem = members(); v = visits(mem.sample(6000, random_state=1))
    mem.to_csv("soulfyas_members.csv", index=False)
    v.to_csv("soulfyas_visits.csv", index=False)
    inventory().to_csv("soulfyas_inventory.csv", index=False)
    saturday_margins().to_csv("soulfyas_saturdays.csv", index=False)
    print("Universe written: 4 CSV files, seed 20260101.")

```

(The listing is the compact core — members, visits with seasonality/payday/promo/churn structure, the dead-stock inventory, and the Saturday-margin engine. The full script adds the till timestamps with the $\rho = 0.9$ Saturday queue and the Case 2 leaky feature table, following the same recipe.)

F.4 The Domain Datasets, Briefly

The Part X playbooks' companies (Ndineka, Zuva Mobile, ZuvaPay, Batanai, Chikafu/NGA, Sable Energy, Harare Metro, Sabvura, Kunaka, Nhaka FC) each follow the same recipe — and the recipe is the point:

1. **Write the plants first** — a table like F.2 for each domain (the vintage curves' deterioration months, the 78%-missing year of Case 5, the elasticity of Case 4, the xG overperformers of Ch. 60).
2. **Generate with a seed**, documented at the top of the script.
3. **Score against the plants** — every lab's "answer key" is the plant table; publish it to instructors only (Ch. 50).

The compact core above gives you the pattern for all of them; Appendix J's cases carry their key numbers in the text ("The Numbers, for the instructor"), which are the plants for the five case datasets.

F.5 Working the Labs

- **Generate, then analyse blind:** have a colleague generate with an unknown seed shift, and analyse without the plant table — the closest experience to real data, with an answer key still possible.
- **Corrupt deliberately:** introduce nulls, duplicates, and unit errors (Ch. 11's vocabulary) into a copy, and work the cleaning labs.
- **Plant your own:** the capstone exercise — add one pattern to the generator, document it, and have a classmate's analysis graded against your documentation.

From Your Toolkit — Python: the generator is a Python discipline, but its outputs are deliberately boring CSVs — so the analysis labs run in your SQL engine, your spreadsheet, your Stata or SPSS, or your Python stack. The universe is generated once; the grammar is practised in every dialect.

Appendix I: Instructor Slide Kit

Twelve starter decks — one per part — for teaching the course

"Slides are promises about time: each one says 'this many minutes, this one idea'."

How to Use This Kit

Each deck below is a skeleton: slide titles, the one message per slide, and the teaching note that anchors it. Build them in PowerPoint, Google Slides, or (for the reproducible) Marp or Quarto — the structure survives any medium. The decks are deliberately short: 8–12 slides each, one idea per slide, so that the lecture's remaining minutes belong to the live demo and the lab, where learning actually happens.

General deck rules (Chapter 17 applies to slides too):

- One message per slide, stated as the slide's title — a claim, not a label ("Joins duplicate fact rows" beats "Joins").
- Evidence on the slide: one chart or one table, never both.
- Speak the takeaway; the slide carries only what words cannot — numbers, shapes, structure.
- End every deck with the lab brief: what students do in the next ninety minutes.

Deck 1 — Part I: Foundations of Big Data Analytics

1. **Title.** Course name, your name, the promise: from six tools to one discipline.
2. **Analytics is decisions, not data.** The question ladder: decision → question → data → analysis → decision again.
3. **The maturity ladder.** Descriptive → diagnostic → predictive → prescriptive; where the value and the difficulty both live.
4. **When is data big?** The Vs, honestly: most "big data" problems are big-mess problems.
5. **Data types and where they hide.** Structured, semi-structured, unstructured; JSON in the wild.
6. **The six-tool bridge.** SQL, Excel, Python, Power BI, Stata, SPSS — one grammar, six dialects.
7. **Soulfya's Restaurant Group.** Meet the running case: three outlets, one loyalty programme, a delivery channel, and every pattern in this course.
8. **Lab brief.** The data inventory.

Teaching note: the whole deck is one idea — questions before data. Everything else in Part I is vocabulary for that idea.

Deck 2 — Part II: Data Engineering for Analysts

1. **Title.** Part II — where trustworthy data comes from.
2. **Grain first.** The single question that prevents most analysis errors: what does one row mean?
3. **Joins.** Inner/left/anti — and the fan-out that multiplies revenue.
4. **Star schemas.** Facts and dimensions; why analysts build wide tables from narrow ones.
5. **Pipelines.** Extract, validate, load; the pipeline that runs is worth ten that are elegant.
6. **The pandas-to-SQL bridge.** Same grammar, different engine; push work to the data.
7. **Cleaning is contract negotiation.** Names, nulls, duplicates, units — each one a business question.
8. **Quality gates.** Validation as code; trust is a pipeline property.
9. **Lab brief.** Build Soulfya's star schema.

Teaching note: run the duplicated-orders demo live — the gasp when revenue triples is the whole part.

Deck 3 — Part III: Mathematics and Statistics

1. **Title.** Part III — how to argue with numbers.
2. **Descriptives lie honestly.** Mean/median/spread; Anscombe's lesson in four datasets.
3. **Probability by counting.** The joint table; Bayes as arithmetic on a grid.
4. **The logic of a test.** Hypothesis, statistic, p-value, decision — and what a p-value is not.
5. **Simulation over formula.** The coin-flip lab: watch false positives appear.
6. **Sampling.** Random, stratified, weighted; the exit poll that went wrong.
7. **Regression.** One line, three stories; slope as rate, residuals as information.
8. **Lab brief.** The Bayes table; the permutation test.

Teaching note: never teach the formula first. Count first, simulate second, formula third — the formula then arrives as a shortcut, not a mystery.

Deck 4 — Part IV: Visualization and BI

1. **Title.** Part IV — charts that change decisions.
2. **Form follows question.** The chart-choice matrix: comparison, distribution, composition, relationship, trend.
3. **Pre-attentive craft.** Colour, position, length; ink spent on the message.
4. **The metric tree.** Goals → drivers → metrics; dashboards as forests, not trees.
5. **Dashboard genres.** Operational, tactical, strategic — one audience, one question, one screen.

6. **Power BI as product.** The toolkit bridge: relationships, measures, bookmarks.
7. **Storytelling.** Context → conflict → resolution; the executive summary that survives the stranger test.
8. **Lab brief.** The one-page dashboard.

Teaching note: bring bad charts (there are repositories of them) and let students perform the audit — critique teaches faster than admiration.

Deck 5 — Part V: Machine Learning Foundations

1. **Title.** Part V — the modelling loop.
2. **Framing.** Classification, regression, ranking, clustering — which question is this?
3. **Data before models.** Split logic, target leakage — the leak that flatters and lies.
4. **Baselines.** Majority class, seasonal naive, last year: the cheapest honest competitor.
5. **Linear models.** The line, regularized; interpretability as a feature.
6. **Evaluation.** The metric matches the decision: precision/recall trade-offs, calibration, cost curves.
7. **Trees and ensembles.** Splits, forests, boosting; variance and bias made visible.
8. **Ethics from day one.** Fairness metrics, disparate impact, the model card.
9. **Lab brief.** The leaky-feature audit; the baseline league table.

Teaching note: the planted-leak dataset (Appendix F) is the part's centrepiece — let students discover the AUC of 0.99 and then take it apart.

Deck 6 — Part VI: Advanced Machine Learning

1. **Title.** Part VI — beyond the first model.
2. **Clustering.** Segmentation as hypothesis generation, not truth.
3. **Recommenders.** Collaborative filtering; the cold start and the popularity trap.
4. **NLP.** Text as features: bag-of-words to embeddings.
5. **Deep learning.** When depth earns its keep; data, compute, and the humility curve.
6. **LLMs.** Tokens, prompts, fine-tuning; the analyst's assistant, not the analyst.
7. **Explanation.** SHAP, permutation importance — explaining models to the people who live with them.
8. **Lab brief.** The segmentation and recommender double lab.

Teaching note: keep the deep-learning segment honest with the "when NOT to" slide — the discipline is the lesson.

Deck 7 — Part VII: Big Data Technologies and the Cloud

1. **Title.** Part VII — what actually changes at scale.

2. **The decision test.** Size, speed, structure, spend — most problems are single-node problems.
3. **DuckDB.** The analytical database on a laptop; push the work down.
4. **Spark.** Partitions, the shuffle, lazy execution — distributed thinking.
5. **Streaming.** Windows, watermarks; the dashboard that is always now.
6. **Cloud economics.** Storage, compute, egress; the bill as design feedback.
7. **Lab brief.** The Spark job on Soulfya's delivery events.

Teaching note: open with the same dataset in pandas and DuckDB and let the stopwatch make the argument for the part.

Deck 8 — Part VIII: From Insight to Impact

1. **Title.** Part VIII — running analytics responsibly, forever.
2. **Governance.** Stewardship, lineage, privacy; trust as infrastructure.
3. **MLOps.** The pipeline, the registry, the smoke test; reproducibility as a habit.
4. **Monitoring.** Drift, decay, the champion-challenger rhythm.
5. **Ethics under load.** Consent, harm, the refusal clause.
6. **Careers.** The craft ladder: analyst → senior → lead → principal; the T-shaped professional.
7. **Lab brief.** The model card and the monitoring plan.

Teaching note: this deck lands best after the capstone has broken something — governance reads differently mid-incident.

Deck 9 — Part IX: The Extended Curriculum

1. **Title.** Part IX — special topics and the craft of teaching.
2. **Anomaly detection.** Rules, distances, isolation; the two error costs of fraud.
3. **Optimization.** From prediction to prescription; the roster that writes itself.
4. **Graph analytics.** Entities, edges, communities; influence beyond the average.
5. **Data products.** The rung ladder: report → dashboard → app → API.
6. **Teaching.** Backwards design; the lecture-lab-feedback rhythm.
7. **Lab brief.** Ship one data product.

Teaching note: this part is where students become colleagues — teach it as a seminar, not a lecture.

Deck 10 — Part X: Analytics Across Industries — Domain Playbooks

1. **Title.** Part X — ten industries, one grammar.
2. **The playbook canvas.** Core question, unit of analysis, key metrics, data reality, first project, failure mode — six slots, every industry.
3. **Retail and marketing.** Basket, churn, CLV, uplift; the promotion that pays for itself.

4. **Financial services and health.** Credit risk, fraud, readmission; the regulated industries, where explanation is law.
5. **Agriculture and energy.** Yield, weather, load, tariffs; forecasting where the weather is a feature.
6. **Transport, manufacturing, government, media.** Routing, OEE, programme evaluation, attention; each playbook in six slots.
7. **The pattern across playbooks.** Same grammar, different data realities — the analyst's real skill.
8. **Lab brief.** Fill the canvas for a local organisation.

Teaching note: run this part as case-method seminars — Appendix J's five cases are the syllabus; the deck only frames the method.

Deck 11 — Part XI: The Frontier — Advanced Methods and Emerging Practice

1. **Title.** Part XI — where the working core ends.
2. **Causal inference.** DiD, IV, synthetic control; the question correlation cannot answer.
3. **Space and time.** Moran's I, survival curves, hierarchical forecasts; the structures between rows.
4. **Language and simulation.** Low-resource NLP, synthetic data, agent-based markets.
5. **Experimentation at scale.** Bandits, CUPED, sequential tests; the infrastructure of learning.
6. **Privacy engineering.** Differential privacy; sharing data safely.
7. **Staying current.** The reading discipline; ten predictions to track.
8. **Lab brief.** One frontier replication.

Teaching note: one chapter, one paper, one replication — depth over survey; the frontier changes, the discipline of entering it does not.

Deck 12 — Part XII: The Practitioner's Path — Projects, Clients and the Craft of Delivery

1. **Title.** Part XII — analytics as a delivered service.
2. **The project arc.** Scope, plan, analyse, deliver; the stranger test at every phase.
3. **Engagements.** The options paper; pricing and the refusal clause.
4. **Stakeholders.** The question ladder; the no-surprises rule.
5. **Storytelling and documentation.** Three structures; the four artefacts; the analysis log.
6. **Audits, constraint, onboarding.** Ten audit questions; the \$200 stack; the first ninety days; the trust ledger.
7. **The business of the craft.** The value ledger; productization; compounding practice.

8. **Lab brief.** The full engagement role-play.

Teaching note: the role-play is the assessment — hostile questions, scope creep, a pricing decision, and a refusal, all in one session.

Deck Hygiene Checklist

Before any deck ships:

- [] One message per slide, stated in the title
- [] Every chart follows Part IV's craft rules
- [] Code screenshots never exceed half a slide
- [] The lab brief is the final slide
- [] You have spoken the deck aloud once, with a timer

Twelve decks, one course, roughly 110 slides — about two semesters of teaching material when each deck anchors a lecture and the labs carry the rest. Adapt freely: the skeletons are yours. The course, like the book, ends where your own practice begins.

Appendix J: Extended Case Portfolio

Five end-to-end cases for seminar use, capstone inspiration, and interview preparation

"Cases are compressed experience: you live a project's hardest week in ninety minutes."

How to Use These Cases

Each case is written to be attacked, not admired. The recommended seminar format is the case method: students read the case cold (15 minutes), identify the decision and the decision-maker (10 minutes), work the analysis in small groups (30 minutes), then defend their recommendations against challenge (35 minutes). The teaching notes at the end of each case list the traps — the places where the obvious answer is the wrong one.

For interview preparation, use them differently: pick a case, work it alone under a timer, write the one-page memo, and then deliver the executive summary aloud. Chapter 74's structures and Chapter 73's question ladder apply to cases exactly as they apply to live engagements.

The five cases share a world — they can be run in sequence as a single course-long narrative, or independently. All data referenced is generated by Appendix F's Soulfya's generator or by the small generator scripts included in each case; all numbers reconcile across cases, so a cohort can build a shared "institutional memory" of the fictional companies.

Case 1: The Promotion That Ate the Margin

Soulfya's Restaurant Group — the Soul Circle double-points promotion

Domain: uplift modelling, causal reasoning, experiment design — Chapters 24, 32, 61, 62

The Situation

Azu, the founder of Soulfya's, is staring at a report she commissioned and half-regrets. Six months ago, the marketing team ran a double-points promotion through the Soul Circle loyalty programme: members who visited during the promotion window earned twice the usual points. The post-mortem report shows a triumph — visits among promoted members rose 31% versus the prior quarter, and the campaign was hailed internally as the best in the programme's history.

But Azu has two doubts. First, the finance director points out that the promotion's cost (points redeemed, discount processing, campaign administration) rose faster than revenue, and gross margin among promoted members fell 4.2 percentage points. Second, Azu has just read Chapter 61 of this book, and one line is stuck in her head: *the counterfactual is the customer who would have come anyway*.

The loyalty programme has 48,000 members. During the six-week promotion window, 11,200 members visited at least once and received the promotion. The remaining 36,800 members were not targeted — the campaign was sent by email, and only members with an email address on file were eligible. A junior analyst's quick check shows that email-holding members skew higher-income and more urban.

You have member-level data: visit logs for 52 weeks before and 6 weeks after the promotion, redemptions, satisfaction surveys, and a channel flag (email on file or not). You are hired to answer one question: **did the promotion create value, and should it run again?**

The Work

Your group must produce:

1. **A re-analysis of the 31% lift** — decomposed into what the promotion caused and what would have happened anyway. Email-eligibility selection bias must be addressed; a simple before-after comparison is not acceptable.
2. **An uplift segmentation** — following Chapter 32's four-cell logic: persuadables, sure things, lost causes, sleeping dogs. Estimate the proportion of the 11,200 in each cell (methods: matched controls from the non-email pool, or a model trained on the treated with matched untreated comparators; defend your choice).
3. **A margin-aware value calculation** — the promotion's true incremental profit, using the finance director's numbers: average margin among promoted members fell 4.2 points; points redemption cost averaged \$1.85 per redeeming member; campaign administration was \$9,000.
4. **A recommendation memo** — one page to Azu: run again, kill, or redesign; if redesign, the targeting rule (which cells to target), the control discipline (how the next run is measured), and the expected value of the redesign.

The Numbers (for the instructor)

The generator plants: a true treatment effect of +9% visits among persuadables (about 58% of the treated), zero effect among sure things (about 22%), and a small negative effect among sleeping dogs (about 8%, who deferred visits to earn points they then let lapse). The remaining 12% are lost causes. The 31% headline lift decomposes as: +9 percentage points true uplift among persuadables, +18 points of selection bias (email members were visiting more anyway and the pre-quarter was seasonally weak), +4 points of regression to the mean. The honest incremental profit is negative \$23,400 — the promotion lost money — but a redesigned campaign targeting only persuadables (identified by pre-period visit recency and frequency) is worth about \$31,000 per run with a 10% holdout for measurement.

Teaching Notes

The traps: (1) the before-after comparison, which every group tries first — let them, then ask what the non-email members did that quarter; (2) averaging uplift across cells — the sleeping dogs' deferral is invisible in the average; (3) forgetting the margin — revenue lift with margin collapse is a well-dressed loss; (4) recommending "run it again but bigger" — the case is designed so that bigger means worse. The strongest groups will ask for the holdout *before* being told it exists, and will design the next campaign as an experiment, not a broadcast.

Case 2: The Churn Model That Was Too Good

Soulfya's — the Soul Circle attrition model, second edition

Domain: leakage, evaluation honesty, model deployment — Chapters 22, 23, 25, 43

The Situation

Six months ago (see Chapter 22's framing), the first churn model went live: gradient boosting on behavioural features, AUC 0.83, used weekly by the retention team to target win-back offers. It has been a quiet success — churn among contacted members is down 1.8 points.

Now the data science team has built version 2. It is dramatically better: AUC 0.97 on the holdout. The team lead wants to swap it into production immediately; the retention manager is thrilled; Azu asks the question that gets you hired: "The model is nearly perfect. What is it seeing that we are not?"

You have: the v2 feature list (47 features), the training data dictionary, the holdout evaluation, and SHAP summary charts showing that three features dominate: `days_since_last_redemption`, `loyalty_status_current`, and `support_tickets_last_30d`.

The Work

1. **Audit the feature list for leakage** — for each of the 47 features, state the moment in time it becomes available, and whether that moment is before or after the prediction point (the model scores members on Monday to plan the week's outreach). The data dictionary shows `loyalty_status_current` is refreshed nightly, and a member's status becomes "lapsed" 30 days after their last visit — the same event that defines the churn target.
2. **Diagnose the AUC** — explain in plain language why the leak inflates the metric, and estimate the honest AUC by rebuilding the model with the three leaked features removed (provided: the team's rerun, AUC 0.79 — slightly *below* v1).
3. **The decision** — v2 is honestly worse than v1. Write the memo: what ships, what is communicated to the retention team, and what process change prevents v3 from repeating the error (Chapter 43's review gates).
4. **The harder question** — `support_tickets_last_30d` is *not* leaked (tickets precede the prediction point), but it is dominant. The retention manager asks: "Should we stop reading tickets and let the model do it?" Advise on the organisational risk of a model that quietly becomes the only thing anyone reads (Chapter 45's automation complacency).

Teaching Notes

Traps: (1) groups may accept the "honest" rerun as the fix and miss that v2-without-leaks is *worse than v1* — the decision is not "which model" but "why did we spend three months making a worse model", and the answer is evaluation without baselines; (2) the `days_since_last_redemption` feature is borderline — it is available at prediction time but correlates with the target's definition; a strong group distinguishes *timing* leaks from *definitional* leaks; (3) the automation-compliance question has no clean answer — reward groups that surface it rather than resolve it. The case's quiet lesson: a model that is too good is not lucky, it is lying.

Case 3: The van in the Wrong Place

Kunaka Logistics — cold-chain network redesign

Domain: optimisation, constraint analysis, simulation — Chapters 47, 67, 71

The Situation

Kunaka Logistics runs a 14-vehicle cold-chain distribution network for a dairy processor: one depot, 62 retail drop points, two vehicle classes (5 refrigerated vans, 9 ambient trucks that can carry insulated boxes), and a hard constraint — every drop must arrive within 4 hours of depot departure or the product fails quality control.

The operations manager has a spreadsheet optimisation (built by an intern) that recommends a new route plan projected to cut total distance by 18%. The board has approved the budget contingent on your independent review. The plan looks clean: routes are contiguous, distances check out, vehicle assignments respect capacity. But the drivers' union rep, invited to the review meeting "for completeness", says one sentence that stays with you: "Your plan has us loading the dairy before the pharmaceuticals."

The load sequence matters: pharmaceuticals must be loaded first (strictly controlled temperature ramp), and the plan's most efficient route has the pharma drop scheduled fourth — meaning 90 minutes of door openings before the pharma delivery, which the quality team says risks the temperature excursion limit.

You have: the 62 drop points with coordinates and demand, the current route plan, the intern's optimisation, drive-time matrices (with traffic by hour), the quality standard, and the union rep's account of loading practice.

The Work

1. **Reproduce the 18%** — verify the optimisation's arithmetic, then test its assumptions: the optimiser minimises distance, not time-under-constraint; rebuild with drive-time and the 4-hour constraint. How much of the 18% survives? (Provided generator: the honest saving is 11%, and two of the optimiser's routes violate the 4-hour window in afternoon traffic.)
2. **Model the loading constraint** — add the pharma-first rule; solve again (a simple heuristic — cluster then sequence — is acceptable and preferred to a black-box solver you cannot explain).

3. **Simulate before you sign** — Monte Carlo the recommended plan: demand $\pm 25\%$, drive times from the traffic model, one vehicle breakdown per week. What is the plan's service-level distribution? What does the p10 day look like? (Chapter 67's kitchen-sink method.)
4. **The recommendation** — one page to the board: what to approve, what to renegotiate (the intern's plan is not wrong, it is *incomplete*), and what to measure in the first month of operation.

Teaching Notes

Traps: (1) accepting distance as the objective — the case is engineered so that time-under-constraint reverses two route assignments; (2) the union rep is not an obstruction, he is the constraint's voice — groups that marginalise him miss the case's central lesson (Chapter 73: the person closest to the work knows the model's missing variable); (3) simulation is not optional — the deterministic plan's service level looks fine until the p10 day; (4) the intern's work should be *built on*, not discarded — Chapter 72's options-paper discipline: credit, correct, extend. The board does not need a new optimisation; it needs an honest one.

Case 4: The Anti-Model

Harare Metro Water — social tariff design

Domain: causal inference, ethics, policy — Chapters 45, 61, 63

The Situation

Harare Metro Water serves 1.2 million connections. A cross-subsidy tariff design is on the table: raise the volumetric rate on the top consumption decile to fund a lifeline tariff (first 5 kilolitres free) for the bottom two deciles. The engineering consultancy's model — a sophisticated microsimulation using 2019 consumption data — projects the policy is revenue-neutral and lifts 40,000 households above the water-poverty line.

The city council's data office (your client) has doubts about three things: the 2019 data predates two major tariff shocks that reshaped consumption; the model assumes no behavioural response to price changes (no elasticity); and the lifeline eligibility is based on registered household size, which audit samples suggest is under-reported by about 30% in exactly the households the policy targets.

Your brief is narrow and uncomfortable: **you have two weeks to say whether the model's projection can be trusted, not what the policy should be.**

The Work

1. **The data audit** — recompute the consumption distribution on post-shock data (provided: a 2023 sample of 40,000 connections). How much does the top decile's share move? (Planted: the top decile's consumption share falls from 34% to 29% — the subsidy base is smaller than projected, and the policy is no longer revenue-neutral; it runs a 6% deficit.)

2. **The elasticity bound** — the model assumes zero behavioural response. Use the two tariff shocks as natural experiments (difference-in-differences against a comparable metro's published data, provided) to estimate a price elasticity range for the top decile; run the policy under your range. (Planted: elasticity around -0.35; under it the deficit widens to 9-11%, because the subsidised decile conserves when priced up.)
3. **The eligibility problem** — model the consequence of 30% under-reporting: how much of the lifeline benefit leaks to households above the poverty line? What is the cost of verification (provided: a doorstep verification pilot cost of \$2.10 per household, and a 1.2 million-connection scale)? Is the policy still progressive after verification costs?
4. **The memo** — you were not asked for a recommendation, but Chapter 74's honest-practitioner rule says the analysis implies one. State what the model gets right, what it gets wrong, the bounded range of outcomes, and the decision the council actually faces (fund the deficit, verify eligibility, or redesign). Resist the temptation to decide the politics; do not resist the temptation to be useful.

Teaching Notes

Traps: (1) groups overstep the brief and become policy advocates — the teaching point is the boundary of the analyst's mandate (Chapter 72's scoping discipline), and the memo's final line should acknowledge it; (2) the elasticity estimate is genuinely uncertain — reward honest bounds over false precision, and expect a DiD table that students struggle to make clean (real natural experiments are messy; that is the lesson); (3) the under-reporting finding cuts *against* the policy's intended beneficiaries, which is uncomfortable — the case tests whether students report unfashionable findings; (4) "two weeks" is real: the deliverable is a bounded, honest memo, not a perfect study. The strongest close: "The model is trustworthy as arithmetic, and misleading as a projection; here is the projection range under defensible assumptions."

Case 5: The Grant That Would Not Close

Sable Fields Co-operative — programme evaluation under data loss

Domain: data engineering, missing data, survival analysis, integrity — Chapters 11, 15, 63, 75

The Situation

Sable Fields is a 2,400-farmer co-operative that took a four-year productivity grant. The funder's close-out report requires an impact evaluation: did grantee farmers' yields and incomes rise more than non-grantees'? The co-operative's manager delivers your data drop with an apology: "The tablets broke in year two. We have complete records for 2021, then nothing until mid-2023, then complete records again."

The dataset: 2,400 farmers (1,100 grantees, 1,300 non-grantees), annual yield and income records, plot sizes, and a participation log. The gap: year 2 (2022) is 78% missing, and — you discover in your first hour — the missingness is not random:

farmers with small plots were less likely to be re-enrolled in the record system after the tablet failure, and grantees with small plots were *more* likely to remain recorded (extension officers chased them for compliance photos). The co-operative's board needs the evaluation to succeed for the next grant; the funder's evaluator will pounce on any gap in the data story.

The Work

1. **Quantify the missingness** — build the missingness table: recorded vs unrecorded by grant status, plot size, and baseline yield. State the mechanism in Rubin's terms (Chapter 11's typology) and defend the classification: this is MAR conditioned on observables, not MNAR — the case is engineered so that the small-plot differential is fully explained by grant status and baseline yield.
2. **Choose and defend a strategy** — multiple imputation (chained equations) versus inverse-probability weighting versus complete-case with bounds. Run the evaluation under your primary strategy and at least one bound; Chapter 15's discipline applies: the sensitivity analysis is the result.
3. **The survival question** — the co-operative also lost farmers: 380 exited the co-operative during the grant period, and exit is correlated with poor yields (you observe exit dates for all 2,400). Naive comparisons of "remaining farmers" overstate impact. Model time-to-exit as a survival curve (Chapter 63), stratified by grant status, and reconcile: does the grant *reduce* exit, or delay it? (Planted: it delays it — the survival curves converge by year 4; the grant's honest effect on yields is +6.1% with CI [2.3, 9.8], and the effect on exit is a delay, not a prevention.)
4. **The close-out memo** — to two audiences at once: the funder's evaluator (technically literate, adversarial) and the co-operative's board (practical, anxious). One document, an executive summary that serves both, and an appendix that pre-empts the evaluator's ten questions (Chapter 76's audit list). Your integrity test: the memo must state the data gap in its first paragraph, not its last.

Teaching Notes

Traps: (1) complete-case analysis — which the case's construction makes deliver a +14% effect (the planted answer is +6.1%); the naive answer is more than twice the truth, and the board will love it; (2) imputation without the missingness table — the mechanism must be established before the strategy is chosen, or the imputation is decoration; (3) the survival analysis is optional-looking but load-bearing — the exit-delay finding changes the grant's story from "transformation" to "bridging", which is a defensible and fundable finding; (4) the two-audience memo is the hardest deliverable in the case portfolio — most groups write either a technical report or a pep talk, not both in one document. The integrity arc: the data loss is not the co-operative's shame, it is the evaluation's context — and honesty about it is what makes the +6.1% credible enough to fund the next grant.

The Portfolio's Arc

The five cases are one curriculum. Case 1 teaches that correlation dressed as a campaign report is still correlation. Case 2 teaches that models can lie with perfect accuracy. Case 3 teaches that optimisation without constraints is a plan for a world that does not exist. Case 4 teaches the boundaries of the analyst's mandate. Case 5 teaches that honesty about data's limits is the strongest thing an analyst can publish. Run in sequence, they are a capstone: a student who can work all five — the causal decomposition, the leakage audit, the constrained optimisation with simulation, the bounded policy memo, and the two-audience close-out with missing data — is ready for practice, and has the portfolio to prove it.

This was Appendix J — the final chapter/appendix of an eighty-chapter, twelve-part book. The course is complete: the grammar of the craft, the domains where it lives, the frontier that extends it, and the path of delivering it. What remains is practice — and the practice never ends, which is the good news. Go build the trustworthy pipeline, ship the honest model, write the memo that respects its reader, and teach someone else to do the same. The data is already pouring in somewhere near you.

— prepared for Tatenda Makuvaza, "*The Big Data Analyst*", 2026

About this Edition

Production notes

Big Data Analytics and Machine Learning — The Complete Practitioner's Course: From SQL, Excel and SPSS to Spark, Deep Learning and Production AI was typeset directly from its Markdown manuscript with `build_pdf.py` (ReportLab): 7 x 10 inch textbook trim, running headers, a linked table of contents, and PDF outline bookmarks.

- Edition: First Edition, 2026 (compiled 05 October 2026)
- Volume: the Extended Practitioner's Edition — Parts IX-XII of the two-volume course
- Contents: 32 chapters in 4 parts, plus 4 appendices
- Volume: approximately 53,544 words of prose and 325 lines of code (estimated 200 print pages)
- Toolchain: Markdown manuscript, `build_book.py` (HTML edition), `build_pdf.py` (this PDF)
- Typeface: DejaVu (serif body, sans display, mono code)

To rebuild either edition after editing the manuscript, run `python3 build_book.py` then `python3 build_pdf.py`. The companion HTML edition is screen-optimised and regenerates in seconds.

Thank you for reading this book — and for teaching from it. The data is already pouring in somewhere near you.